

Algorithms for Speech and Natural Language Processing

Speech processing 1, 2, 3

Introduction

Speech: the primary, most natural medium for communication

	Spoken Language	Written Language
Origin	Biological	Cultural
Universality	Universal in all human societies	Some societies lack writing systems
Developmental Timeline	Emerges spontaneously in childhood	Requires explicit instruction, acquired later
Neurological Basis	Specialized brain structures (Broca's, Wernicke's)	Recruits generalist visual and motor brain areas
Mode of Transmission	Transmitted naturally through social interaction	Requires formal instruction in schools

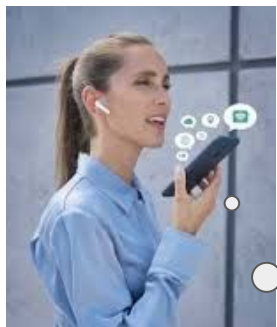


Introduction



Hey, find a French
restaurant nearby

Introduction



Hey, find a French restaurant nearby

The device will:

- **Identify your voice**
- Recognize what you say
- Understand what you mean
- Answer with a natural voice

Main user is speaking

Introduction



Hey, find a French restaurant nearby

The device will:

- Identify your voice
- **Recognize what you say**
- Understand what you mean
- Answer with a natural voice

user says "find a French restaurant nearby"

Introduction



Hey, find a French restaurant nearby

The device will:

- Identify your voice
- Recognize what you say
- **Understand what you mean**
- Answer with a natural voice

<API><search
<french_restaurant>,
<geoTAG></API>

Introduction



Hey, find a French restaurant nearby

Sure, 2 minute walk,
open now, 4 stars

The device will:

- Identify your voice
- Recognize what you say
- Understand what you mean
- **Answer with a natural voice**



The device will:

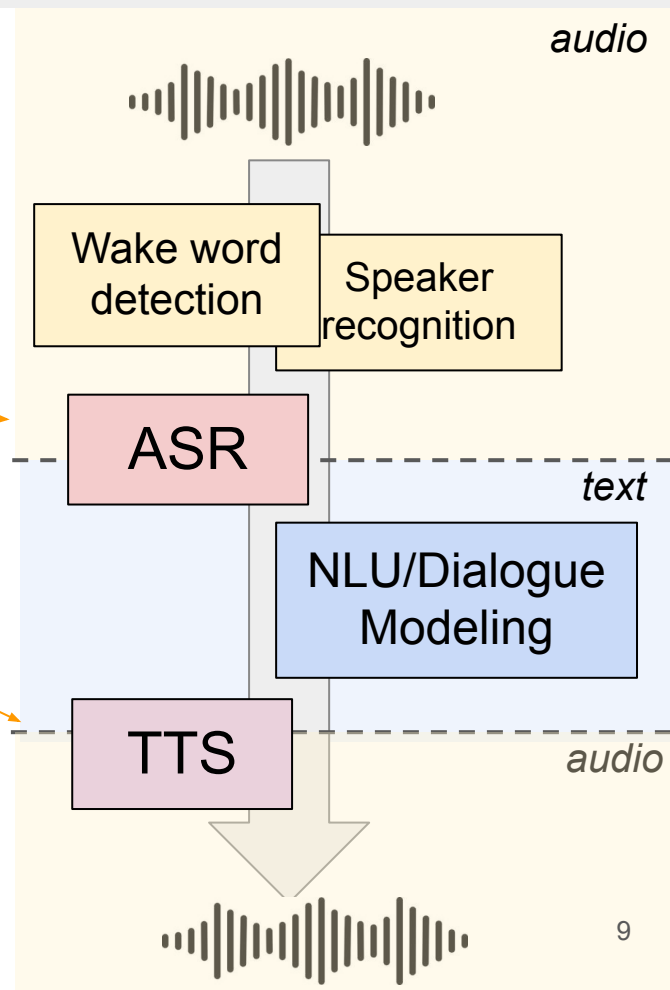
- Identify your voice
- Recognize what you say
- Understand what you mean
- Answer with a natural voice

Introduction

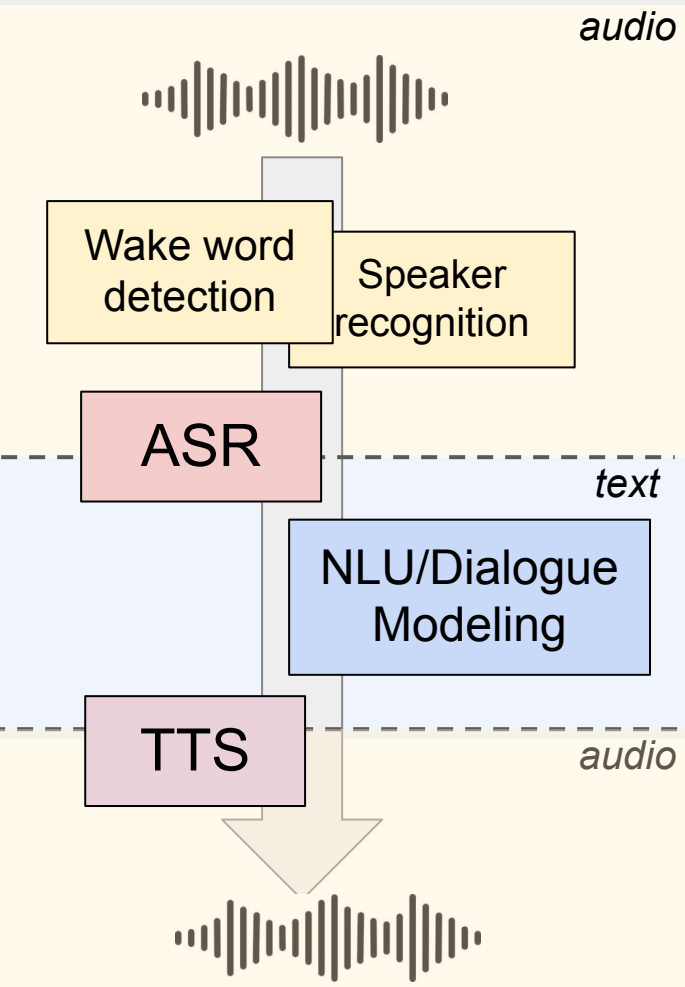


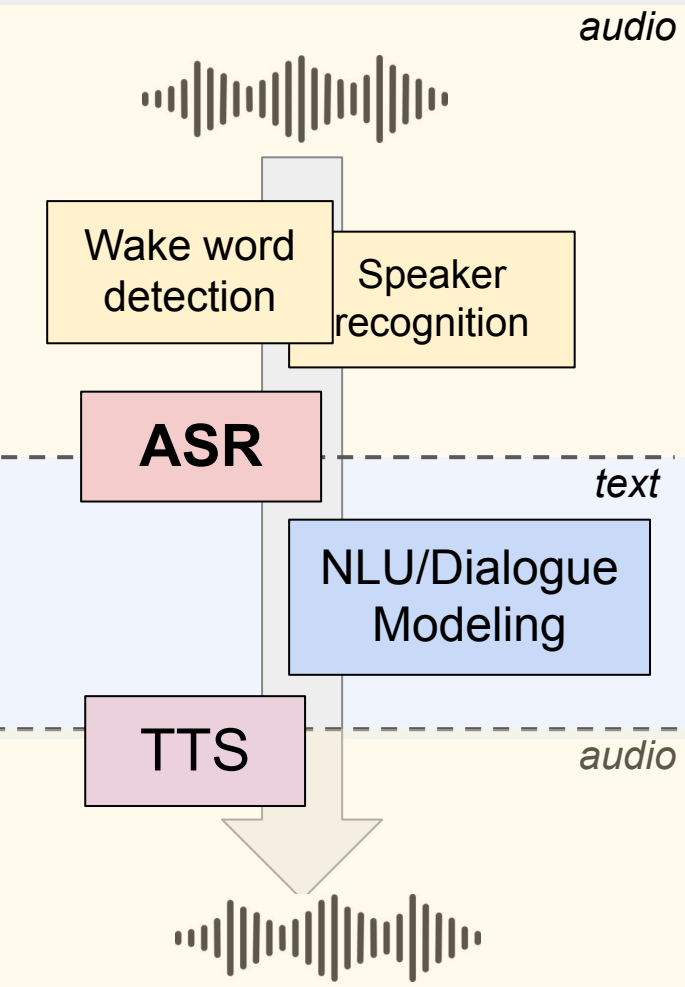
The device will:

- Identify your voice
- Recognize what you say
- Understand what you mean
- Answer with a natural voice



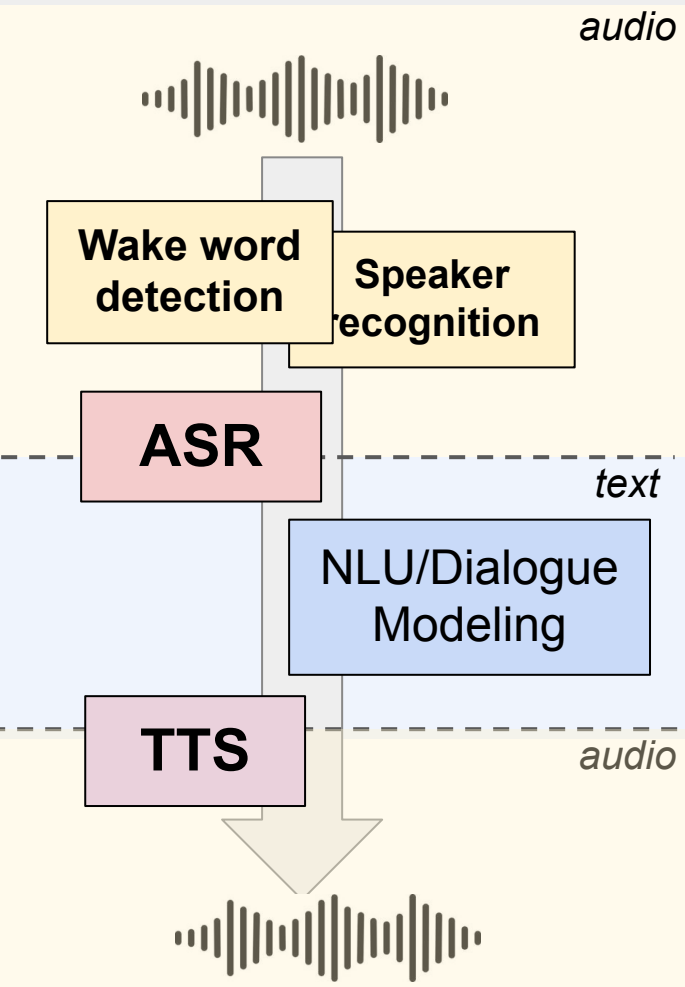
Introduction





Course 1: ASR basics

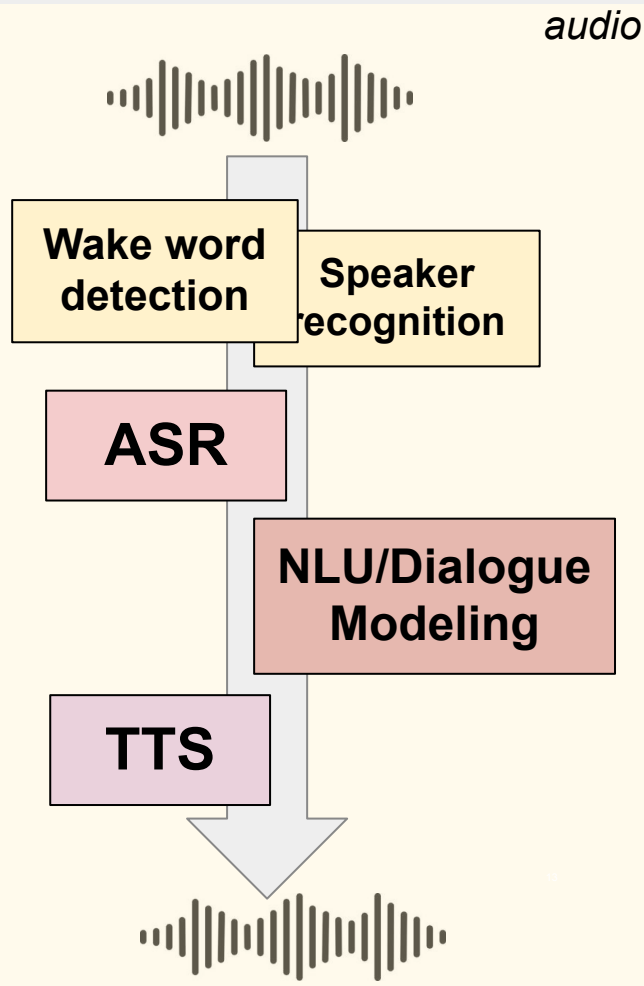
- Features
- Acoustic Models
- Word Models
- Language Models



Course 1: ASR basics

- Features
- Acoustic Models
- Word Models
- Language Models

Course 2: End-to-end Everything



Course 1: ASR basics

- Features
- Acoustic Models
- Word Models
- Language Models

Course 2: End-to-end Everything

Course 3: Audio Foundation Models



Course 1. ASR Basics

Course 1: ASR Basics

- Features
- Acoustic Models
- Word Models
- Language Models

Recommended Reading:

Jurafsky and Martin. Speech and Language Processing, Third Edition (Jan 6, 2026 release). Chapters 14-16, Appendix A

<https://web.stanford.edu/~jurafsky/slp3>

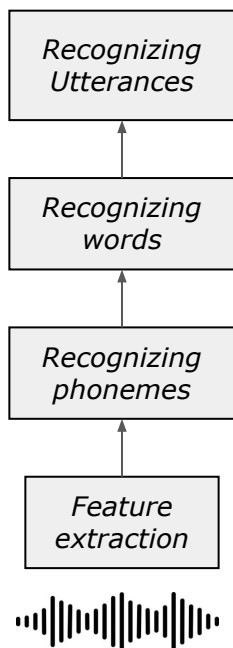
Computational approaches to speech processing over time

1950

1980

2012

Hand engineered system



1952: Audrey: single voice, single digits

1960: Shoebox: 16 words

1970: Harpy: 1000 words

Problems :

Labor intensive (procedures, hard coded heuristics)

Brittle (early errors catastrophic)



[Shoebox](#)

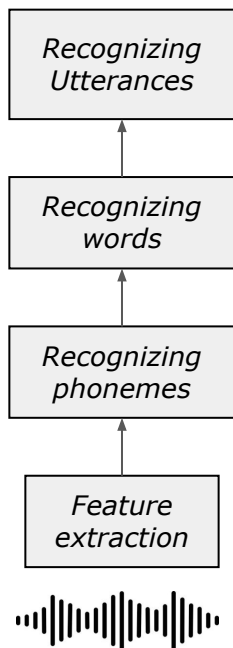
Computational approaches to speech processing over time

1950

1980

2012

Hand engineered system



Probabilistic system

Acoustic model

Language model

$$P(W|X) \sim P(X|W).P(W)$$

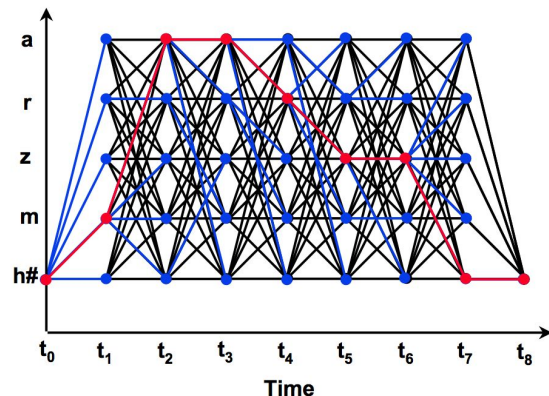
Decoding

$$\mathbf{W} = \operatorname{argmax}_{\mathbf{W}} P_{\theta}(X|W).P(W)$$

Learning

$$\theta = \operatorname{argmax}_{\theta} P_{\theta}(X)$$

Algorithms: State search space
Dynamic programming, EM



Advantage: robust to error

Problem: lots of annotated data

Computational approaches to speech processing over time

1950

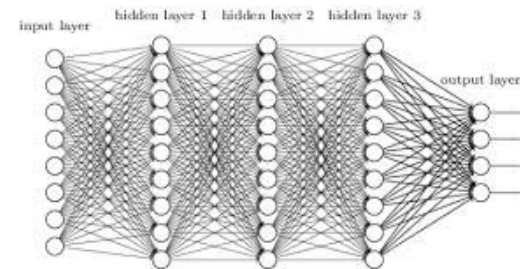
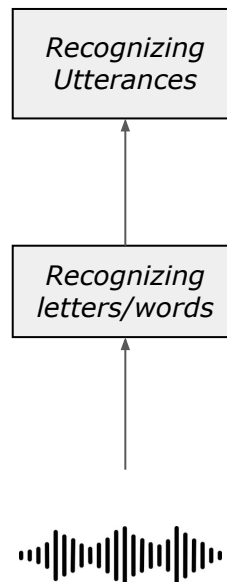
1980

2012

Hand engineered system

Probabilistic system

Deep learning systems



Decoding:

Distributed intermediate representations
Probabilistic interpretation of final layer

Learning:

Stochastic Gradient Descent

Advantage: more robust, scalable

Problem: less interpretable, needs even more
(less) annotated data

Computational approaches to speech processing over time

1950

1980

2012

2021

Hand engineered system

Probabilistic system

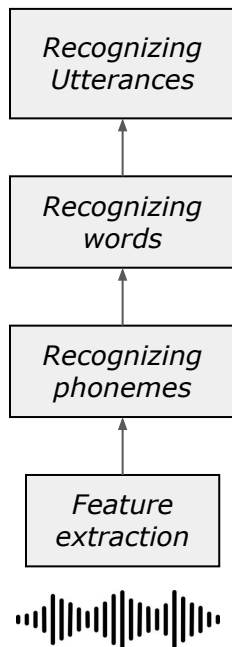
Deep learning systems

Audio foundation models

*Recognizing
Utterances*



Computational approaches to speech processing over time



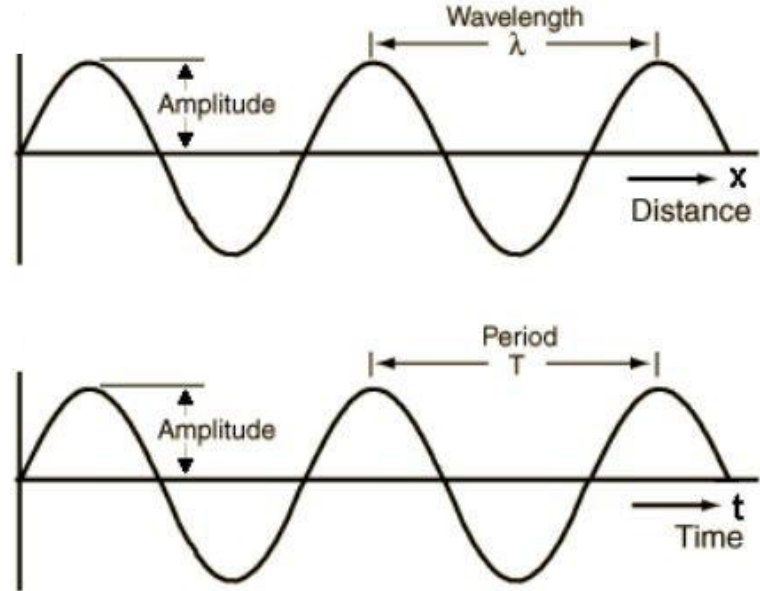


Chap 1. Speech and speech features

- The speech waveform
- Spectrogram
- Speech production
- Matching human perception

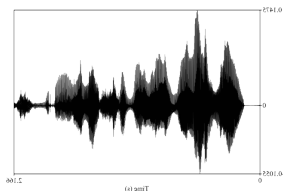
Some reminders on sound waves

- 3D wave
- The wave length is a distance
- The period is a duration
- Hertz = nb of vibration per seconds
- Frequency = $1 / T$



What does a microphone record?

- A microphone measures amplitude of the variation in air pressure (in Pascal, coded in 16bits)
- Sounds is a continuous phenomenon discretized at a given sample rate
- Example with sampling rate=16kHz:



$[-0.10, 0.01, -0.02, \dots, 0.05]$

**3 seconds * 16 000 Hz =
vector of
length 48000**

What sampling rate should we choose for ASR?

- Nyquist frequency: highest detectable frequency = half of the sampling rate
- Human hearing frequencies: between 20Hz and 20kHz
- Speech production frequencies: mostly < 5kHz
- Therefore:
 - Music: sampling rate of 48kHz
 - Speech: sampling rate of 16kHz (and mobile phones: 8kHz)

Pascal and dB

Sound intensity is measured not in Pascal (Pa) but in decibels (dB)

$$1 \text{ dB} = 10 \cdot \log_{10}(P/P_0) \quad (P_0 = \text{threshold of hearing})$$

Why decibel:

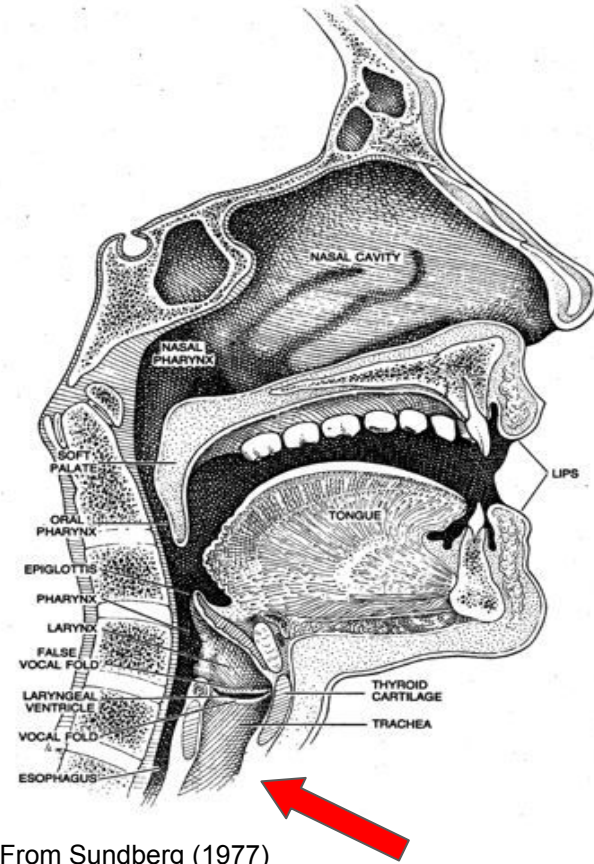
decibels are correlated (i.e linear increase)
with human perception of loudness

“Double the decibels sounds twice as loud”

A mosquito flying 3 m away	0 dB	Somebody shouting	100 dB
Softest audible 1000 Hz sound	6 dB	Pneumatic drill	100 dB
Quiet living room	20 dB	Helicopter	110 dB
Soft whispering	25 dB	Loud rock concert	110 dB
Refrigerator	40 dB	Threshold of pain	120 dB
Soft talking	50 dB	Air raid siren	130 dB
Normal conversation	60 dB	Gunshot	140 dB
Busy city street noise	70 dB	Instant perforation of eardrum	160 dB
Passing motorcycle	90 dB	Rocket launch	180 dB

Speech production

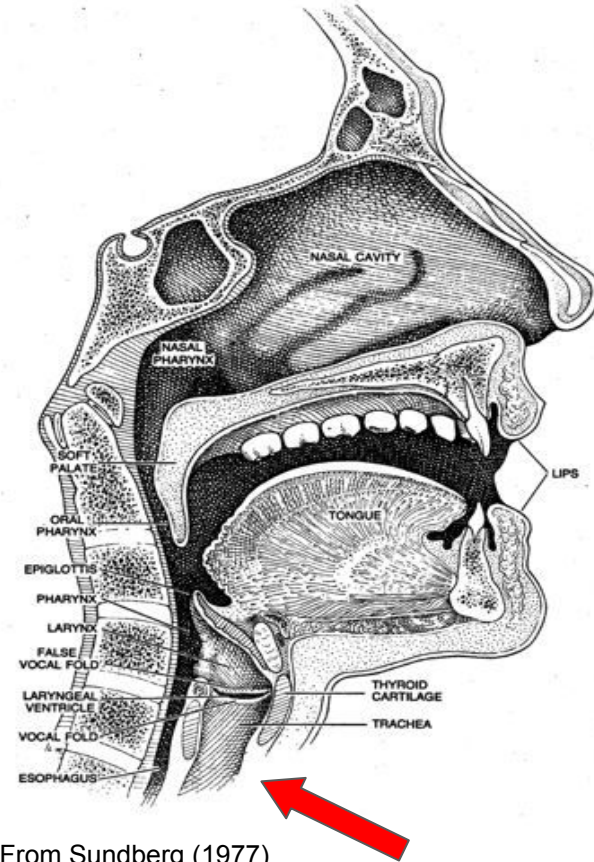
- Air is expelled from the lungs through the trachea



From Sundberg (1977)

Speech production

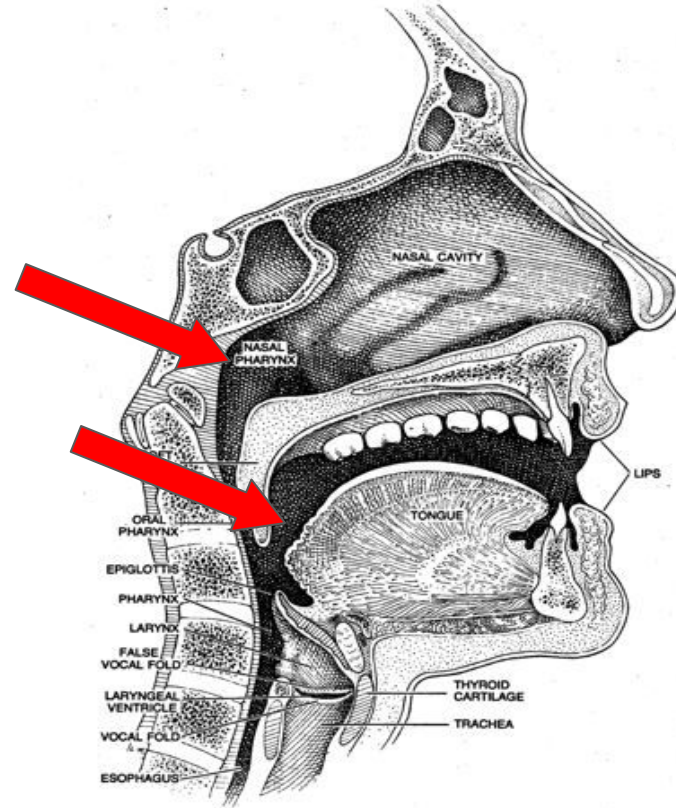
- Air is expelled from the lungs through the trachea
- Passes through vocal cords



From Sundberg (1977)

Speech production

- Air is expelled from the lungs through the trachea
- Passes through vocal cords
- Then through the nose and/or the mouth

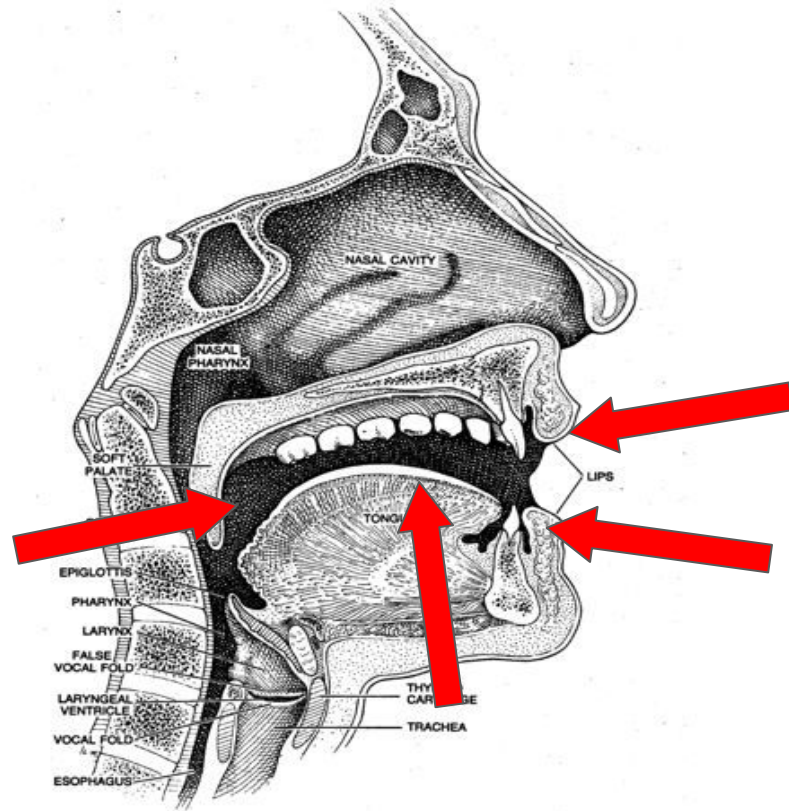


From Sundberg (1977)

Speech production

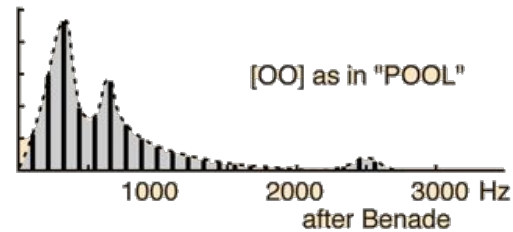
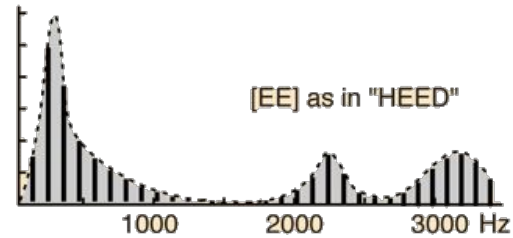
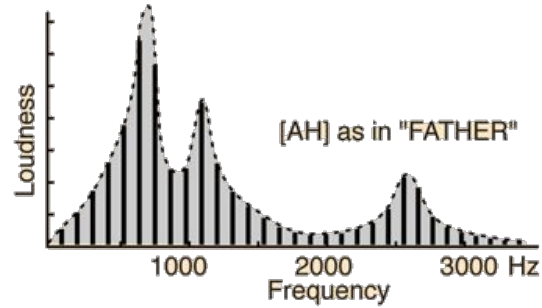
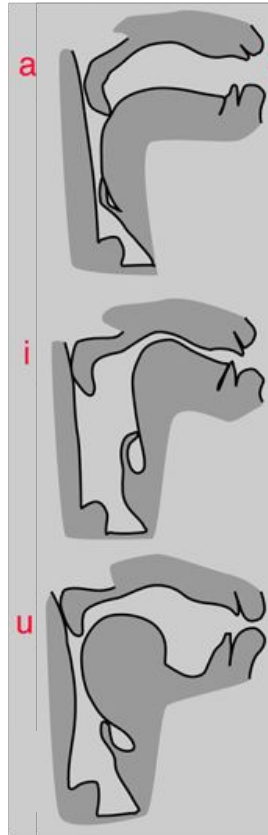
- Air is expelled from the lungs through the trachea
- Passes through vocal cords
- Then through the nose and/or the mouth
- Vocal tract: the “articulators”
 - Lips
 - Tongue
 - Teeth
 - Palate
 - ...

The way we control the articulators defines the sound we produce

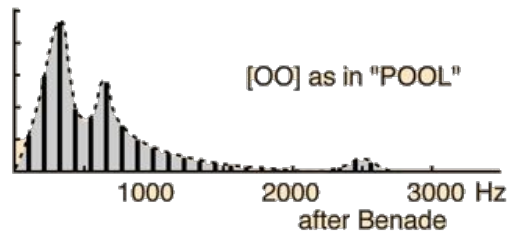
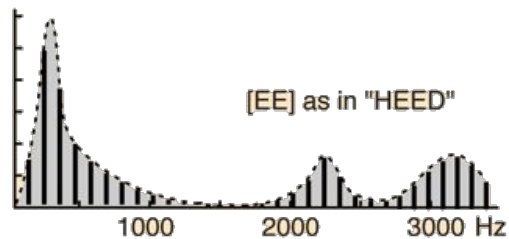
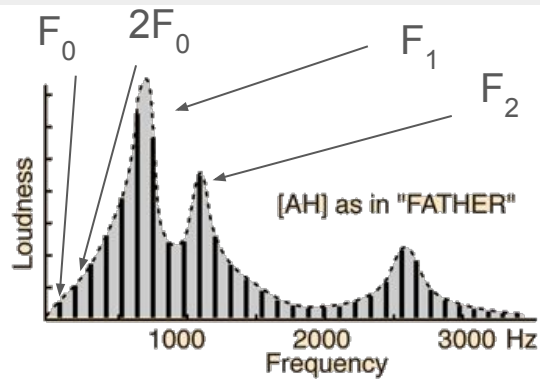


From Sundberg (1977)

Speech production



Speech production

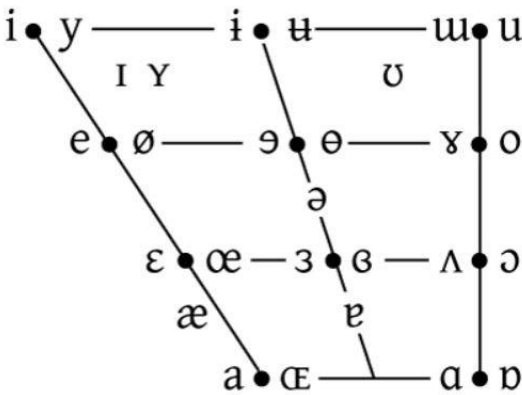


F_0 : pitch
 $n F_0$: harmonics
 F_1, F_2 : formants

Phonemes, phones, vowels and consonants

- **Phones:** any speech sounds (independent of a language)
- **Phonemes:** speech sounds that can discriminate words of a specific language
 - ex: 'r' and 'l' are phonemes in English and French but not in Japanese

Phones are usually vowels or consonants



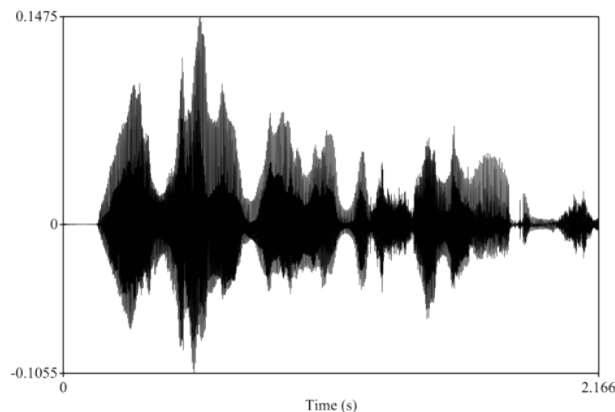
CONSONANTS (PULMONIC)

	Bilabial	Labiodental	Dental	Alveolar	Postalveolar	Retroflex	Palatal	Velar	Uvular	Pharyngeal	Glottal
Plosive	p b			t d		ʈ ɖ	c ɟ	k ɡ	q ɢ		ʔ
Nasal	m	ɱ		n		ɳ	ɲ	ŋ	ɴ		
Trill	ʙ			r					ʀ		
Tap or Flap				ɾ		ɽ					
Fricative	ɸ β	f v	θ ð	s z	ʃ ʒ	ʂ ʐ	ç ʝ	x ɣ	χ ʁ	ħ ʕ	h ɦ
Lateral fricative				ɬ ɮ							
Approximant		ʋ		ɹ		ɻ	j	ɰ			
Lateral approximant				l		ɭ	ʎ	ʟ			

Where symbols appear in pairs, the one to the right represents a voiced consonant. Shaded areas denote articulations judged impossible.

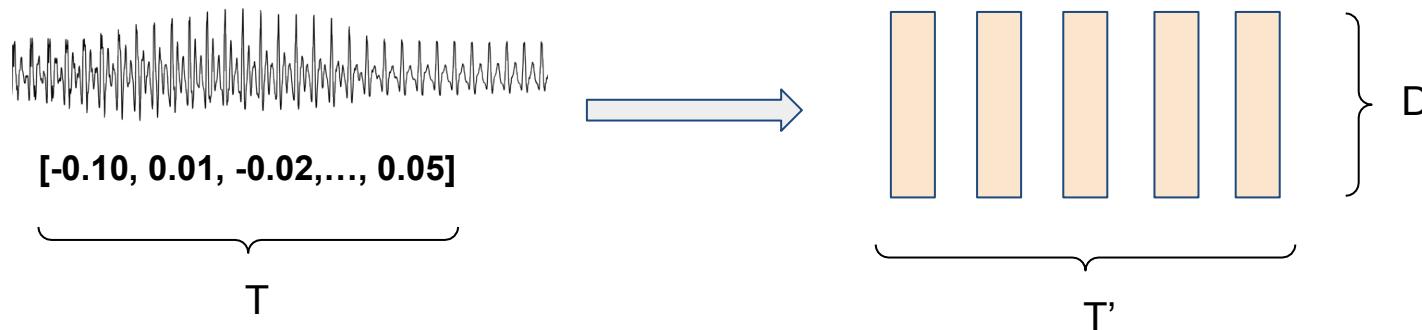
“Better” representation of speech than raw signal

Make the job of subsequent ASR model easier



Speech features: definition and properties

Definition: Speech features are the output of a function from $\mathbb{R}^T$ into $\mathbb{R}^{T' \times D}$



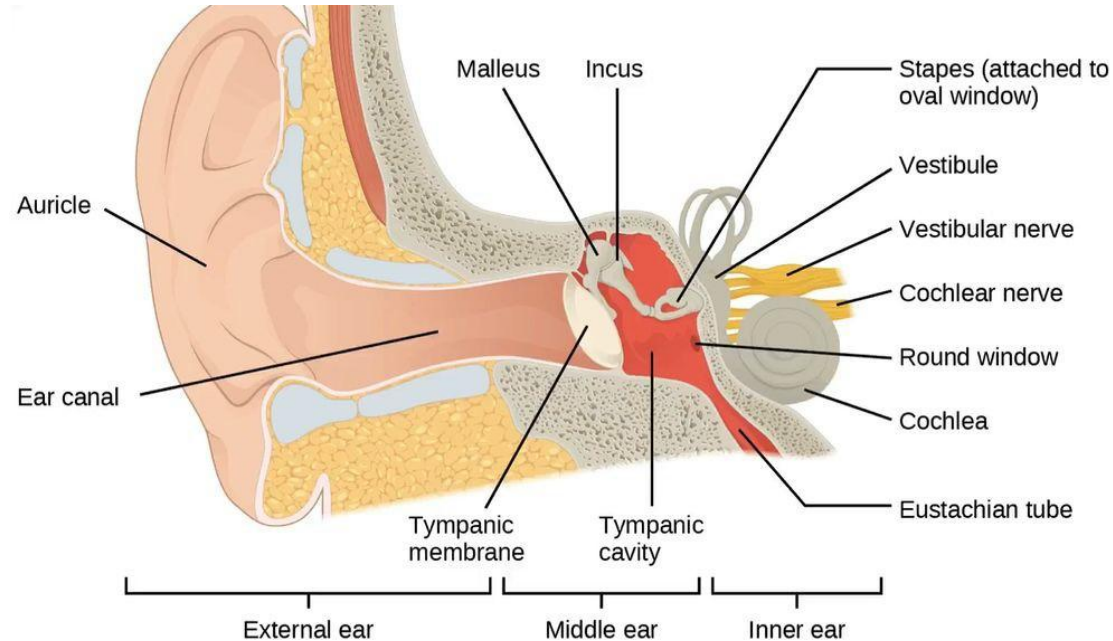
Properties:

Each vector is called a frame

- 1) $T' \ll T$ (necessary for any causal model)
- 2) locality: each frame represent a small portion of the speech signal
- 3) invariance: 'similar' speech sounds have similar vector representations

How to encode speech?

The cochlea has a range of sensors that reacts at the beginning to high frequencies (20kHz) and at the end to low frequencies (20Hz)

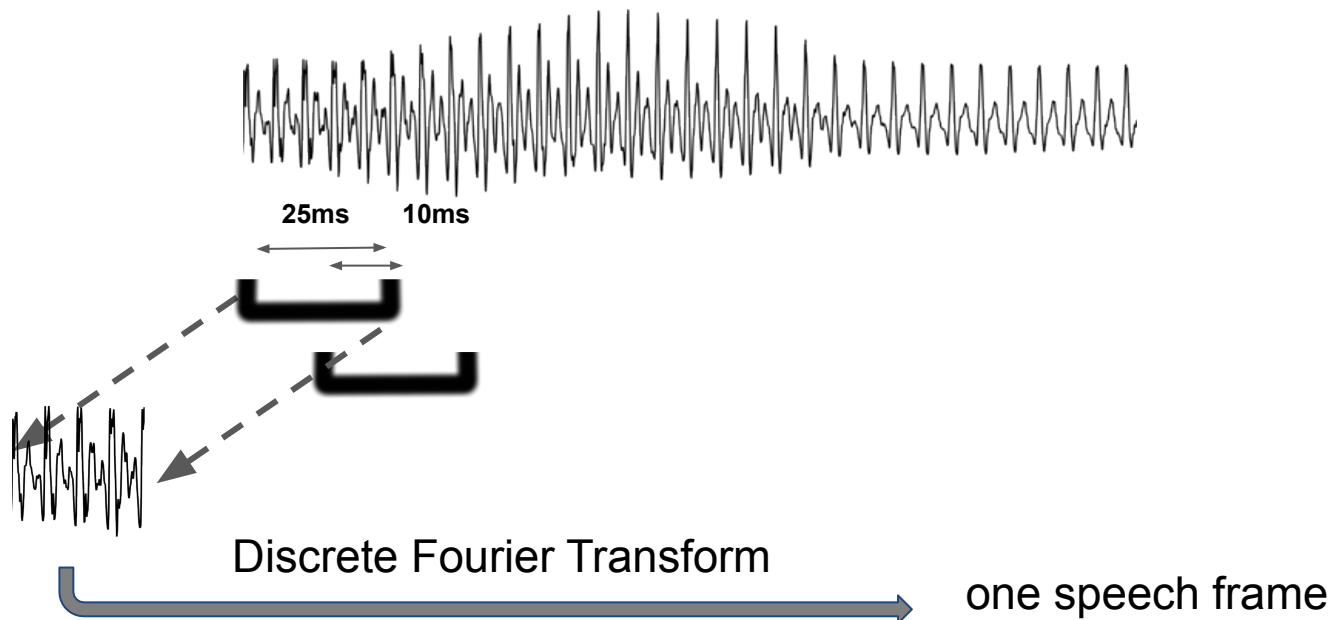


Main idea for speech features: spectral analysis

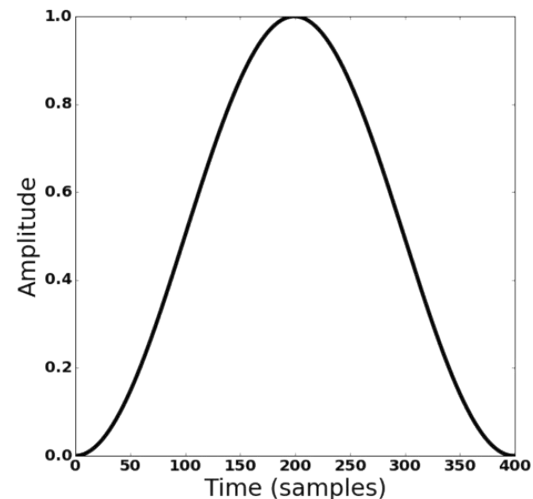
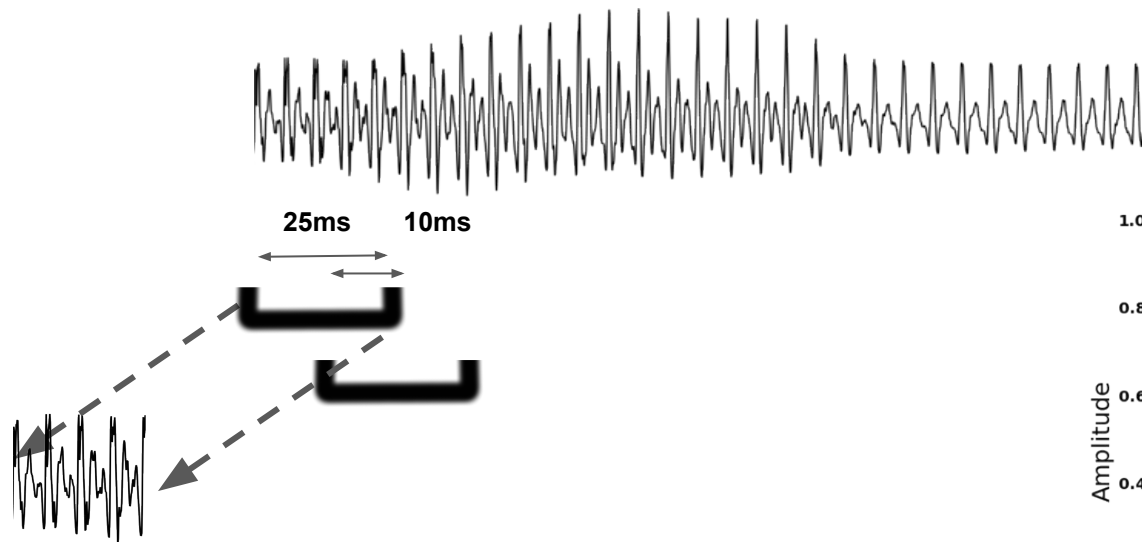
Speech is a vibration of local chords: locally periodic

Can be decomposed as a sum of frequencies !

Apply Fourier series on each small chunk of speech



Main idea for speech features: spectral analysis



Speech chunks are not really periodic:

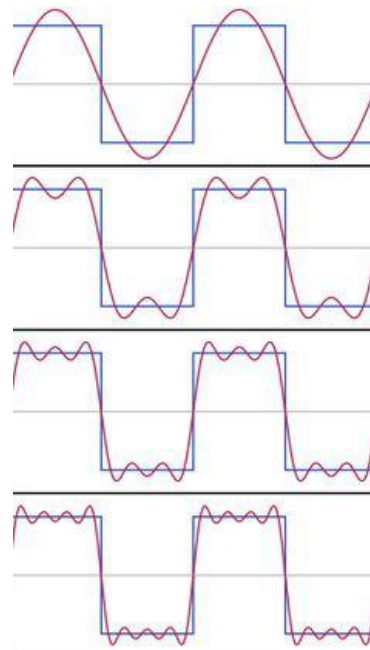
→ Apply Hanning window to smooth both ends

$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right)$$

Fourier Series

- Periodic function with frequency f , can be expressed by the sum of sine and cosine waves whose frequencies are multiple of f .
- These waves are called the harmonics

Let us take all sine waves in the human voice range, and compute how present they are in a speech chunk



The Discrete Fourier Transform (DFT)

- How present is a frequency in a speech chunk? Dot product!

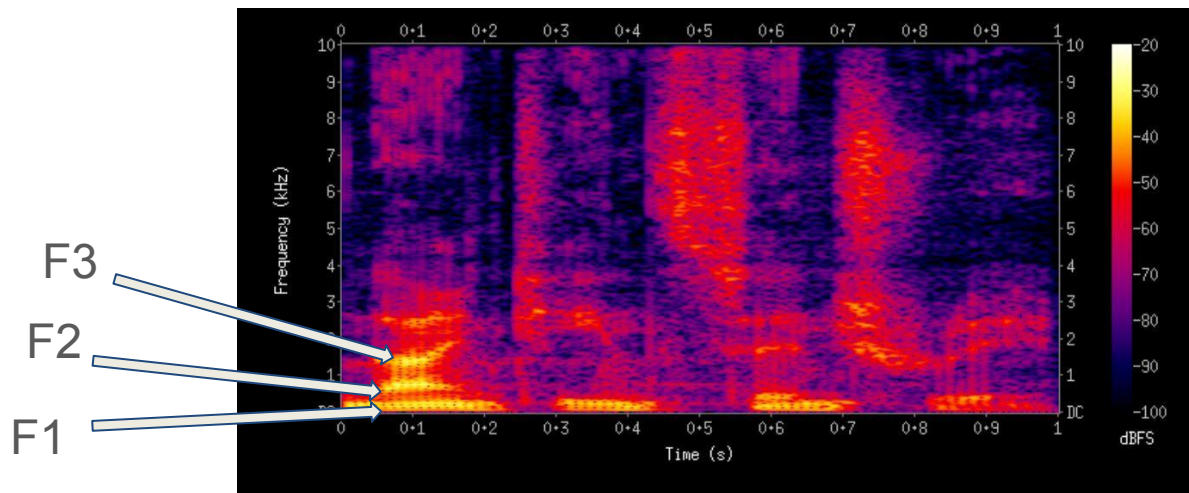
$$X(\omega) = \sum_{n=0}^{N-1} x_n \cdot e^{-i2\pi\omega \frac{n}{N}} \leftarrow \text{a sine wave}$$

Frequency $\uparrow$ $\uparrow$ Waveform value over the 25ms chunk

- Complex value, so we take the modulus of it (i.e the power of a frequency)
- For each chunk: do this for all frequencies in human voice range
- In practice, 256 frequencies evenly space in $[0, \text{samplerate}/2]$
- The phase is ignored!

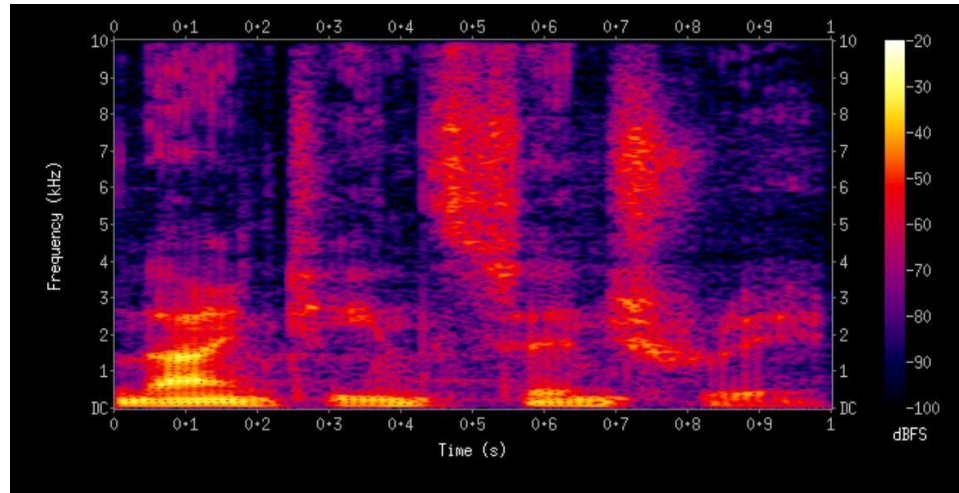
DFT of a speech utterance: spectrogram

The spectrogram is the power of all frequencies in all speech chunks
Depending on the temporal/frequency precision of the DFT, we can see F0 and its harmonics, or just the formants



Spectrogram

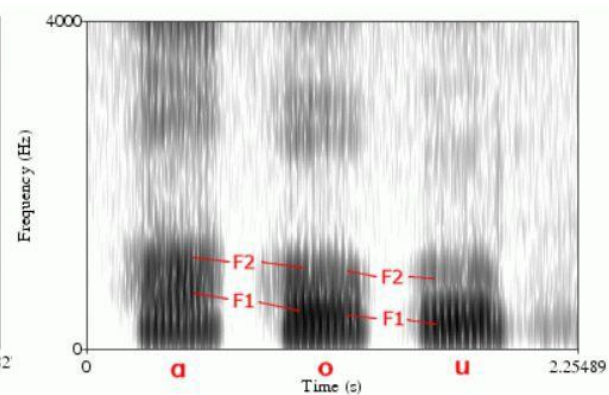
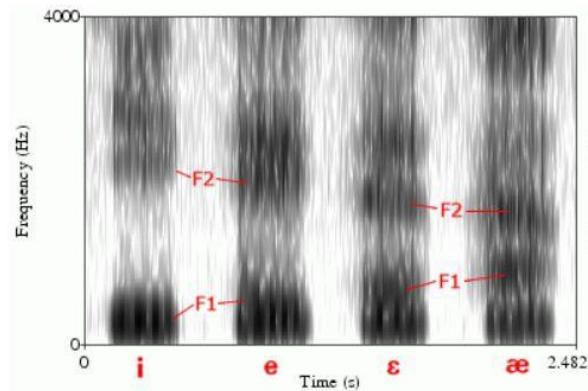
Some sounds have an spectrographic 'signature'
You can learn to 'read' a spectrogram



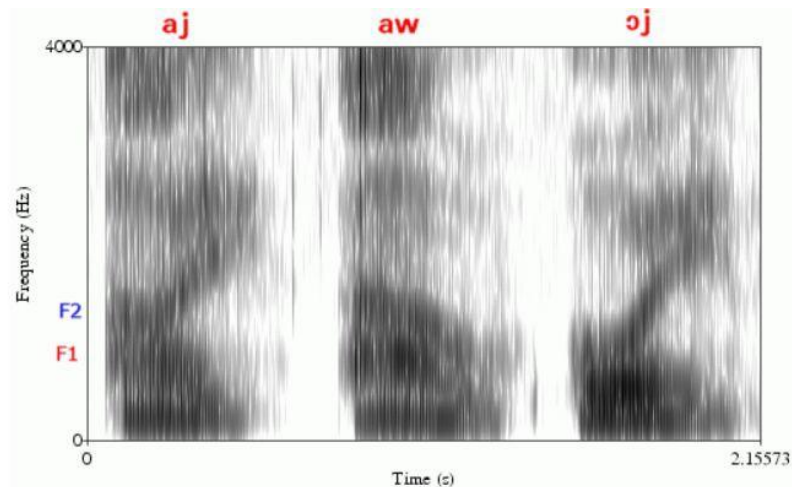
Spectrogram

Example of vowels and diphthongs

Steady state vowels
(the formants are stable)



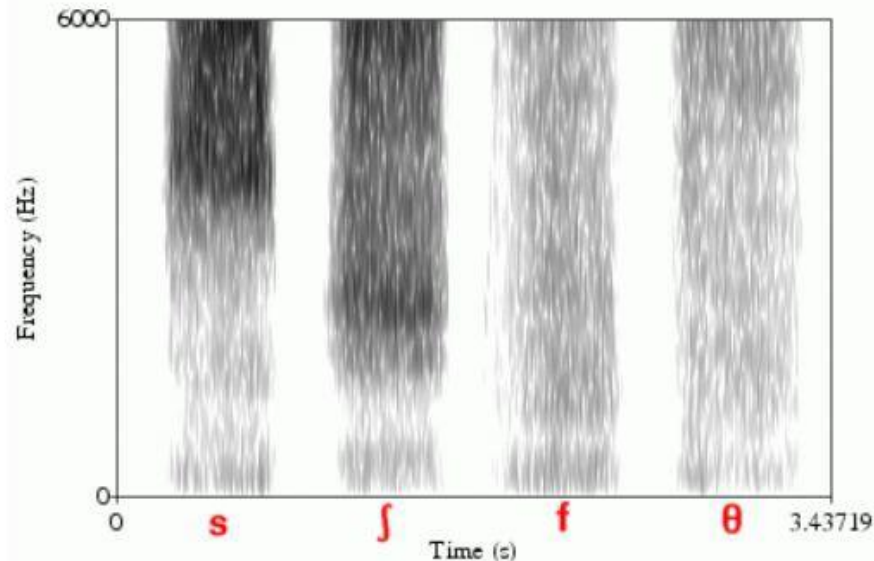
Diphtongs
(the formants change)



Unvoiced phones

Some phones are not voiced:

- No vocal cord periodic signal
turbulences created by air flow, more difficult to see formants



Can we make better features than a spectrogram?

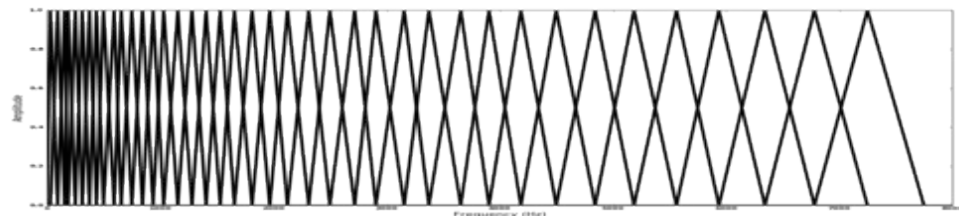
Main ideas:

- Make spectrogram closer to human perception of sounds
- We want less dimensions (currently 256)

Matching human perception: the mel-filterbanks

Human ear more sensitive to variations in low frequencies than in high frequencies

→ We warp the spectrogram to a new scale that gives more importance to low frequencies



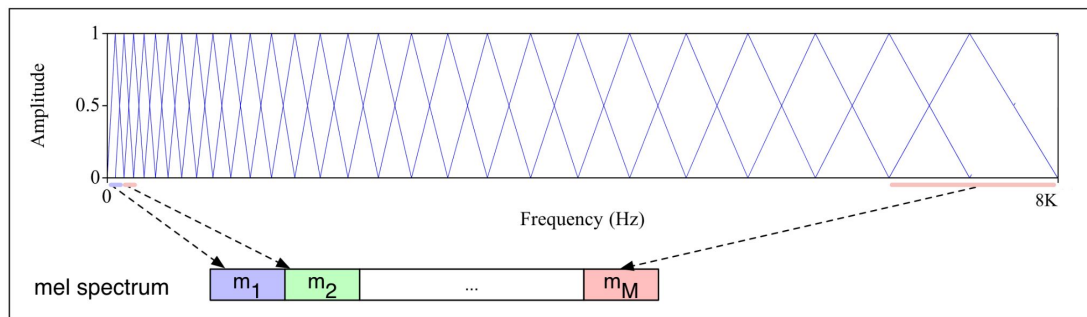
$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

Mel-scale **Linear scale**

Matching human perception: the mel-filterbanks

Human ear more sensitive to variations in low frequencies than in high frequencies

→ We warp the spectrogram to a new scale that gives more importance to low frequencies



$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

Mel-scale

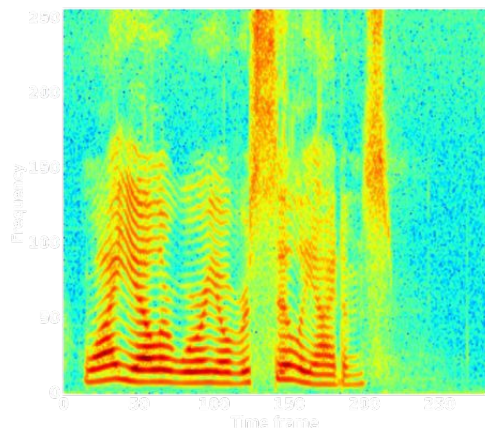
**Linear
scale**

- Filters are linearly spaced in the mel scale
- We compute the product of our spectrum with each filter, then sum the results → our features are now a vector of length $n = \text{number of filters}$

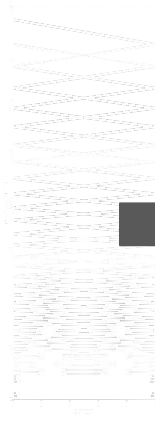
Matching human perception: the mel-filterbanks

~6x fewer dimensions than spectrograms

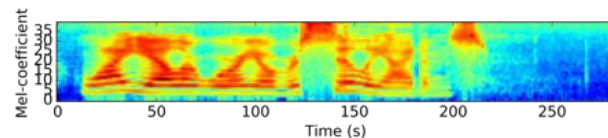
$$\text{Melfbank}_j(k) = \sum_{\omega=0}^{256} \text{Spectrogram}(k, \omega) \text{Melfilter}_j(\omega)$$



Each frame has 256 dimensions



Mel-averaging



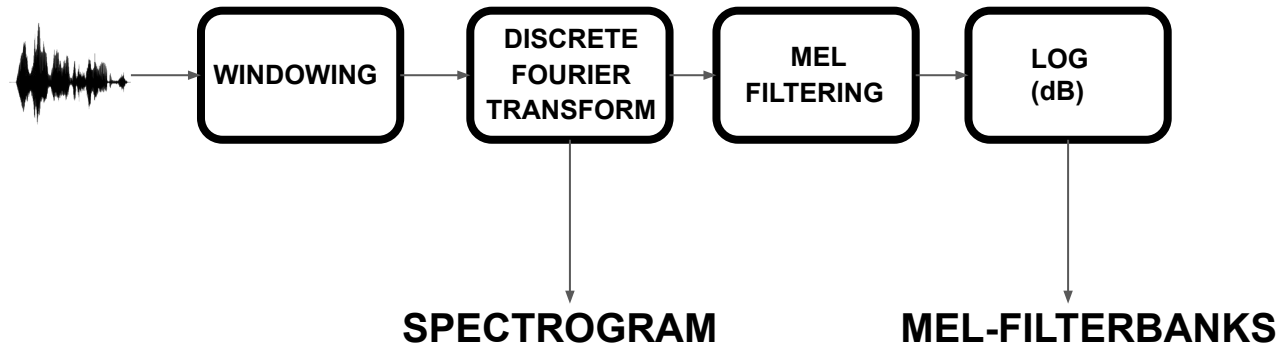
Each frame has 40 dimensions

Matching human perception: logarithmic compression

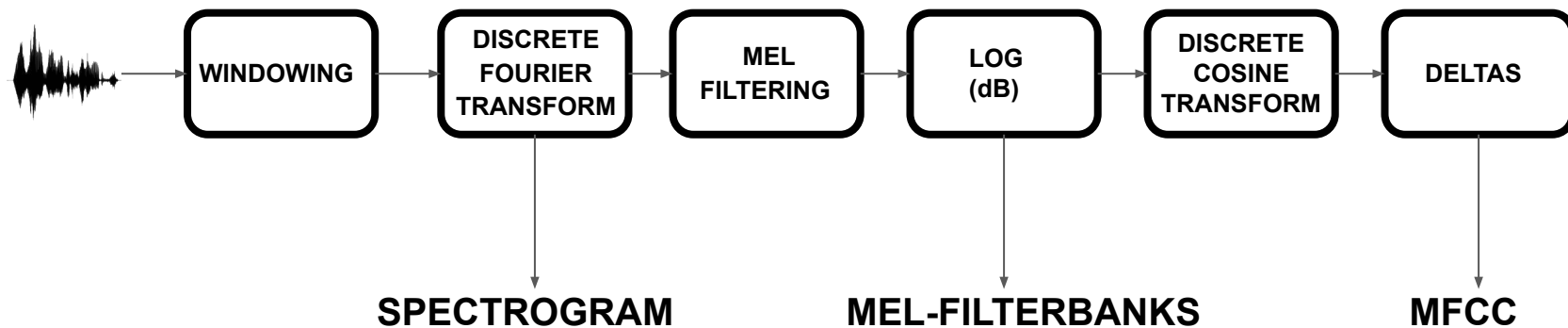
Huge range of variation in energy, but ear has logarithmic sensitivity to loudness

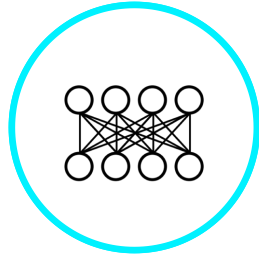


Summing up



(From mel-filterbanks to MFCC)

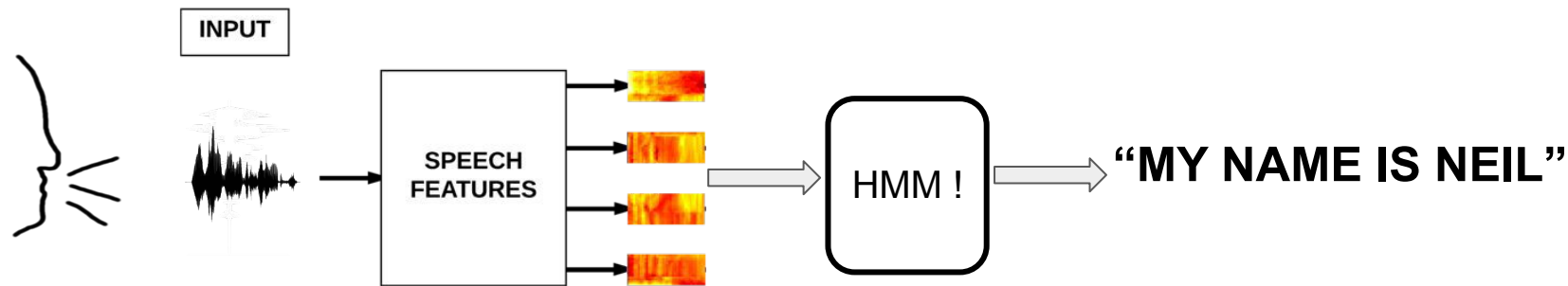




Chap 2. Acoustic modeling

- The statistical approach
- Hidden Markov Models with Gaussian Mixture Models (HMM/GMM)

Speech recognition as a statistical problem



$$X_T = x_1, \dots, x_T$$

A sequence of features frames

$$W_M = w_1, \dots, w_M$$

A sequence of words

$$X_T = x_1, \dots, x_T$$

A sequence of features frames

$$W_M = w_1, \dots, w_M$$

A sequence of words

Goal of ASR: $\hat{W} = \arg \max_W P(W|X)$

Goal of ASR: $\hat{W} = \arg \max_W P(W|X)$

Bayes formula: $\hat{W} = \arg \max_W P(X|W)P(W)$



**ACOUSTIC
MODEL** **LANGUAGE
MODEL**

We omit $P(X)$ as we maximize over W .

Goal of ASR: $\hat{W} = \arg \max_W P(W|X)$

Bayes formula: $\hat{W} = \arg \max_W P(X|W)P(W)$

ACOUSTIC
MODEL

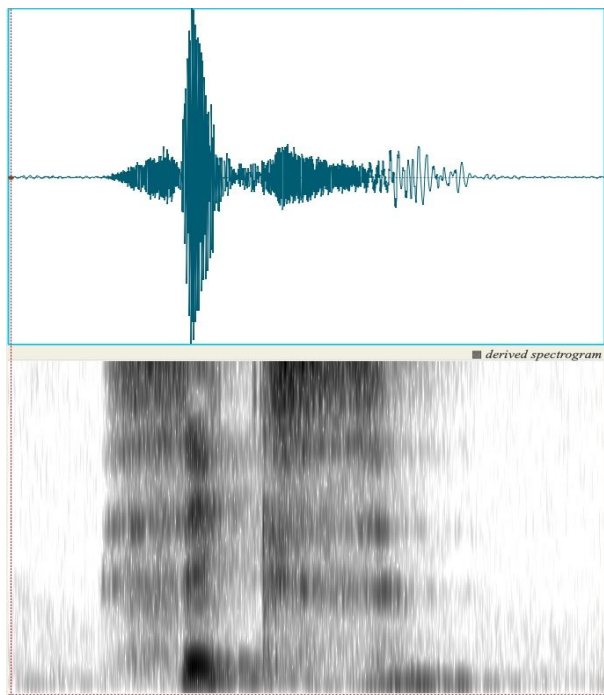
LANGUAGE
MODEL

We omit $P(X)$ as we maximize over W .

Using HMM /GMM for computing $P(X|W)$

Let us consider the word 'six':

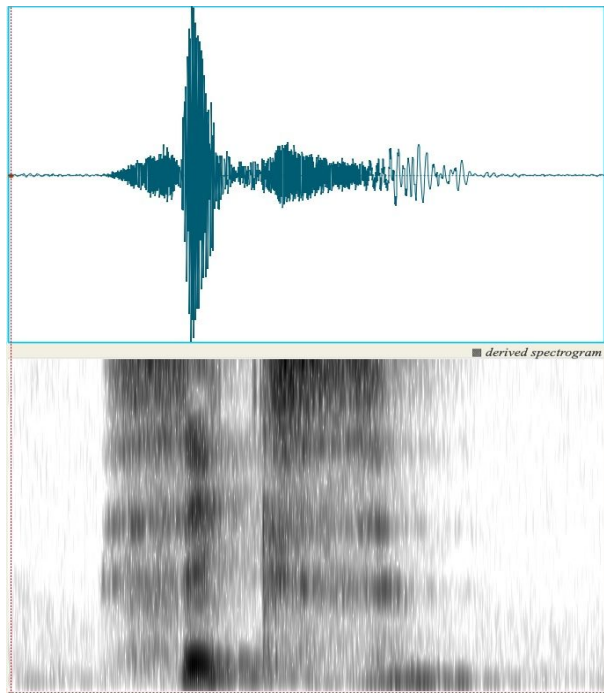
- A sequence of acoustic events



Using HMM /GMM for computing $P(X|W)$

Let us consider the word 'six':

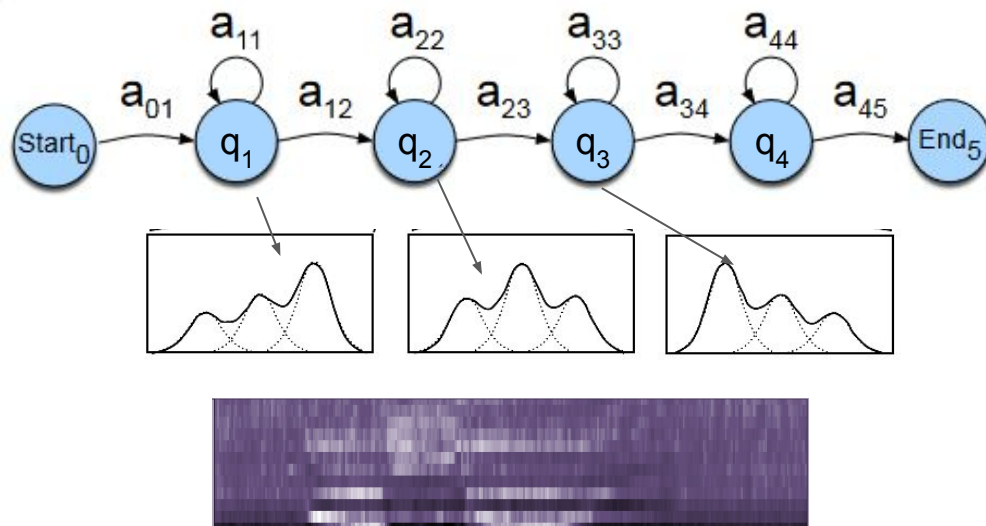
- A sequence of acoustic events
- Represented as a sequence of speech features



Using HMM /GMM for computing $P(X|W)$

Let us consider the word 'six':

- A sequence of acoustic events
- Represented as a sequence of speech features
- Which will be modelled as a sequence of latent states using an HMM/GMM



Definition:

$$Q = q_1 q_2 \dots q_N$$

a set of N **states**

$$A = a_{11} a_{12} \dots a_{N1} \dots a_{NN}$$

a **transition probability matrix** A , each a_{ij} representing the probability of moving from state i to state j , s.t.
 $\sum_{j=1}^n a_{ij} = 1 \quad \forall i$

$$\pi = \pi_1, \pi_2, \dots, \pi_N$$

an **initial probability distribution** over states. π_i is the probability that the Markov chain will start in state i . Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^N \pi_i = 1$

Markov Assumption: $P(q_i = a | q_1 \dots q_{i-1}) = P(q_i = a | q_{i-1})$

Example:

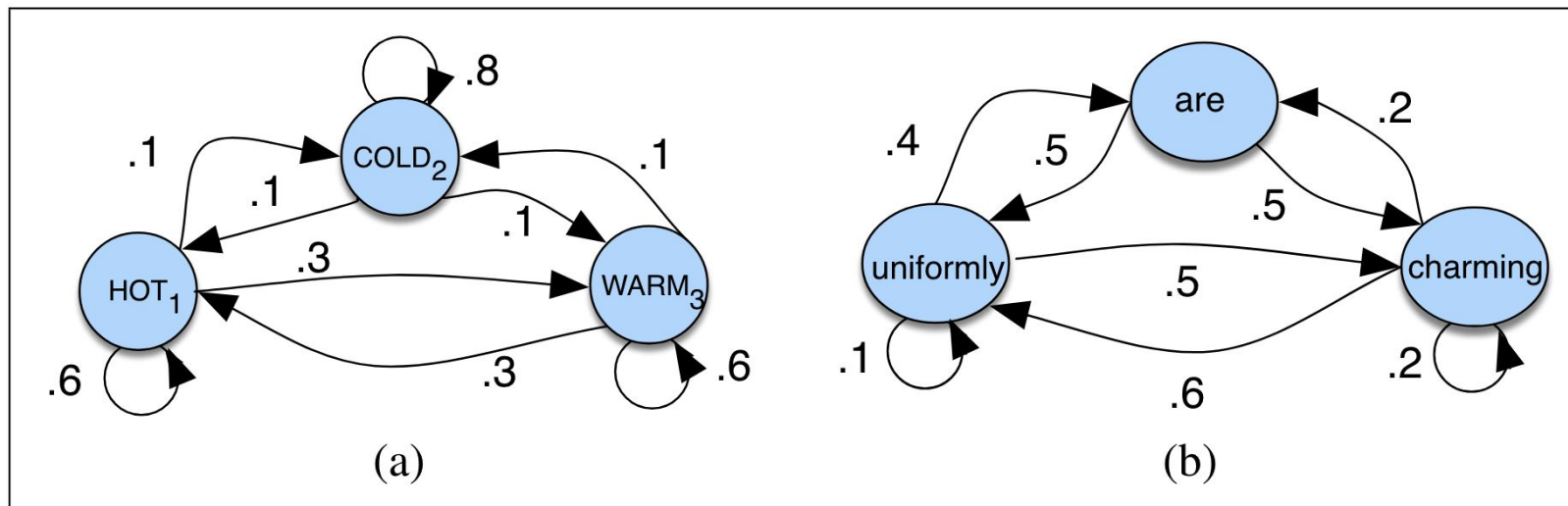
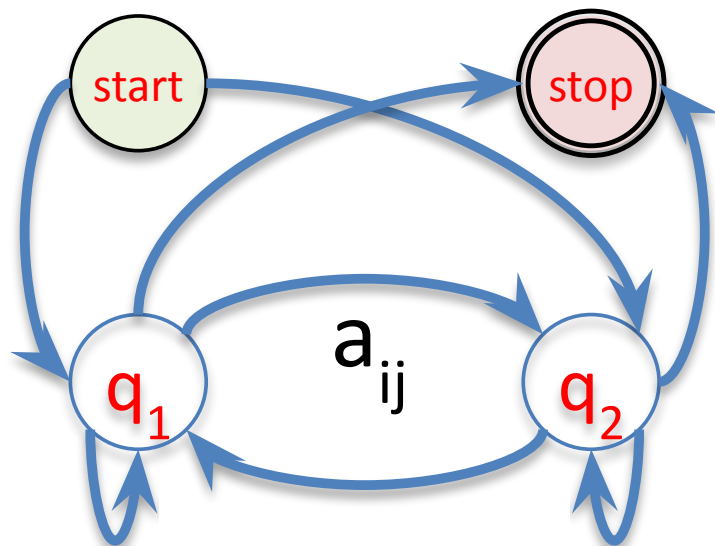


Figure A.1 A Markov chain for weather (a) and one for words (b), showing states and transitions. A start distribution π is required; setting $\pi = [0.1, 0.7, 0.2]$ for (a) would mean a probability 0.7 of starting in state 2 (cold), probability 0.1 of starting in state 1 (hot), etc.

Markov chains

With special states:



A

	q_0 (start)	q_1	q_2	q_F (stop)
q_0 (start)	0	a_{01}	a_{02}	0
q_1	0	a_{11}	a_{12}	a_{1F}
q_2	0	a_{21}	a_{22}	a_{2F}
q_F (stop)	0	0	0	0

Definition:

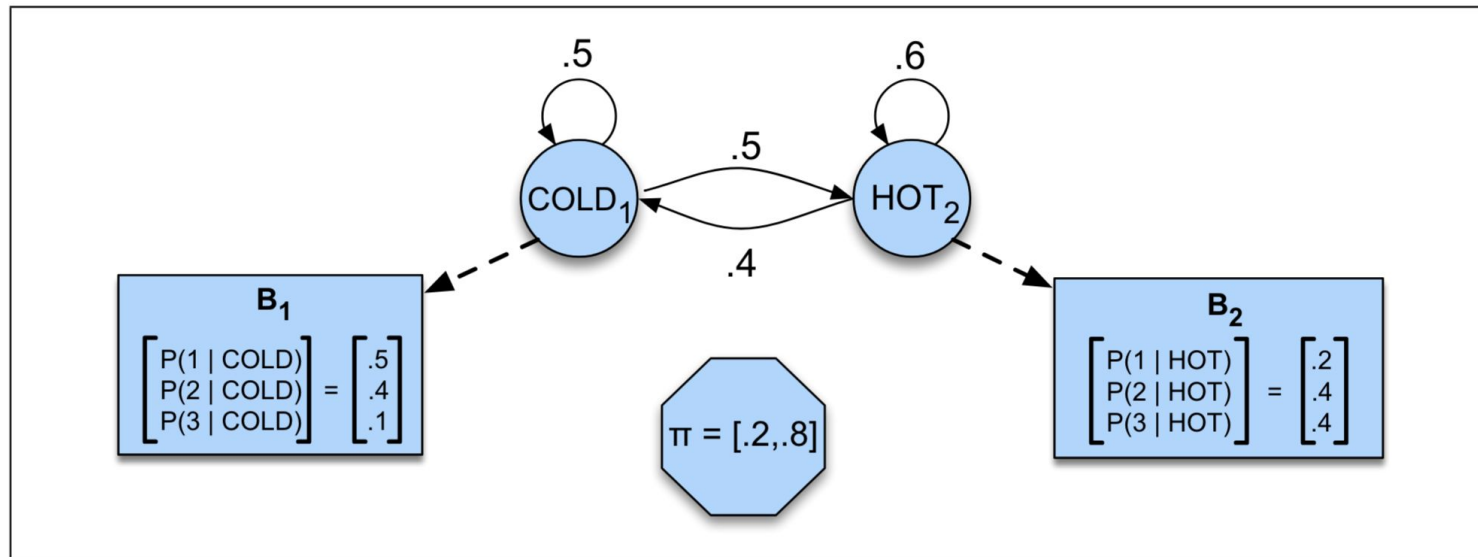
$Q = q_1 q_2 \dots q_N$	a set of N states
$A = a_{11} \dots a_{ij} \dots a_{NN}$	a transition probability matrix A , each a_{ij} representing the probability of moving from state i to state j , s.t. $\sum_{j=1}^N a_{ij} = 1 \quad \forall i$
$B = b_i(o_t)$	a sequence of observation likelihoods , also called emission probabilities , each expressing the probability of an observation o_t (drawn from a vocabulary $V = v_1, v_2, \dots, v_V$) being generated from a state q_i
$\pi = \pi_1, \pi_2, \dots, \pi_N$	an initial probability distribution over states. π_i is the probability that the Markov chain will start in state i . Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^n \pi_i = 1$

Markov Assumption: $P(q_i | q_1 \dots q_{i-1}) = P(q_i | q_{i-1})$

Output Independence: $P(o_i | q_1 \dots q_i, \dots, q_T, o_1, \dots, o_i, \dots, o_T) = P(o_i | q_i)$

Hidden Markov Model

Example:

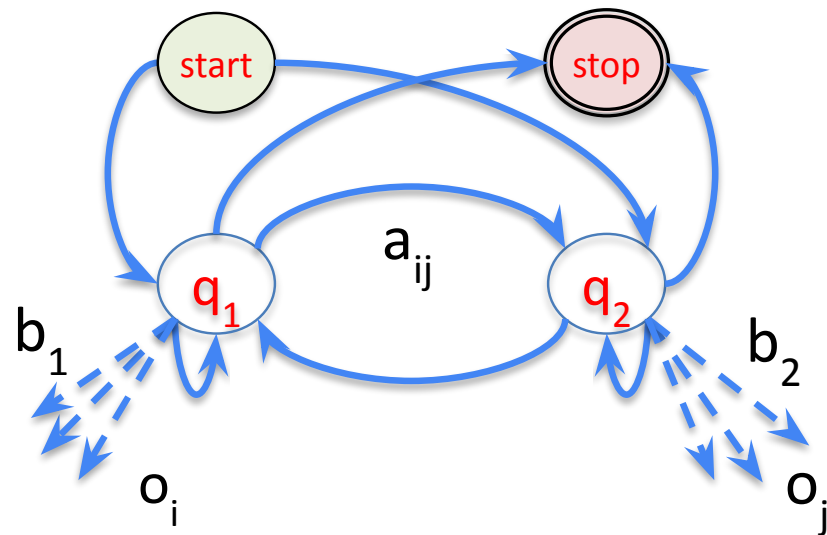


Hidden states (Q): weather (hot or cold)

Observations (O): number of ice cream eaten

Hidden Markov Model

With special states:



A

	q_0 (start)	q_1	q_2	q_F (stop)
q_0 (start)	0	a_{01}	a_{02}	0
q_1	0	a_{11}	a_{12}	a_{1F}
q_2	0	a_{21}	a_{22}	a_{2F}
q_F (stop)	0	0	0	0

B

	q_1	q_2
o_1	b_{11}	b_{21}
o_2	b_{12}	a_{22}
o_3	b_{13}	a_{23}

HMM fundamental problems:

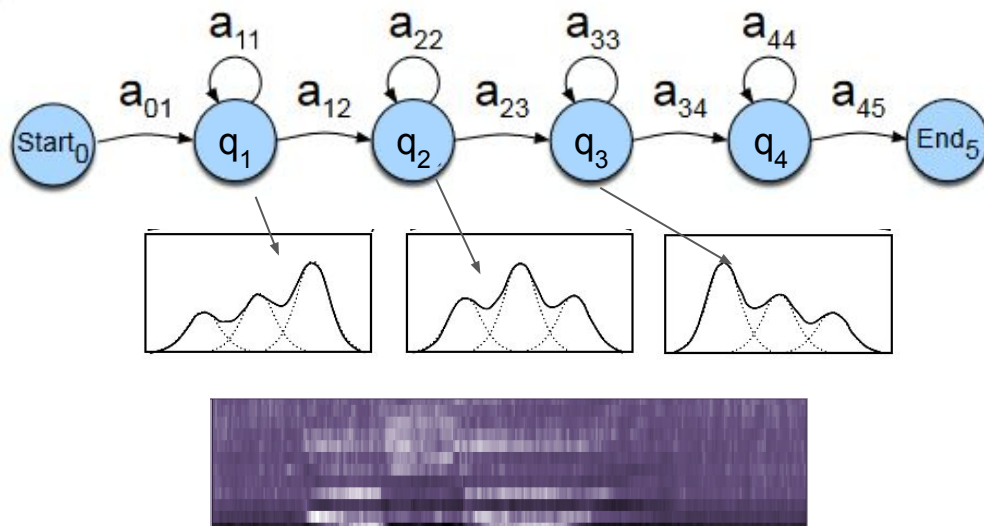
- Problem 1 (Likelihood):** Given an HMM $\lambda = (A, B)$ and an observation sequence O , determine the likelihood $P(O|\lambda)$.
- Problem 2 (Decoding):** Given an observation sequence O and an HMM $\lambda = (A, B)$, discover the best hidden state sequence Q .
- Problem 3 (Learning):** Given an observation sequence O and the set of states in the HMM, learn the HMM parameters A and B .

Break and questions!

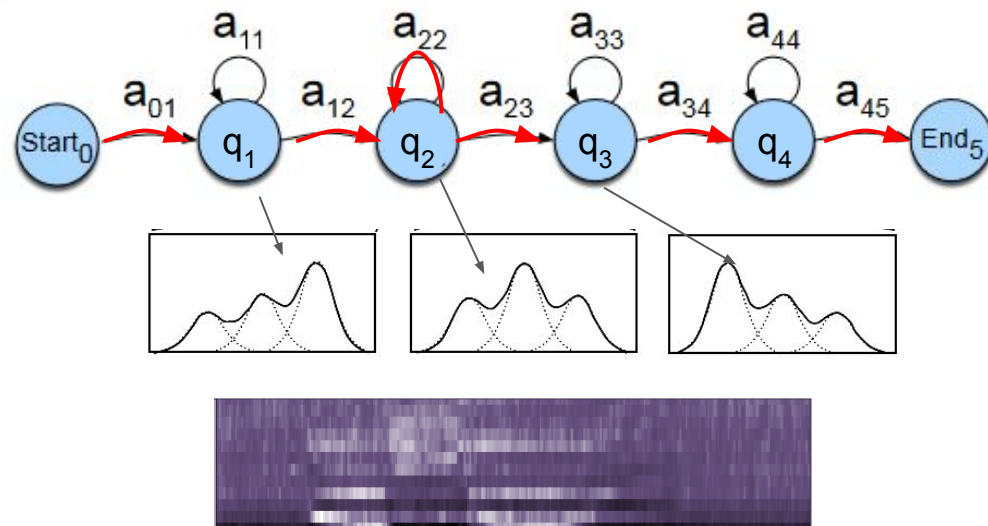
Using HMM /GMM for computing $P(X|W)$

Let us consider the word 'six':

- A sequence of acoustic events
- Represented as a sequence of speech features
- Which will be modelled as a sequence of latent states using an HMM/GMM



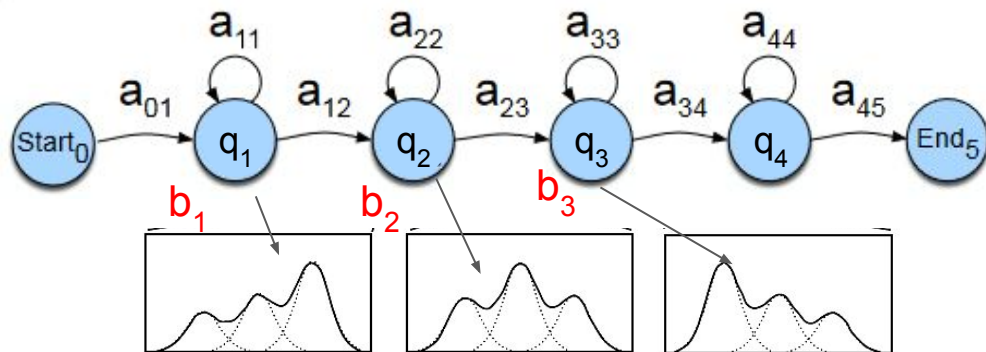
Using HMM /GMM for computing $P(X|W)$



$$p(q_1 q_2 q_2 q_3 q_4) = a_{01} a_{12} a_{22} a_{23} a_{34} a_{45}$$

Problem 1: Likelihood of the observation

What is the likelihood of the observation?



$p(o_1 \dots o_T)$?



$o_1 o_2$

...

o_T

Problem 1: Likelihood of the observation

- What is the probability of an observation O (of length T) under an HMM= (A, B) with N states?

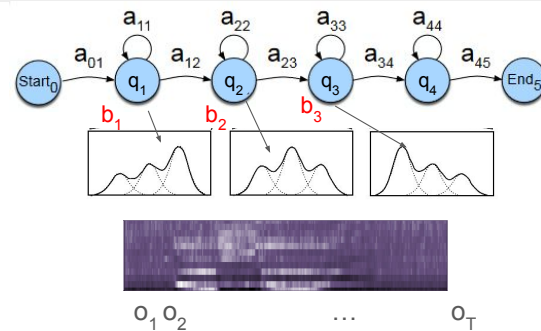
$$P(O) = \sum_Q P(O, Q) = \sum_Q P(O|Q)P(Q)$$

- Using the output independence assumption and the Markov assumption:

$$P(O, Q) = P(O|Q) \times P(Q) = \prod_{i=1}^T P(o_i|q_i) \times \prod_{i=1}^T P(q_i|q_{i-1})$$



B (emission) A (transition)

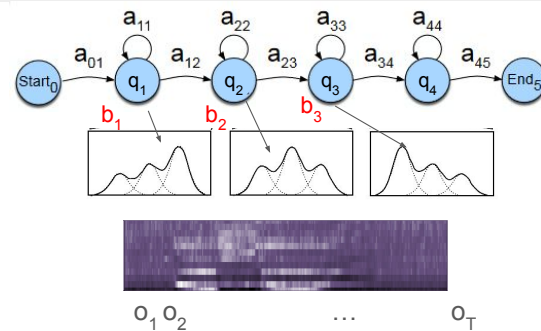


$p(o_1 \dots o_T)$?

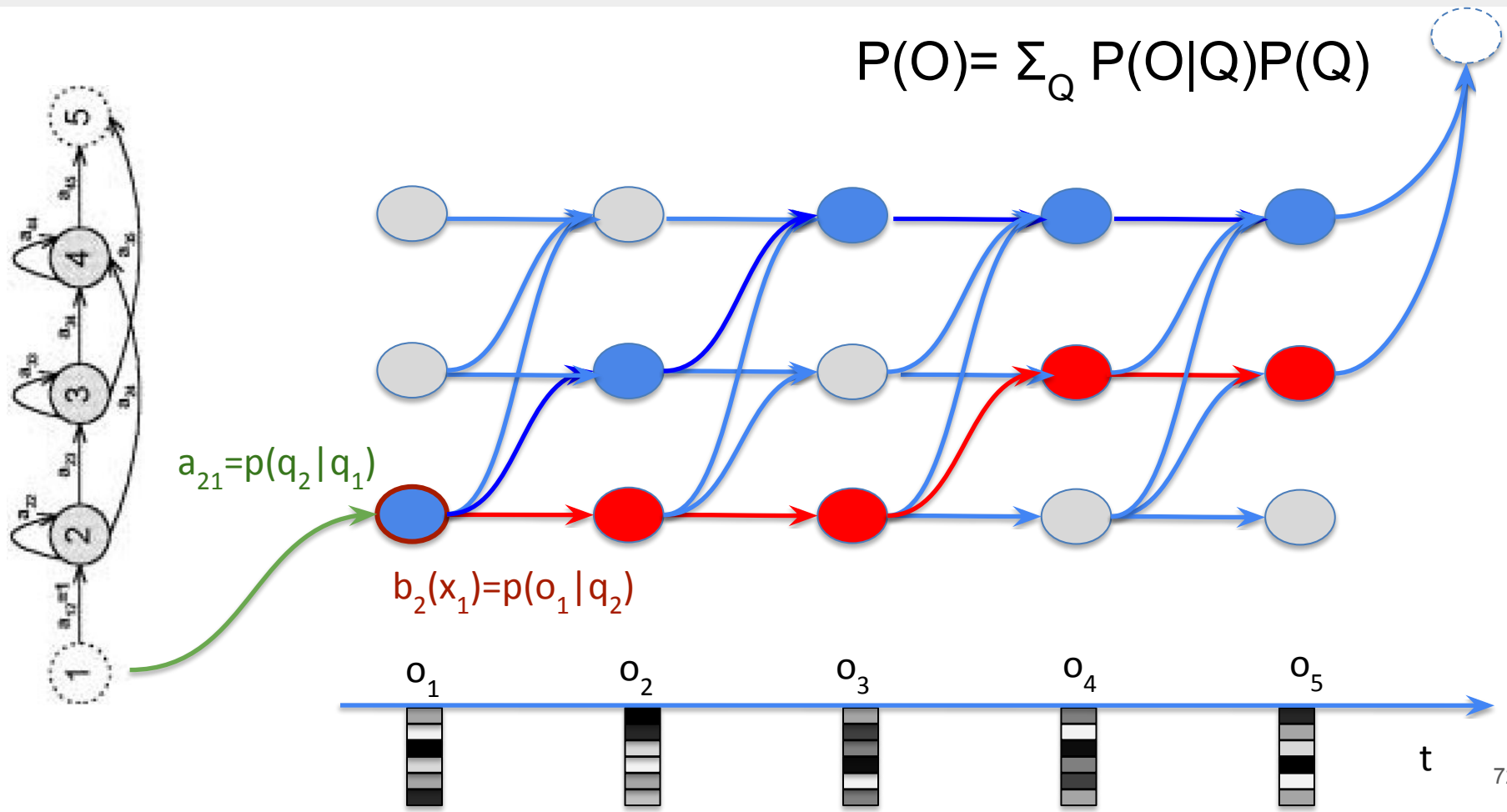
Problem 1: Likelihood of the observation

$$P(O) = \sum_Q \prod_{i=1}^T P(o_i | q_i) \times \prod_{i=1}^T P(q_i | q_{i-1})$$

- For HMM with N hidden states and observation of size T:
 N^T possible paths $\rightarrow$ *Not tractable*
- Use instead the **forward algorithm** (dynamic programming):
Computation cost in $O(N^2 T)$



Likelihood and forward algorithm



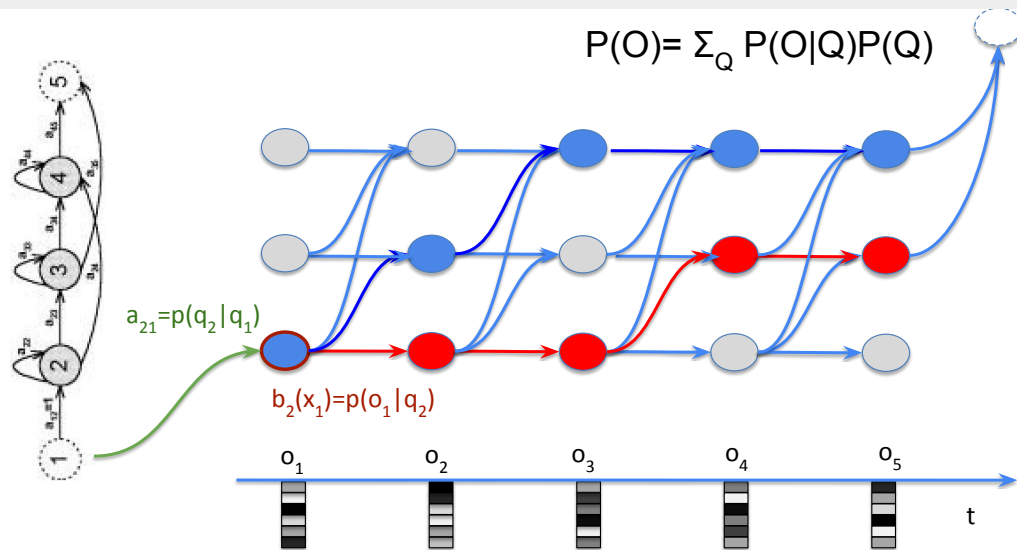
Likelihood and forward algorithm

- Create a **forward treillis**
- Each cell $\alpha_{t,j}$ represents the probability of being in state j after seeing the first t observations

$$\alpha_t(j) = P(o_1, o_2 \dots o_t, q_t = j | \lambda)$$

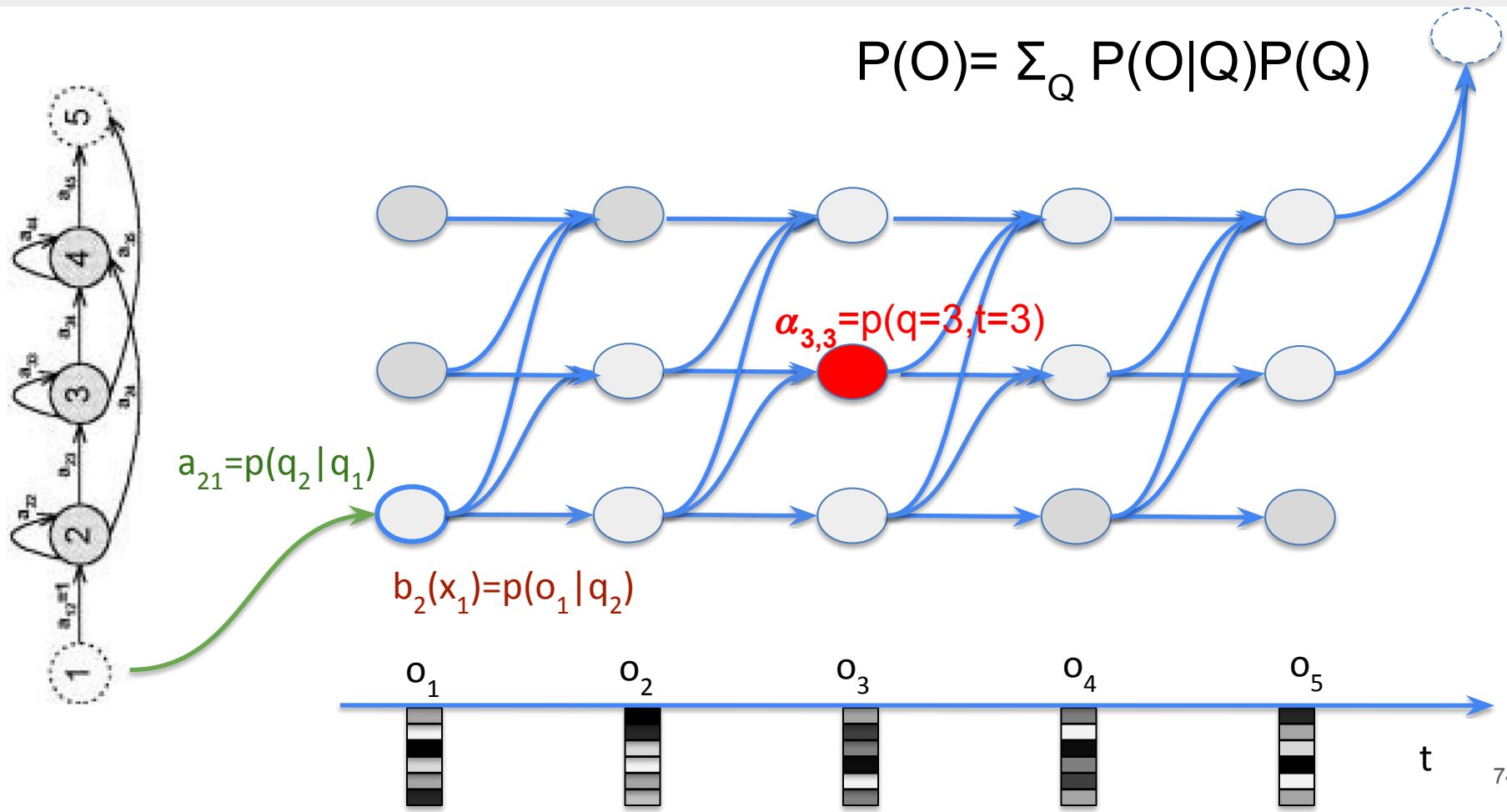
- We compute $\alpha_{t,j}$ by summing over all paths that lead to this

$$\alpha_t(j) = \sum_{i=1}^N \alpha_{t-1}(i) a_{ij} b_j(o_t)$$

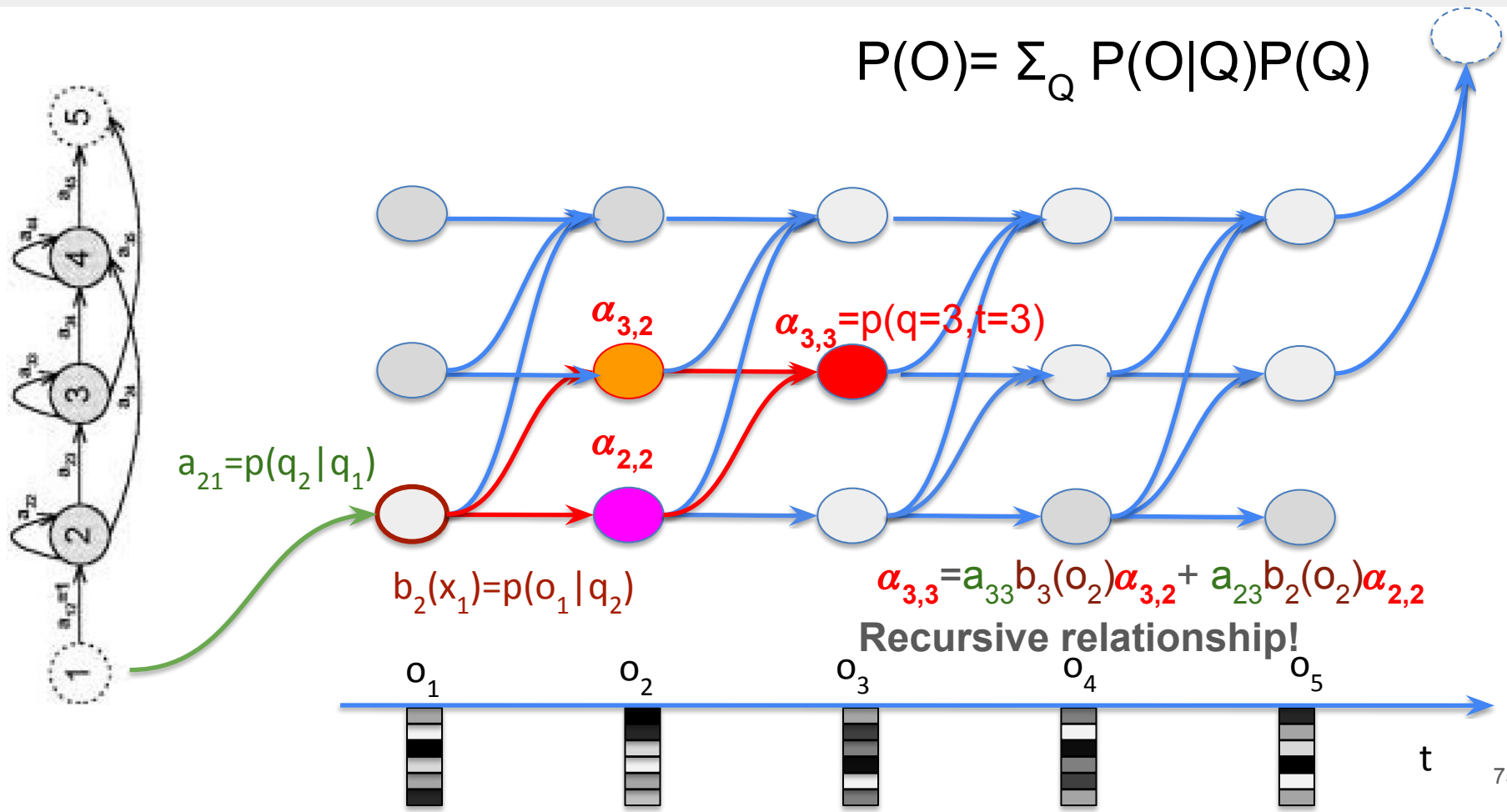


$\alpha_{t-1}(i)$	the previous forward path probability from the previous time step
a_{ij}	the transition probability from previous state q_i to current state q_j
$b_j(o_t)$	the state observation likelihood of the observation symbol o_t given the current state j

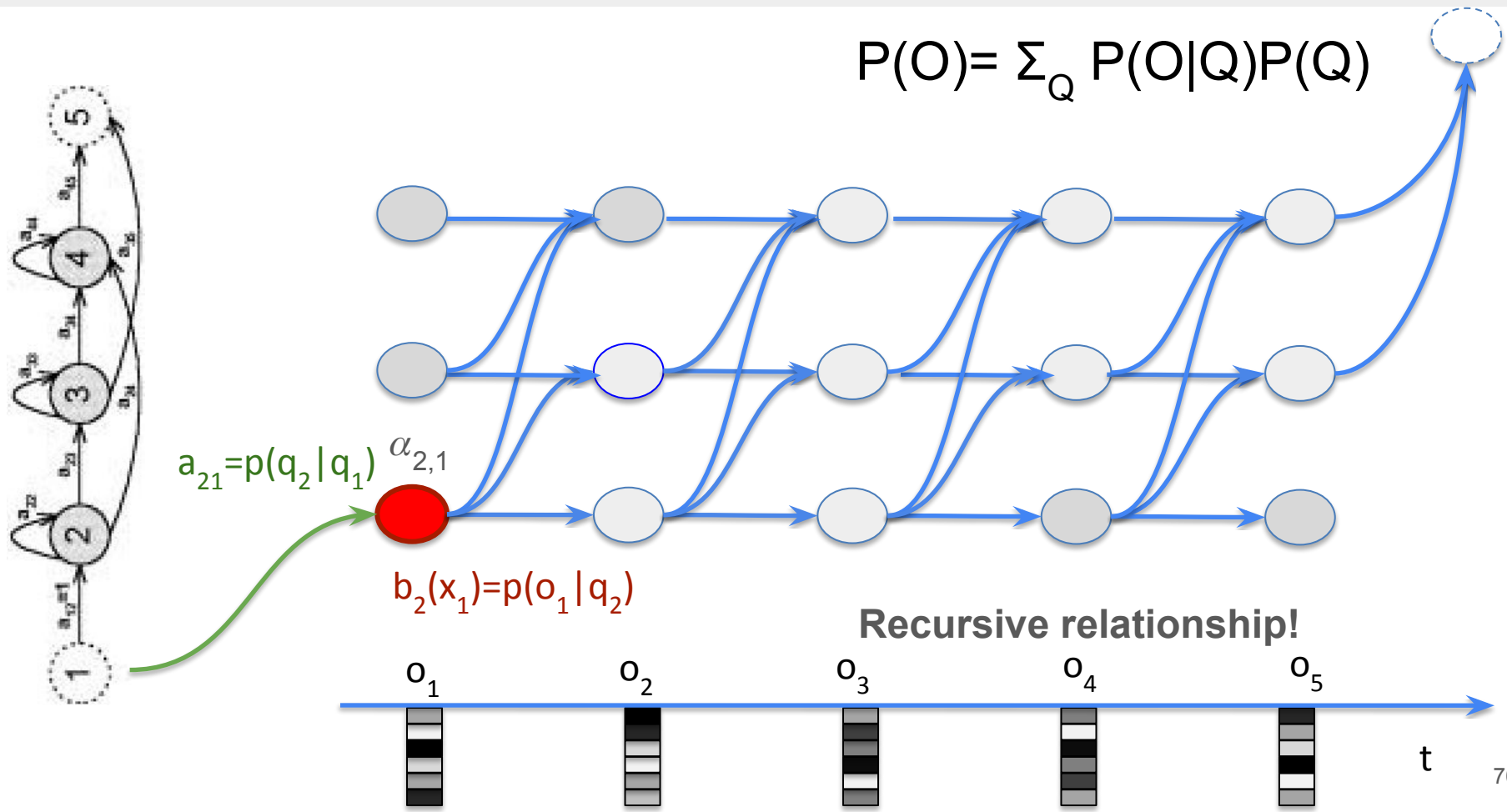
Likelihood and forward algorithm



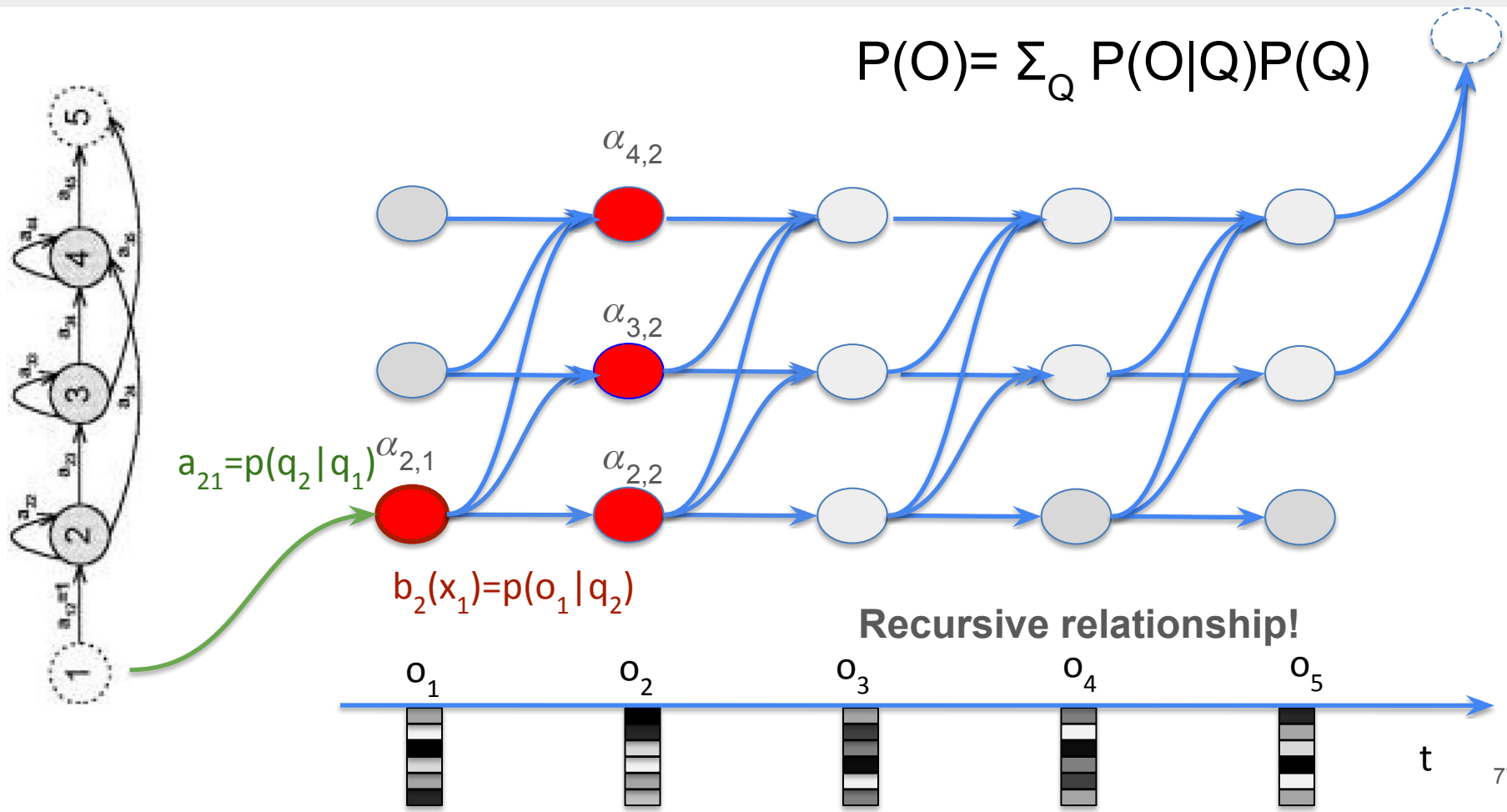
Likelihood and forward algorithm



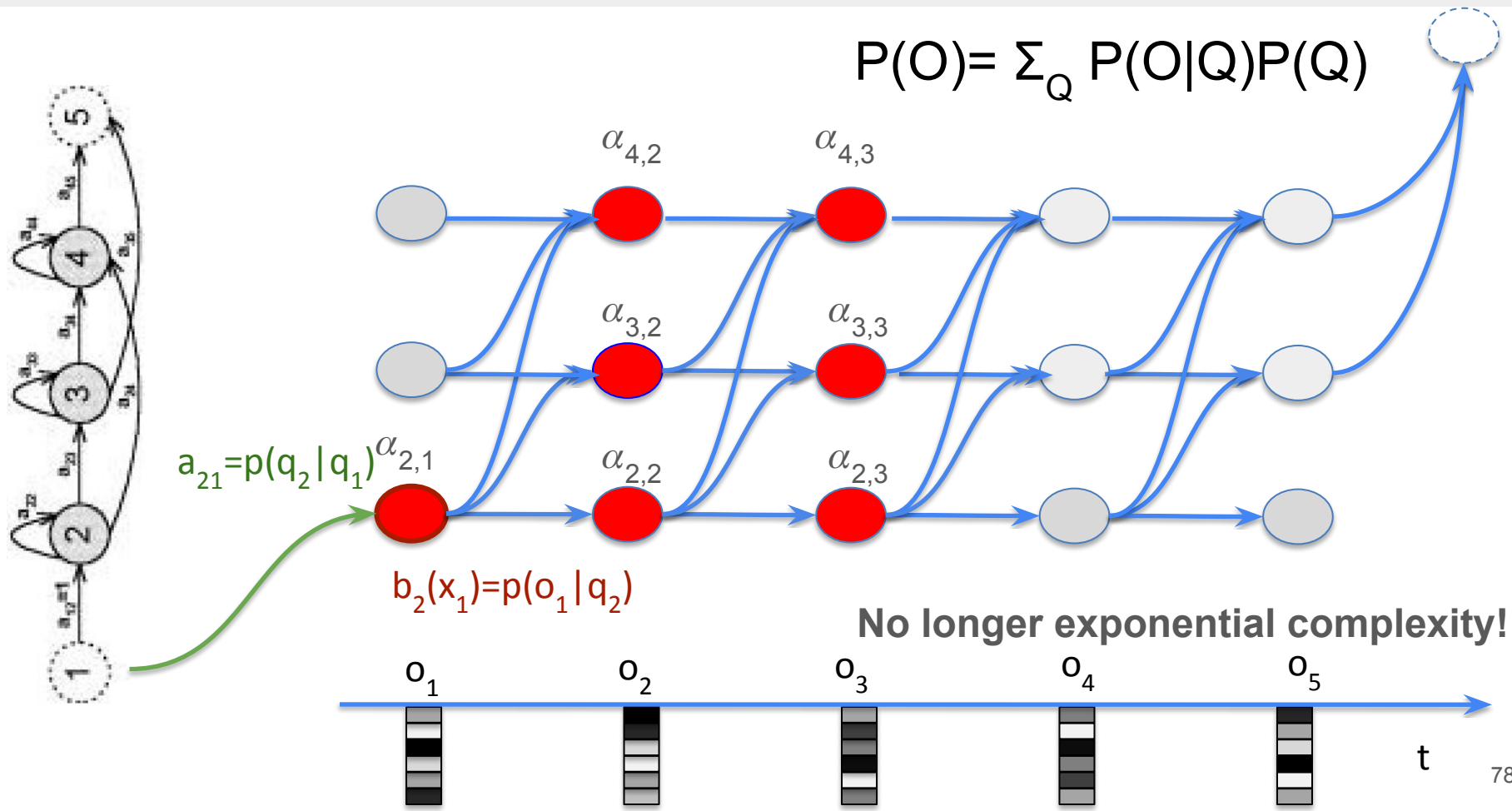
Likelihood and forward algorithm



Likelihood and forward algorithm



Likelihood and forward algorithm



Forward recursive formulation

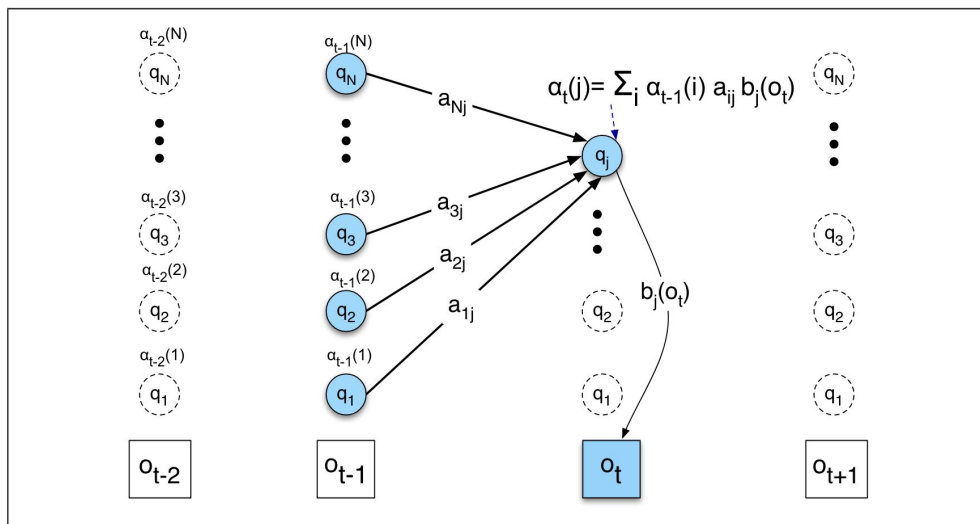


Figure A.6 Visualizing the computation of a single element $\alpha_t(i)$ in the trellis by summing all the previous values α_{t-1} , weighted by their transition probabilities a , and multiplying by the observation probability $b_i(o_t)$. For many applications of HMMs, many of the transition probabilities are 0, so not all previous states will contribute to the forward probability of the current state. Hidden states are in circles, observations in squares. Shaded nodes are included in the probability computation for $\alpha_t(i)$.

1. Initialization:

$$\alpha_1(j) = \pi_j b_j(o_1) \quad 1 \leq j \leq N$$

2. Recursion:

$$\alpha_t(j) = \sum_{i=1}^N \alpha_{t-1}(i) a_{ij} b_j(o_t); \quad 1 \leq j \leq N, 1 < t \leq T$$

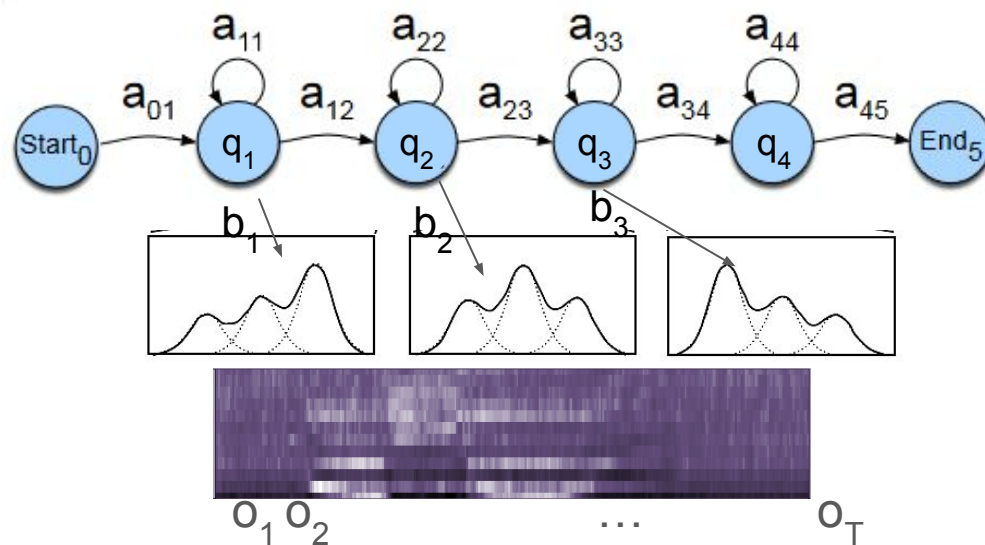
3. Termination:

$$P(O|\lambda) = \sum_{i=1}^N \alpha_T(i)$$

Computation cost in $O(N^2 T)$

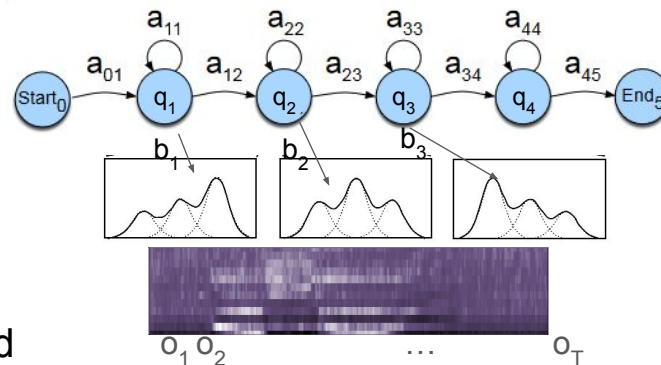
Problem 2: decoding

What is the most probable sequence of states given the observation?



$\text{argmax}_Q p(Q|O) ?$

Problem 2: decoding



$\operatorname{argmax}_Q p(Q|O) ?$

- One possibility:
 - For each hidden state sequence, compute likelihood (forward algo.)
 - Choose the sequence with max likelihood
- But again, not tractable!
- Most common method for HMMs: **Viterbi algorithm**
Also dynamic programming

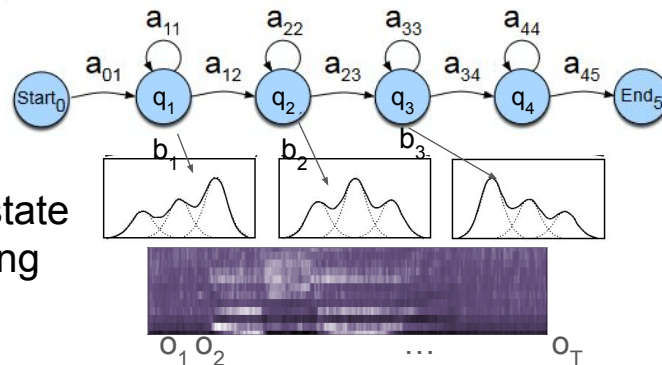
Problem 2: decoding

- Create another treillis
- Cells $v_t(j)$ represent the probability of being in state j after seeing the first t observations and passing through the most probable state sequence

$$v_t(j) = \max_{q_1, \dots, q_{t-1}} P(q_1 \dots q_{t-1}, o_1, o_2 \dots o_t, q_t = j | \lambda)$$

- We compute the Viterbi proba by taking the most probable paths that lead to the current cell

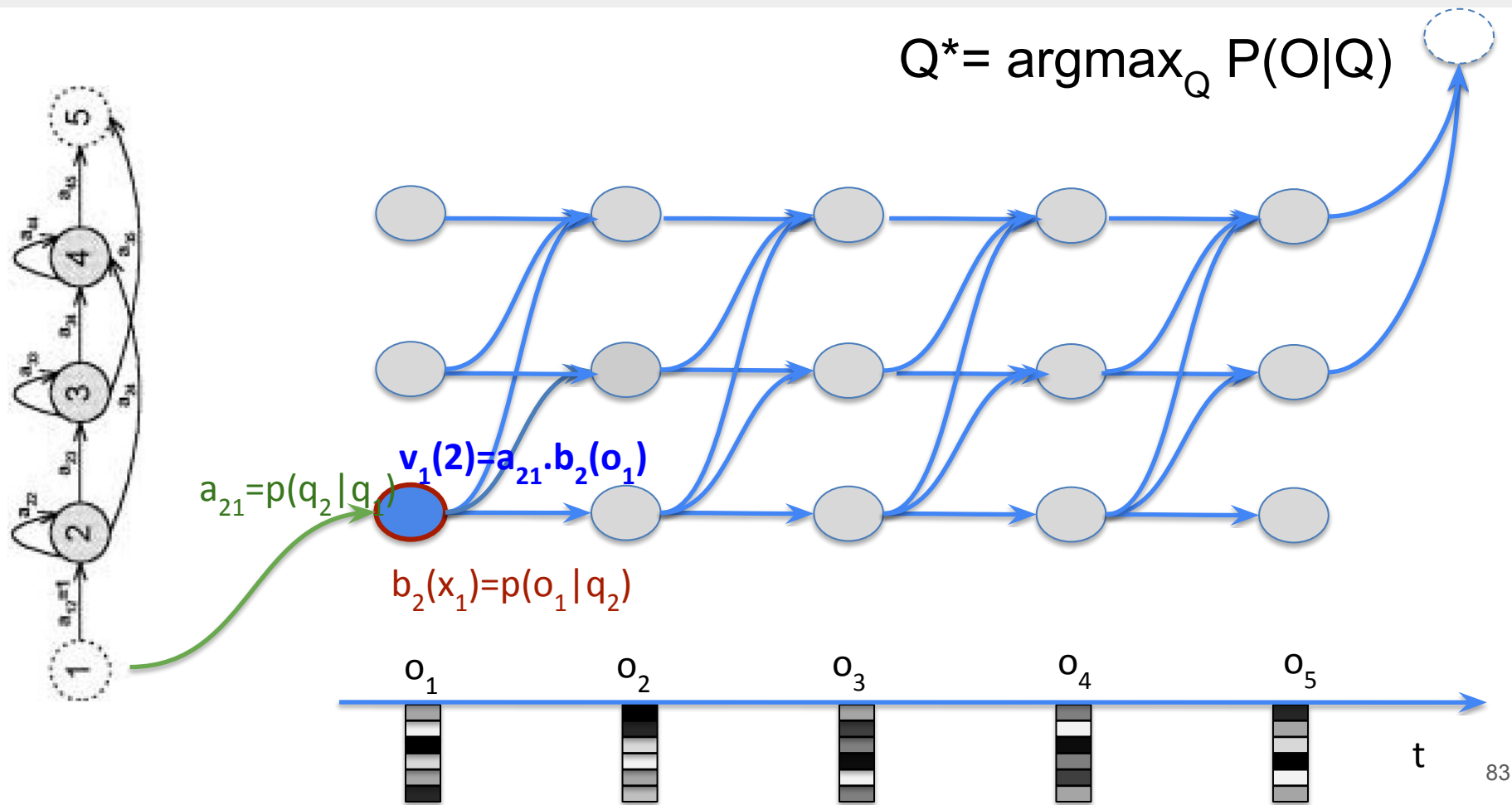
$$v_t(j) = \max_{i=1}^N v_{t-1}(i) a_{ij} b_j(o_t)$$



$\text{argmax}_Q p(Q|O) ?$

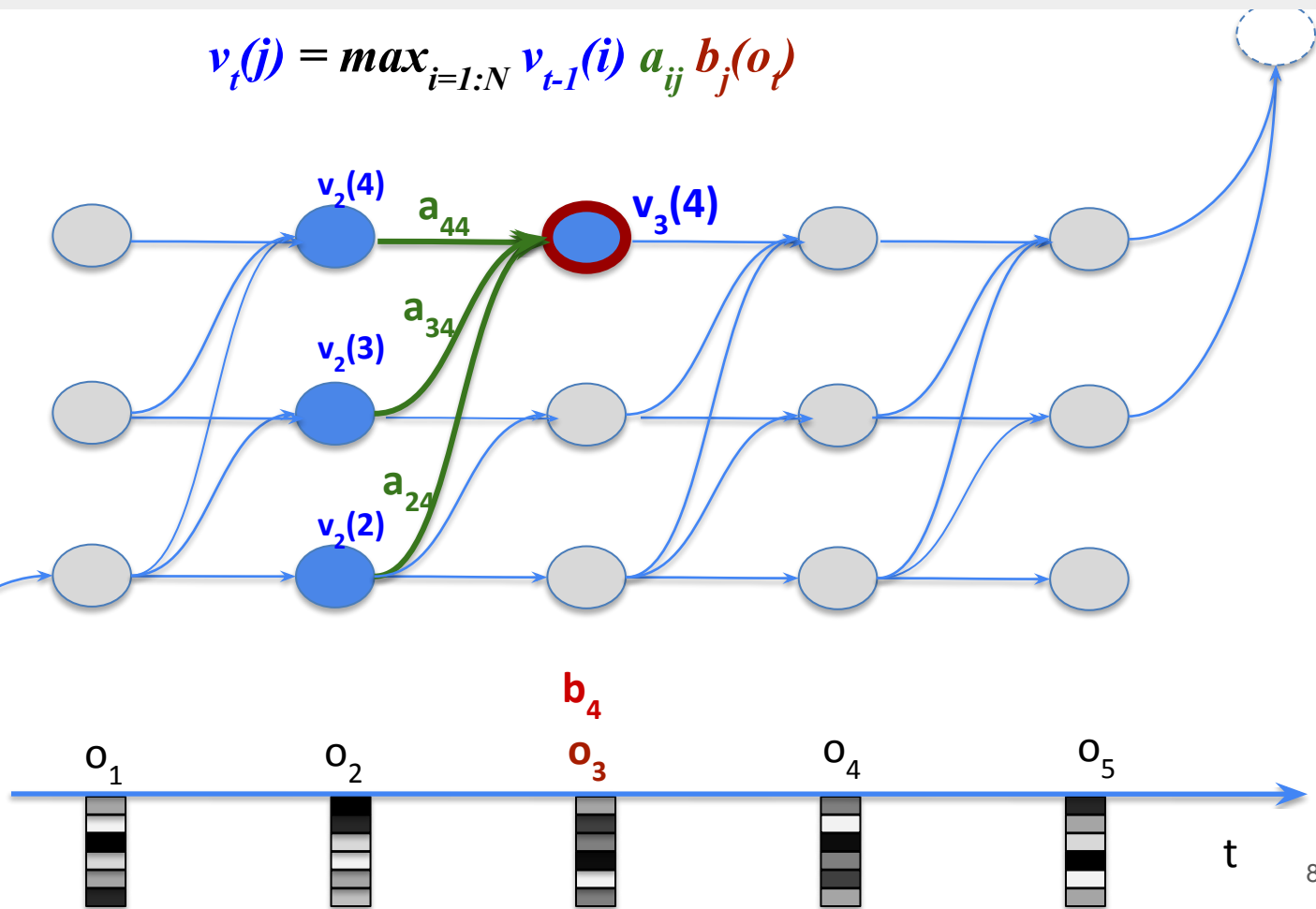
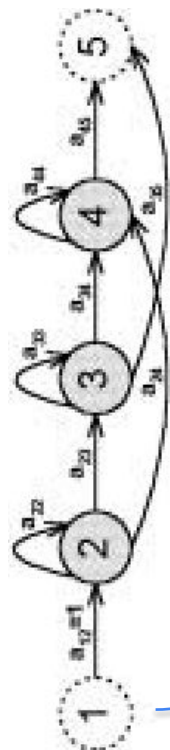
$v_{t-1}(i)$	the previous Viterbi path probability from the previous time step
a_{ij}	the transition probability from previous state q_i to current state q_j
$b_j(o_t)$	the state observation likelihood of the observation symbol o_t given the current state j

Viterbi algorithm

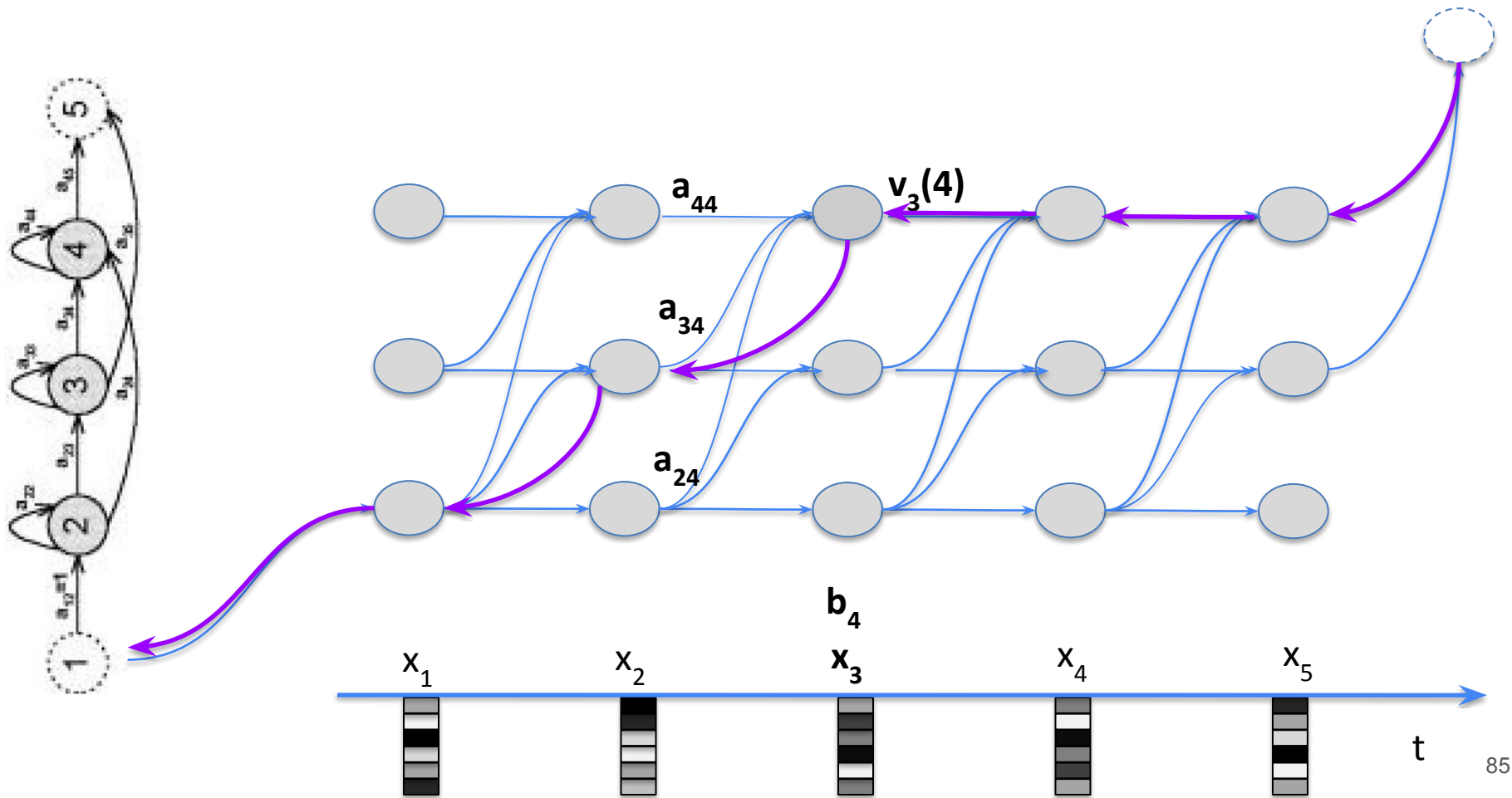


Viterbi algorithm

$$v_t(j) = \max_{i=1:N} v_{t-1}(i) a_{ij} b_j(o_t)$$



Viterbi algorithm: backtrace to recover the hidden states



Viterbi recursive formulation

function VITERBI(*observations* of len T , *state-graph* of len N) **returns** *best-path*, *path-prob*

create a path probability matrix *viterbi*[N, T]

for each state s **from** 1 **to** N **do** ; initialization step

$viterbi[s, 1] \leftarrow \pi_s * b_s(o_1)$

$backpointer[s, 1] \leftarrow 0$

for each time step t **from** 2 **to** T **do** ; recursion step

for each state s **from** 1 **to** N **do**

$viterbi[s, t] \leftarrow \max_{s'=1}^N viterbi[s', t-1] * a_{s', s} * b_s(o_t)$

$backpointer[s, t] \leftarrow \operatorname{argmax}_{s'=1}^N viterbi[s', t-1] * a_{s', s} * b_s(o_t)$

$bestpathprob \leftarrow \max_{s=1}^N viterbi[s, T]$; termination step

$bestpathpointer \leftarrow \operatorname{argmax}_{s=1}^N viterbi[s, T]$; termination step

$bestpath \leftarrow$ the path starting at state $bestpathpointer$, that follows $backpointer[]$ to states back in time

return $bestpath$, $bestpathprob$

Figure A.9 Viterbi algorithm for finding optimal sequence of hidden states. Given an observation sequence and an HMM $\lambda = (A, B)$, the algorithm returns the state path through the HMM that assigns maximum likelihood to the observation sequence.

1. Initialization:

$$v_1(j) = \pi_j b_j(o_1) \quad 1 \leq j \leq N$$

$$bt_1(j) = 0 \quad 1 \leq j \leq N$$

2. Recursion

$$v_t(j) = \max_{i=1}^N v_{t-1}(i) a_{ij} b_j(o_t); \quad 1 \leq j \leq N, 1 < t \leq T$$

$$bt_t(j) = \operatorname{argmax}_{i=1}^N v_{t-1}(i) a_{ij} b_j(o_t); \quad 1 \leq j \leq N, 1 < t \leq T$$

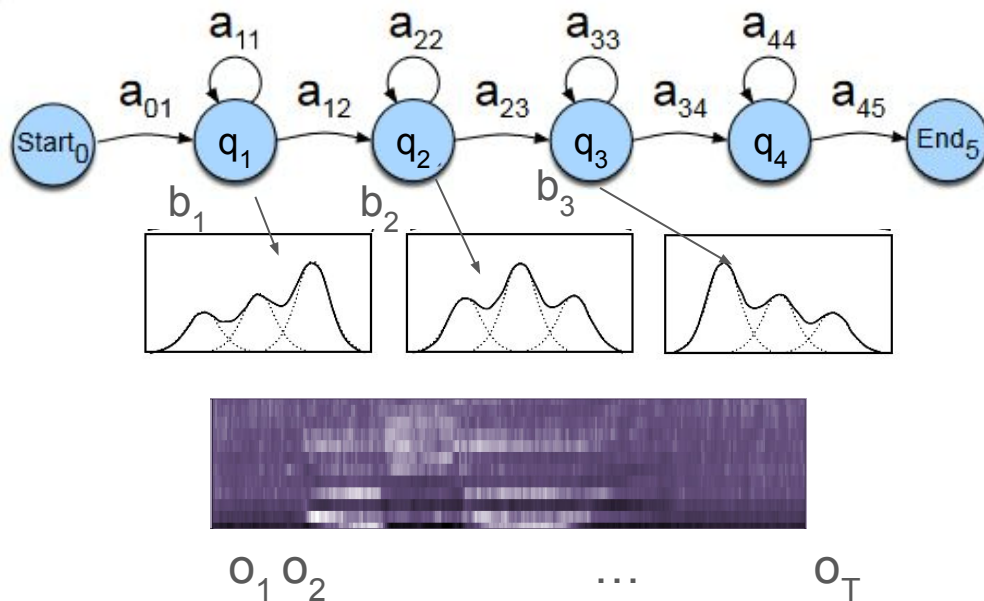
3. Termination:

$$\text{The best score: } P^* = \max_{i=1}^N v_T(i)$$

$$\text{The start of backtrace: } q_T^* = \operatorname{argmax}_{i=1}^N v_T(i)$$

Problem 3: learning

How do we learn the parameters A and B ?



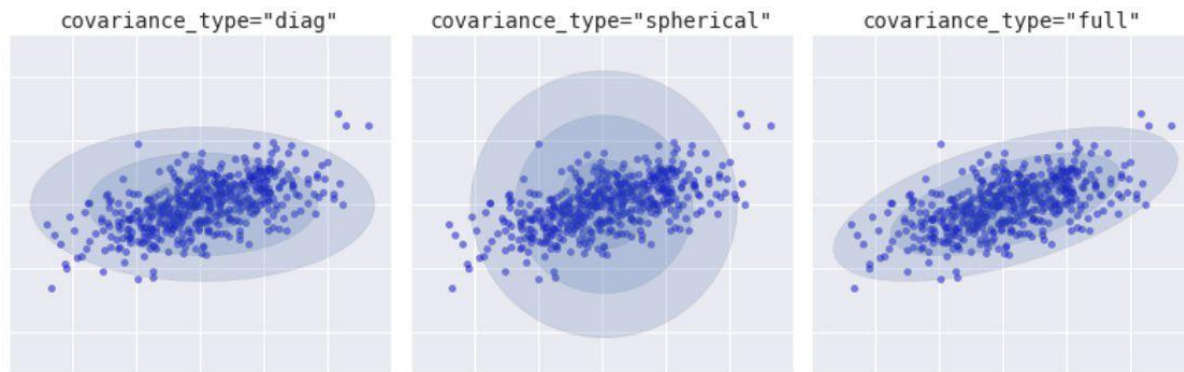
Where do
parameters of
the HMM/GMM
come from??

Parametrization: multivariate gaussian model

Each state is modelled with a multivariate Gaussian that takes in MFCC frames (39 dimension)

$$b_j(o_t) = \frac{1}{(2\pi)^{\frac{D}{2}} |\Sigma|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (o_t - \mu_j)^T \Sigma_j^{-1} (o_t - \mu_j) \right)$$

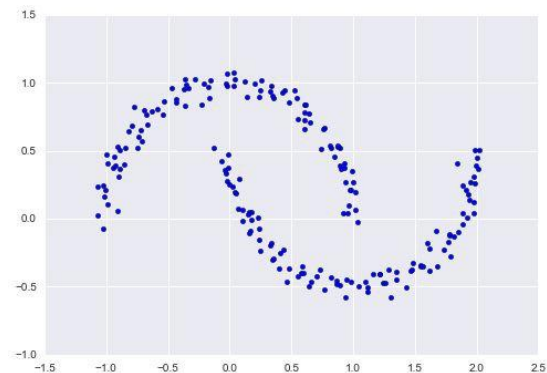
Reminder: MFCCs are decorrelated: use a diagonal covariance



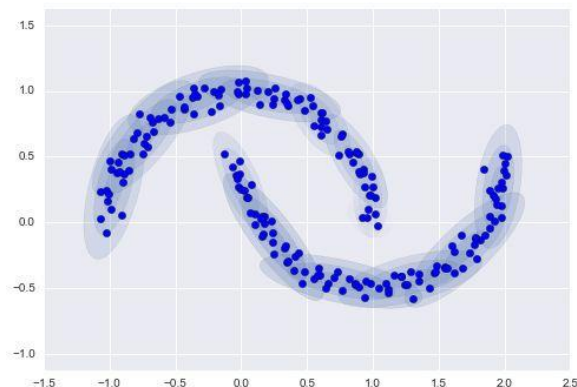
GMM: Mixture of gaussians

You may use several multivariate gaussians with c_k that sums to 1

$$b_j(o_t) = \sum_{m=1}^M c_{jm} \frac{1}{\sqrt{2\pi|\Sigma_{jm}|}} \exp[(x - \mu_{jm})^T \Sigma_{jm}^{-1} (o_t - \mu_{jm})]$$



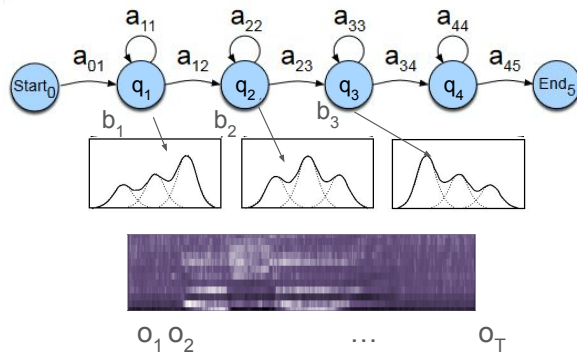
mixture of 16 multivariate gaussians



Forward-backward algorithm

Use the **forward-backward algorithm**

- Special case of Expectation-Maximization
- Compute transition and observation probas even with the actual path hidden
- Iterative algorithm



Where do parameters of the HMM/GMM come from??

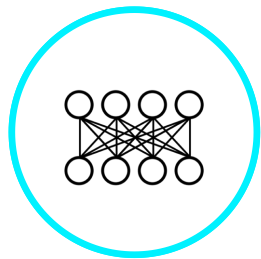
Look at <https://web.stanford.edu/~jurafrsky/slp3/A.pdf> if you want details

Single word HMM recap

Given an database of several recordings of the word 'six' Let us train a HMM-GMM for the word 'six' with $N=4$ states

- 1) Encode your database into MFCC frames
- 2) Train A and B with forward-backward
- 3) At inference:
 - Decode a new utterance: encode in MFCC and use viterbi
 - You get the probability of decoding the word six

(You need to set a threshold for accepting or rejecting the decoding)

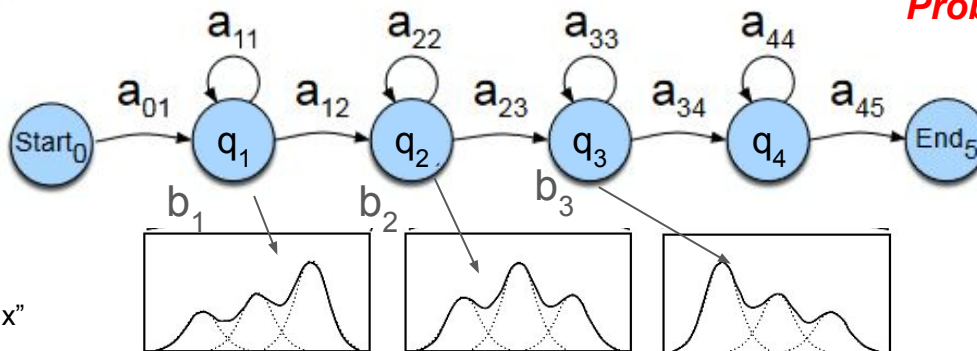


Chap 3. Word modeling

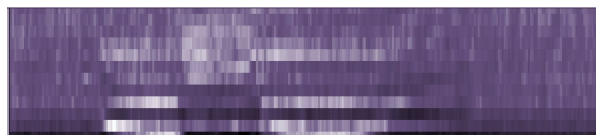
- Large vocabulary ASR
- HMM-GMMs
- WFSTs

How about more words?

Each word is represented by $N \times P \times K + N$ parameters (eg, $20 \times 10 \times 40 + 20 = 8020$)
Problem?



HMM/GMM for the word "six"



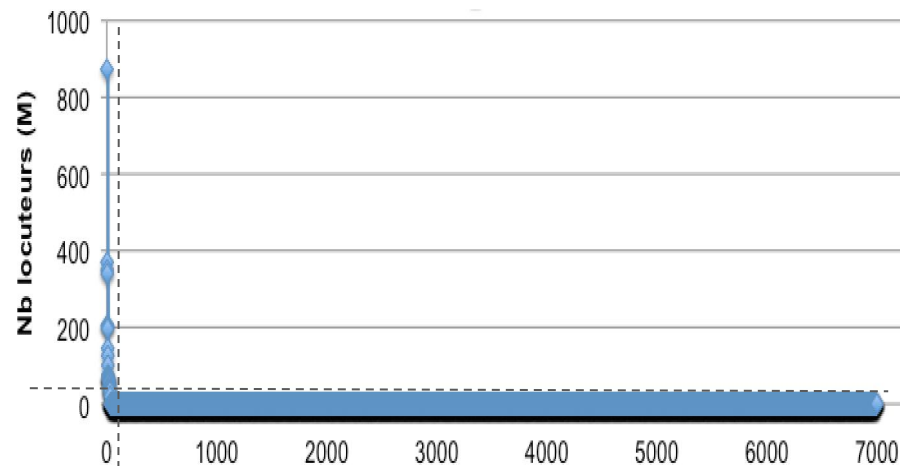
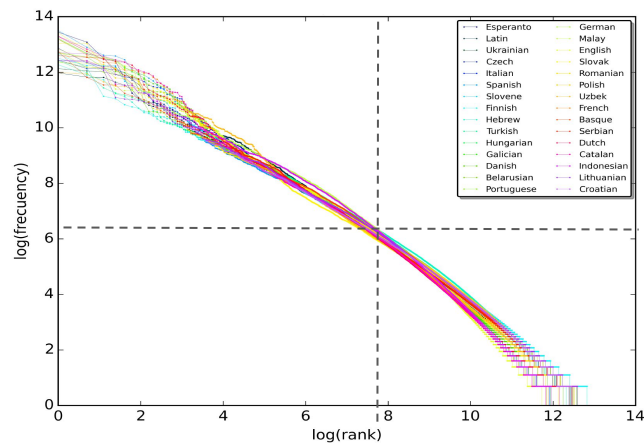
$O_1 O_2 \dots O_T$

Power laws in language



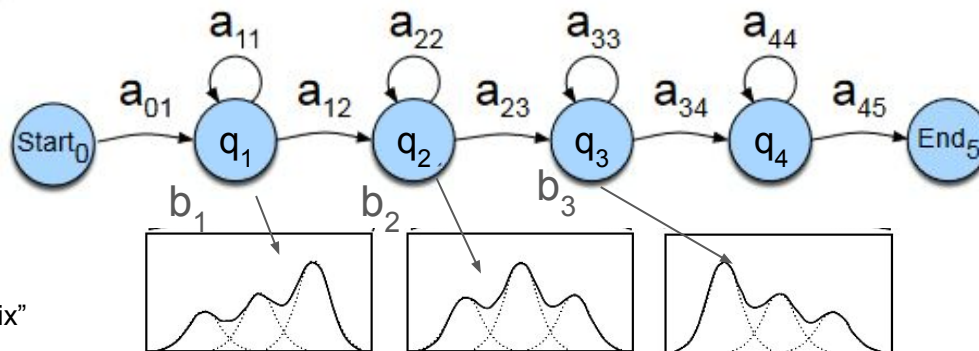
$$\log(f) \sim -\log(r)$$

$$f \sim 1/r$$

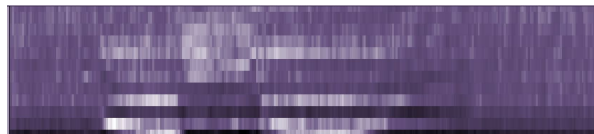


How about more words?

How to learn the parameters for each words, when most words are seen only once or twice in a corpus?



HMM/GMM for the word "six"



$O_1 O_2 \dots O_T$

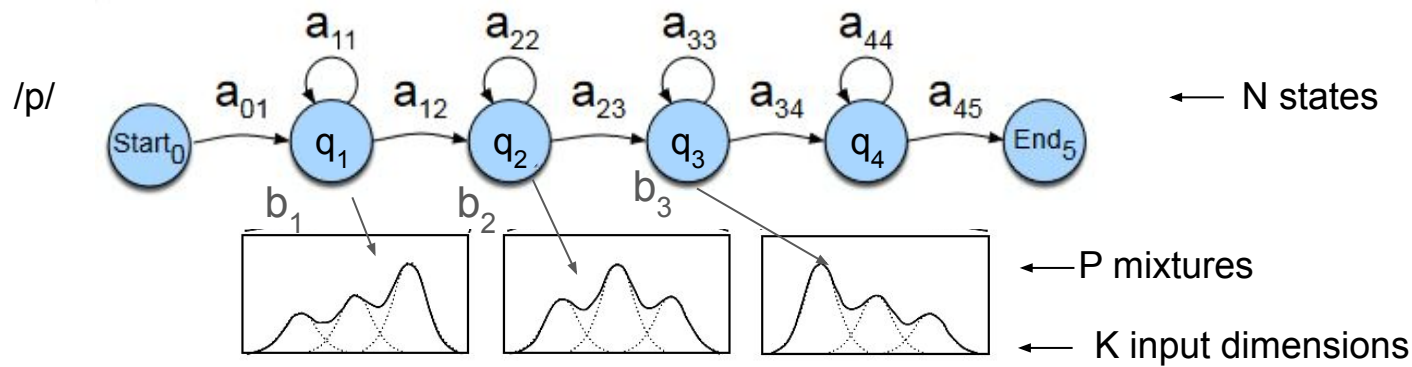
Phoneme inventories: between 20 and 200

Central Rotokas	Bilabial	Alveolar	Velar
Voiceless	p	t	k
Voiced	b ~ β	d ~ r	g ~ ɣ

Ubykh		Labial		Alveolar			Postalveolar					Velar				Uvular					Glottal
							laminal closed		laminal		apical										
		plain	phar.	plain	lab.	lat.	plain	lab.	plain	lab.		pal.	plain	lab.	phar.	pal.	plain	lab.	phar.	phar. & lab.	
Plosive	voiceless	p	pʰ	t	tʷ							kʲ	k	kʷ		qʲ	q	qʷ	qʰ	qʰʷ	
	voiced	b	bʰ	d	dʷ							gʲ	g	gʷ							
	ejective	pʼ	pʰʼ	tʼ	tʷʼ							kʲʼ	kʼ	kʷʼ		qʲʼ	qʼ	qʷʼ	qʰʼ	qʰʷʼ	
Affricate	voiceless			ʈs			ʈʃ		ʈʂ	ʈʂʷ	ʈʂʰ										
	voiced			ɖz			ɖʒ		ɖʂ	ɖʂʷ	ɖʂʰ										
	ejective			ʈsʼ			ʈʃʼ		ʈʂʼ	ʈʂʷʼ	ʈʂʰʼ										
Fricative	voiceless	f		s		ɬ	ʃ	ʃʷ	ɕ	ɕʷ	ɕʰ		x			χʲ	χ	χʷ	χʰ	χʰʷ	h
	voiced	v	vʰ	z			ʒ	ʒʷ	ʐ	ʐʷ	ʐʰ		ɣ			ʁʲ	ʁ	ʁʷ	ʁʰ	ʁʰʷ	
	ejective					ɬʼ															
Nasal		m	mʰ	n																	
Approximant						l						j		w	wʰ						
Rhotic				r																	

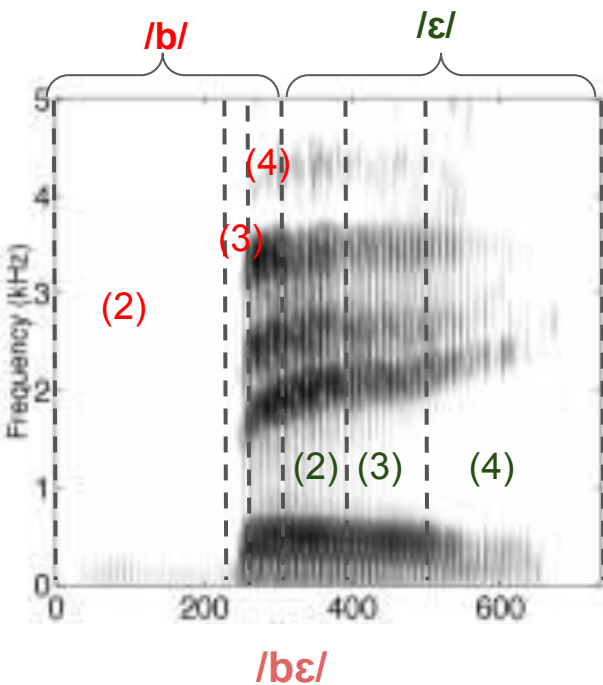
Phone-based ASR

For each phoneme in a language, build an HMM for it



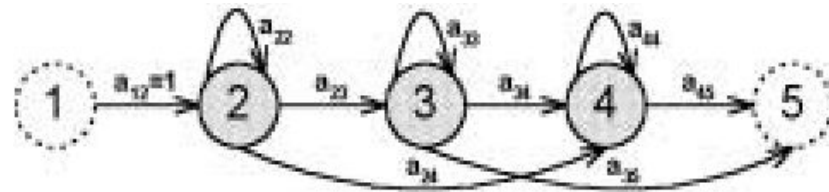
How many states for phonemes?

Phone-based ASR

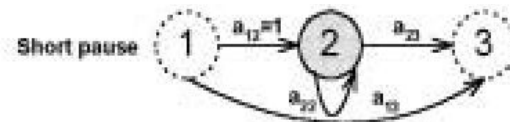
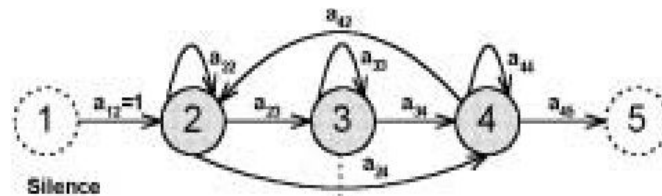
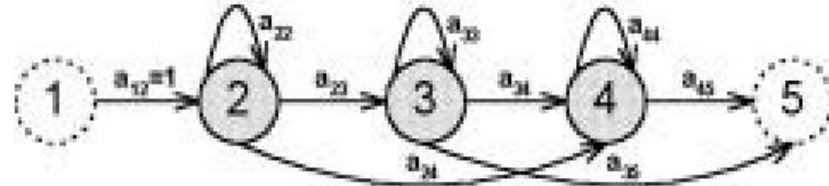


Tristate
phonemes

/b/



/ε/

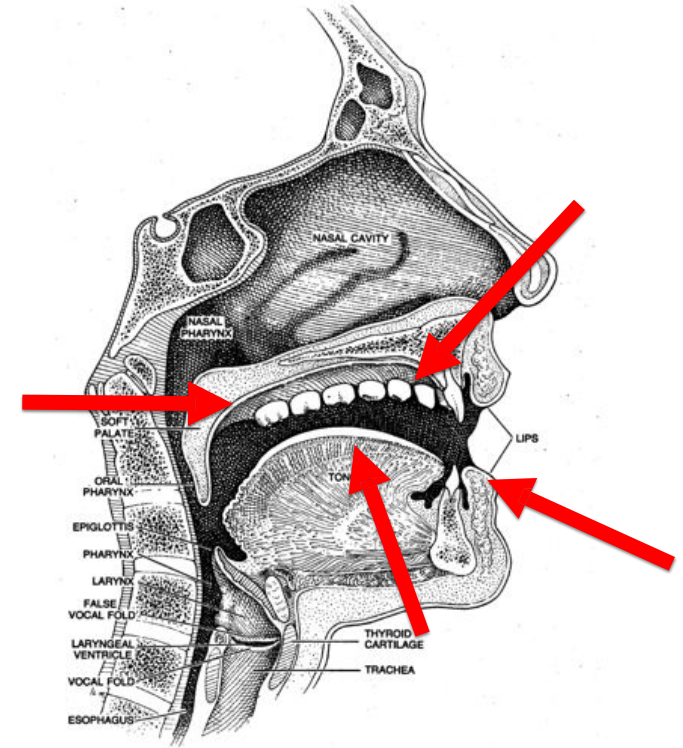


Speech sounds are produced by a biomechanical system

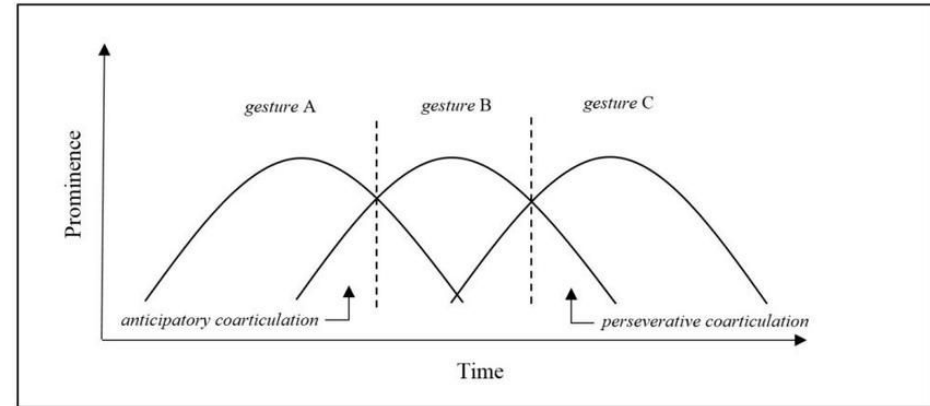
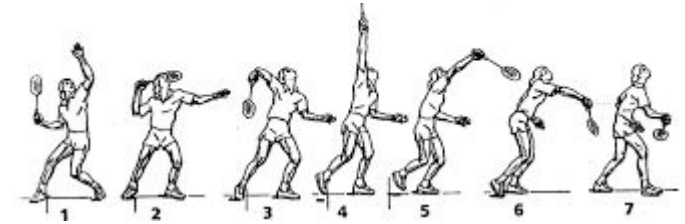
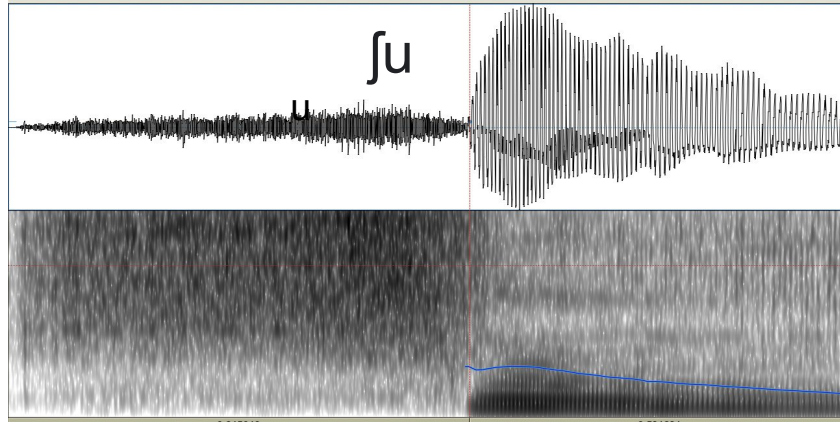
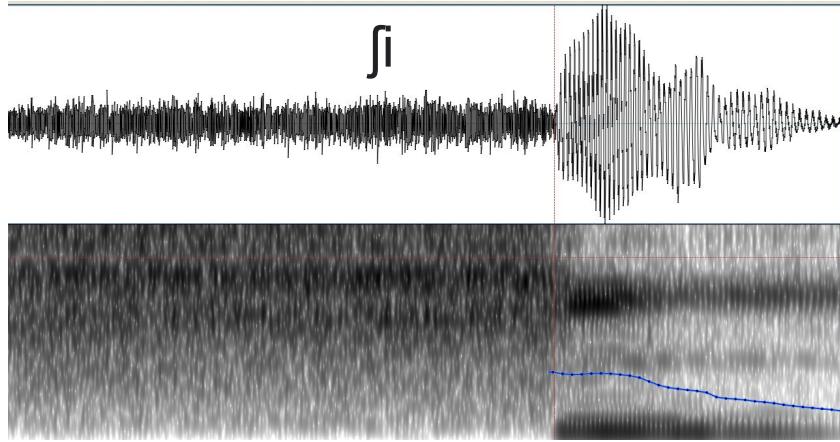


- Air is expelled from the lungs through the trachea
- Passes through vocal cords and resonate in the larynx
- Then through the nose and/or the mouth
- Vocal tract: the « articulators »: lips, tongue, teeth, palate, etc. The way we control the articulators defines the sound we produce.

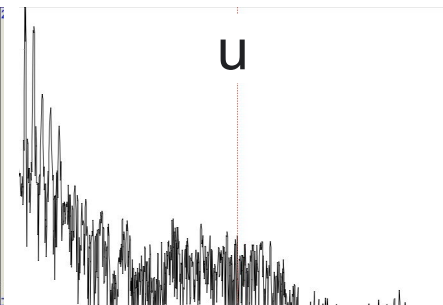
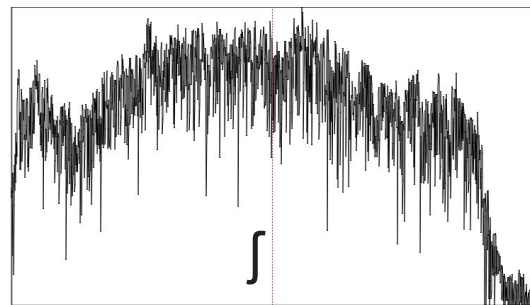
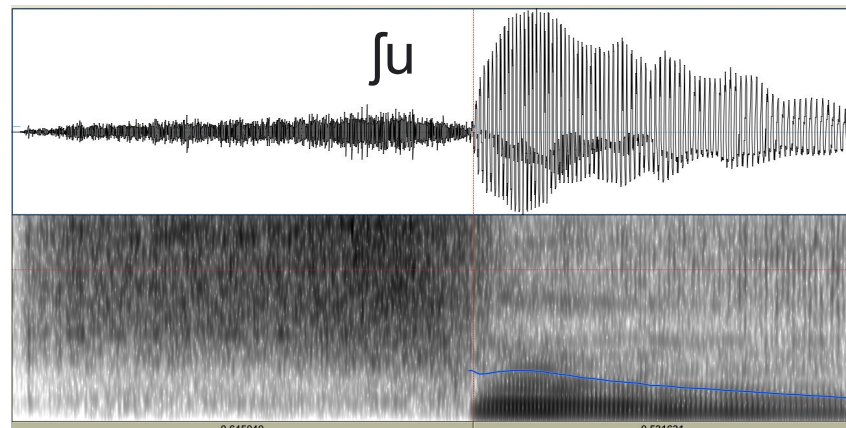
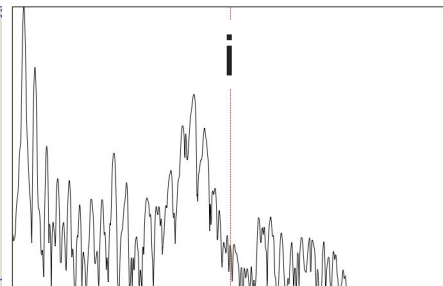
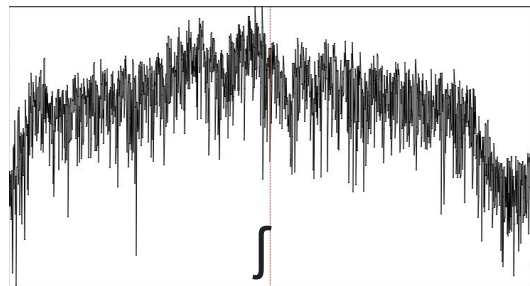
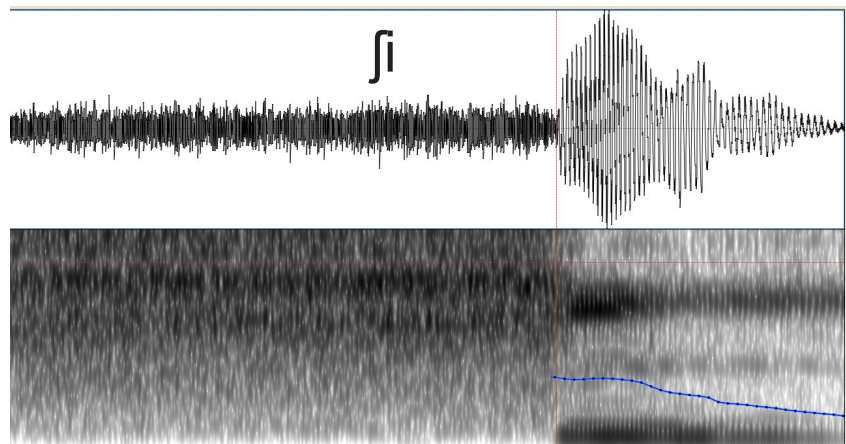
→ **Biomechanical motion: minimization of energy**



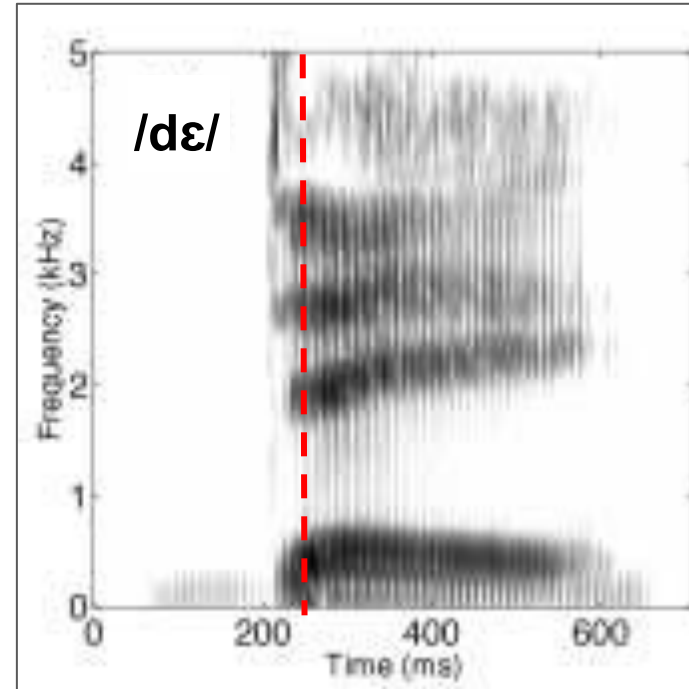
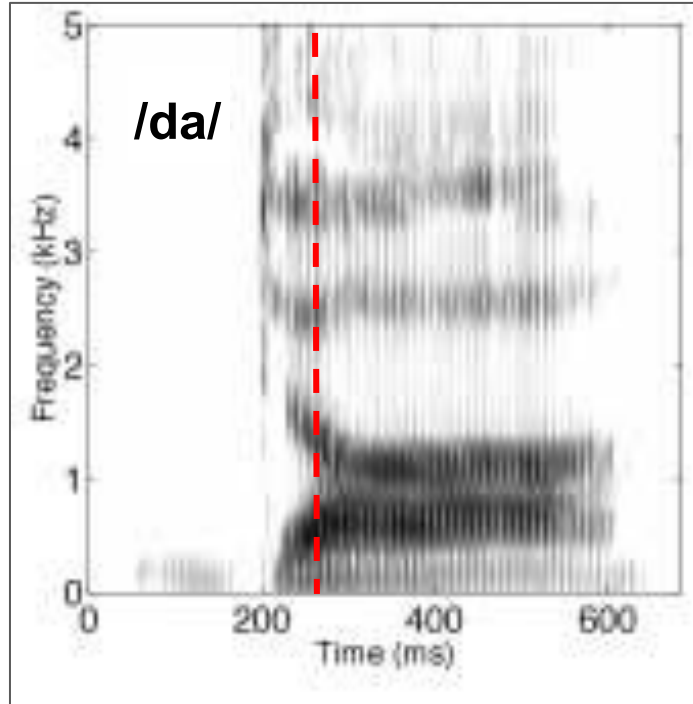
Coarticulation: a consequence of biomechanical control



Coarticulation: a consequence of biomechanical control



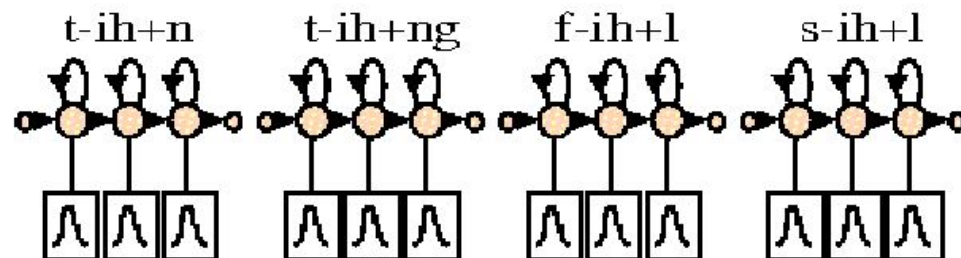
Coarticulation: a consequence of biomechanical control



Contextual-phone-based ASR

Conventional triphones

For each triphone in a language, build an HMM for it



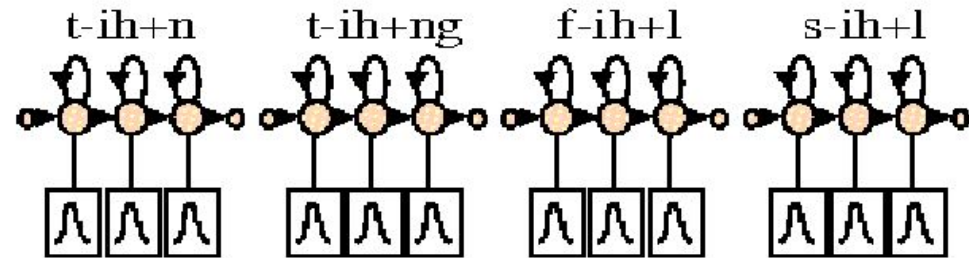
Problem: for English (40 phonemes), 64000 triphones

→ **Power law, again. How do we deal with rare triphones?**

Contextual-phone-based ASR

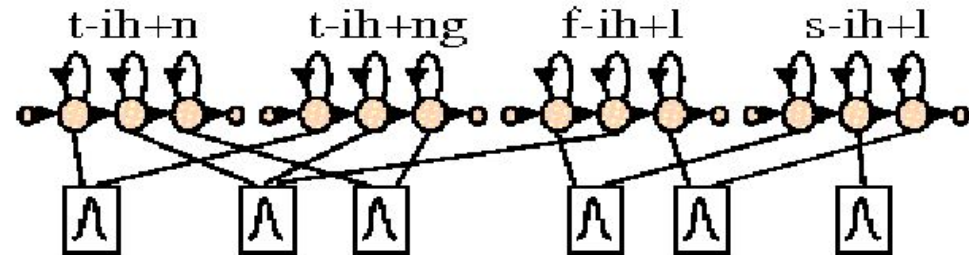
For each triphone in a language, build an HMM for it, and merge them

Conventional triphones



More accurately, merge similar phone states

Tied triphones



Contextual-phone-based ASR

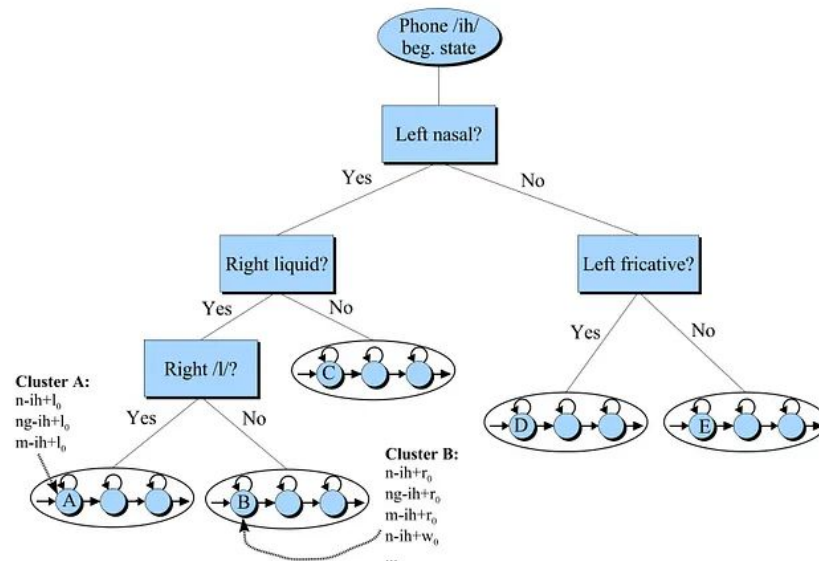
For each triphone in a language, build an HMM for it, and merge them

More accurately, merge similar phone states

Decision trees

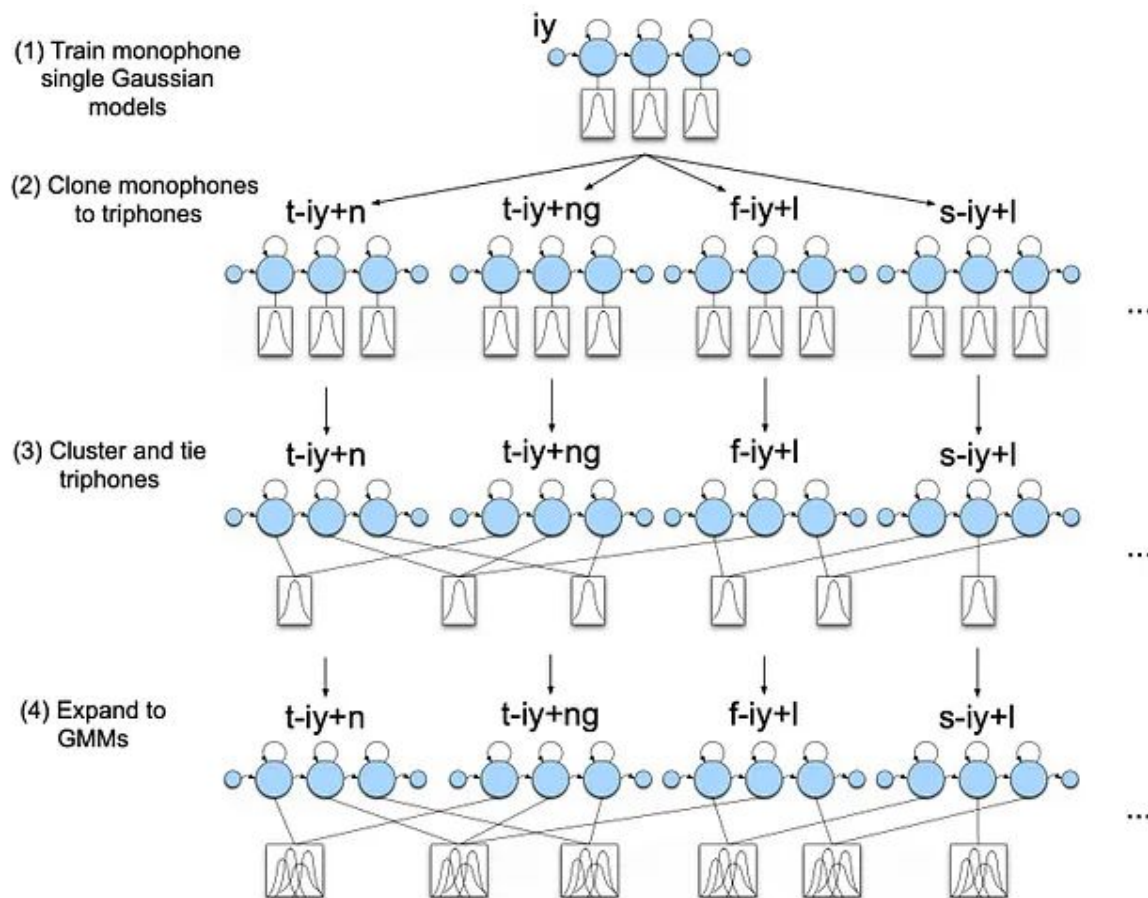
- represent phonemes in terms of phonetic features (eg, voiced vs unvoiced, labial vs velar, etc)
- Split phonemes into contextual phones by asking questions

Data driven: merge triphones states



Contextual-phone-based ASR

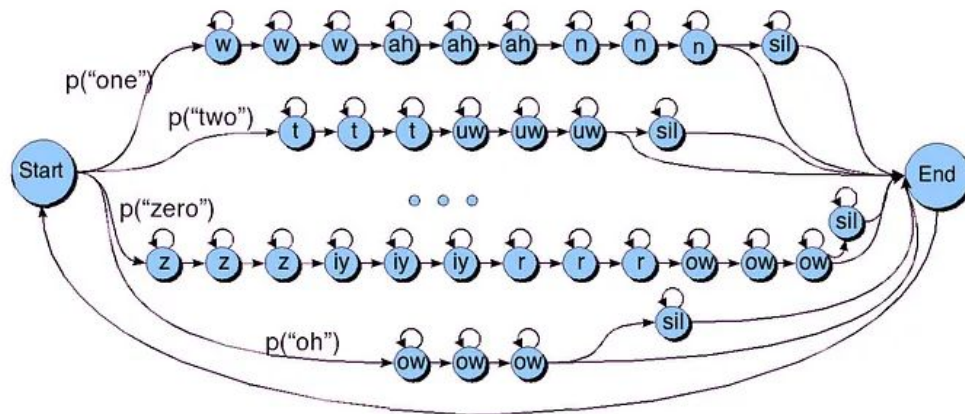
Full recipe:



Contextual-phone-based ASR

For each contextual phone build an HMM for it

Make a list of words and build a large HMM out of it by concatenating the relevant contextual phones



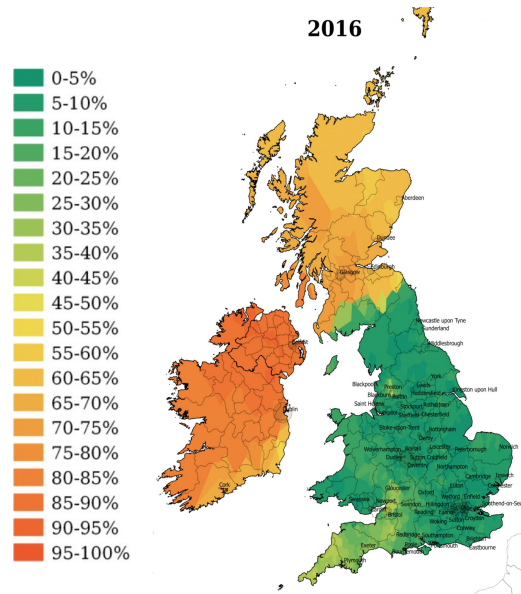
Are we done?

Linguistic “variation”



anagement maagement maanagement
maangement magagement magement
mamagement mamangement manaagement manaement
managaement manageement manageemnt managegment
managemaent managemant managememt managemen managemenet
managementt managemet managemetn managemnent managemnet
managemnt managemrnt managemt managenent managenment managent
managerment managheement managmeent managmement managment managment
manament manamgement mananement manangment manasgement
manegement manegment mangaement mangagement mangagment
mangament mangement manggement mangment
mangmt menagement mgmt mgnt
mnagement mngmnt mngmt

Do you pronounce the “r” in “arm” ?



Phonetic variation

Pronunciations of the word 'eleven' in the state of Pennsylvania, USA

ɤ ʌ lɛ ˈvən
ɤ ʌ lɛvən
ɤ ʌ ɛvən
ə ˈlɛ ˈvən
ɤ ʌ lɛvən
əlɛvən
ə ˈɛvn
əɛbm
ɤ ʌ ˈlɛvn
lɛvən
əɛv ˈvən

əɛvən
ɤ ˈlɛvm

...

(total= 220 variants)



<https://www.youtube.com/watch?v=HbDnxzrbxn4>

Phonological/Phonetic transformations

What are you doing? 'wʌtʃə'duɪn 

I can inquire. 'aɪkən'kwɪə 

Did you eat yet? 'dʒɪtʃe? 

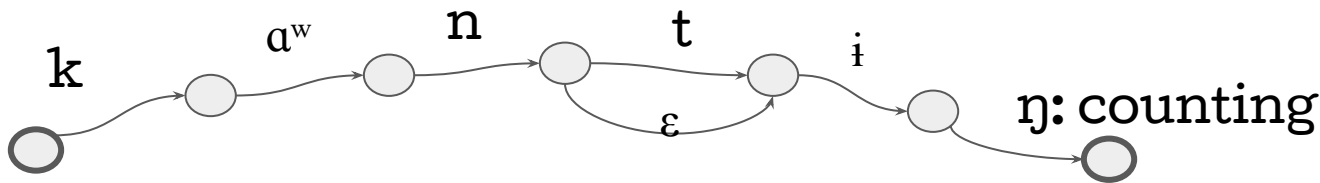
I don't believe him. aɪ'doʊbə'lɪvm 

We ought to have come. wɪ'ɔʊt'kʌm 

Pronunciation models

Using a pronunciation dictionary:

orthography	SAMPA	IPA
counting	kA_wnt1N	kɑ ^w ntɪŋ
counting(1)	kA_wn1N	kɑ ^w nɪŋ
amortization	@mOrt@zeS@n	əmərtəzeʃən
amortization(1)	{mOrt@zeS@n	æmərtəzeʃən
amortization(2)	@mOrtA_jzeS@n	əmərtəʤzeʃən
..etc...		



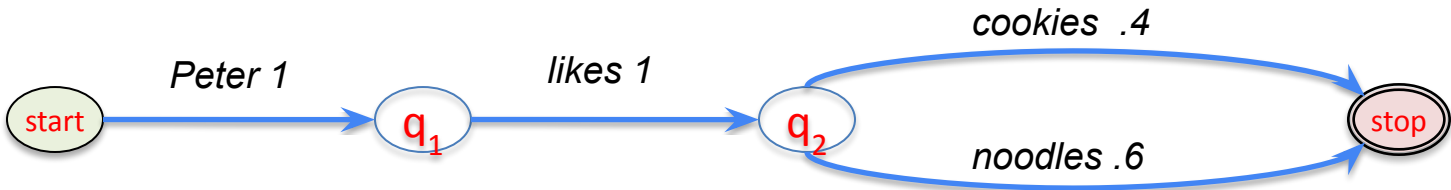
(weighted) Finite State Transducers



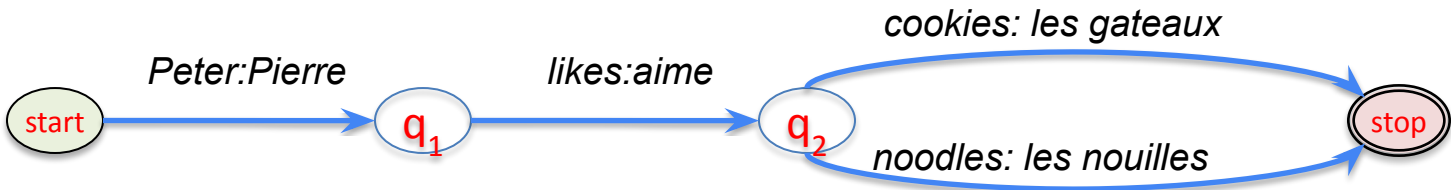
Finite State
Acceptor (FSA)



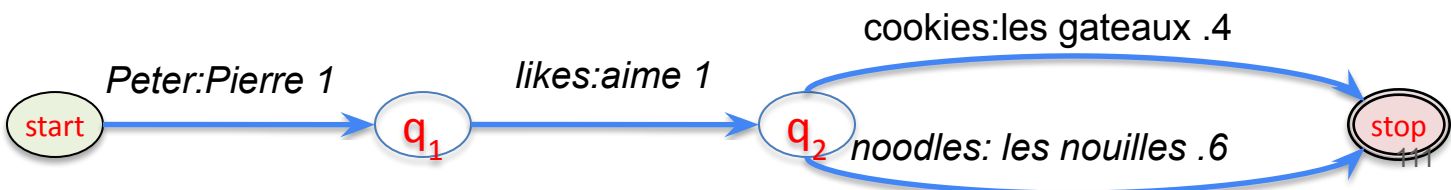
Weighted Finite
State Acceptor
(WFSA)



Finite State
Transducer (FST)



Weighted Finite State
Transducer (WFST)



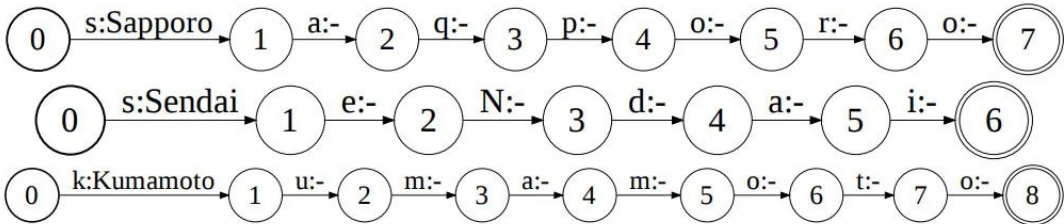
Operations over wFSTs



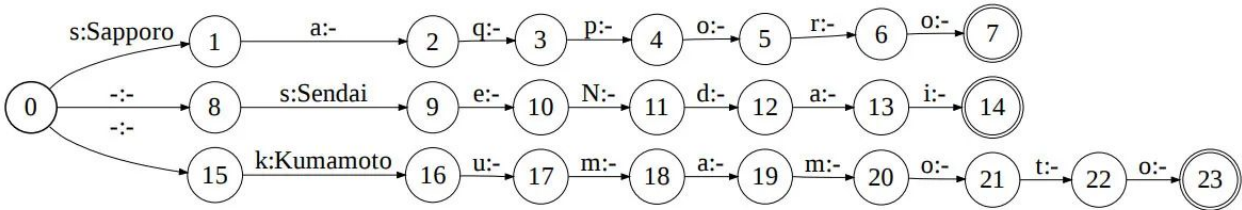
<https://jonathan-hui.medium.com/speech-recognition-weighted-finite-state-transducers-wfst-a4ece08a89b7>

sum of the union

$$\llbracket T1 \oplus T2 \rrbracket (x, y) = \llbracket T1 \rrbracket (x_1, y_1) \oplus \llbracket T2 \rrbracket (x_2, y_2)$$



→

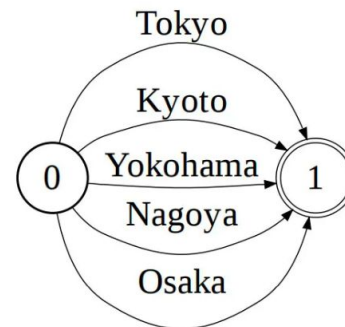
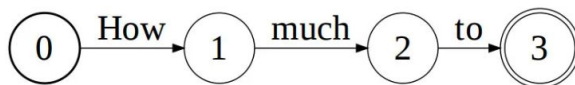


-:- Null Transitions that do not consume or generate symbols (allow deterministic transition if ambiguity occurs).

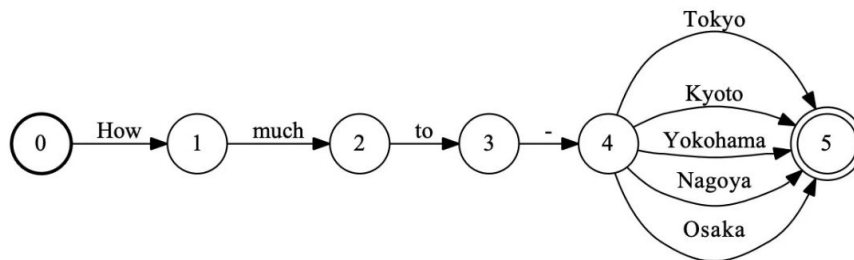
Operations over wFSTs



$$\llbracket T1 \otimes T2 \rrbracket (x, y) = \bigoplus_{\substack{x = x_1 x_2 \\ y = y_1 y_2}} \llbracket T1 \rrbracket (x_1, y_1) \otimes \llbracket T2 \rrbracket (x_2, y_2)$$



→

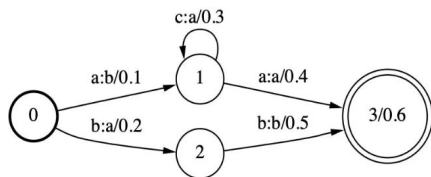


Operations over wFSTs



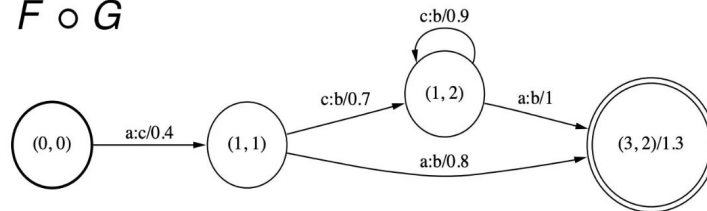
$$\llbracket T_1 \circ T_2 \rrbracket ((x, y) = \bigoplus_z \llbracket T_1 \rrbracket (x, z) \otimes \llbracket T_2 \rrbracket (z, y)$$

F



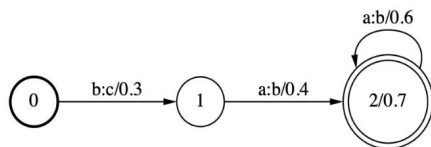
$aa \rightarrow ba$
 $aca \rightarrow baa$

$F \circ G$



$aa \rightarrow cb$
 $aca \rightarrow cbb$

G



$ba \rightarrow cb$
 $baa \rightarrow cbb$

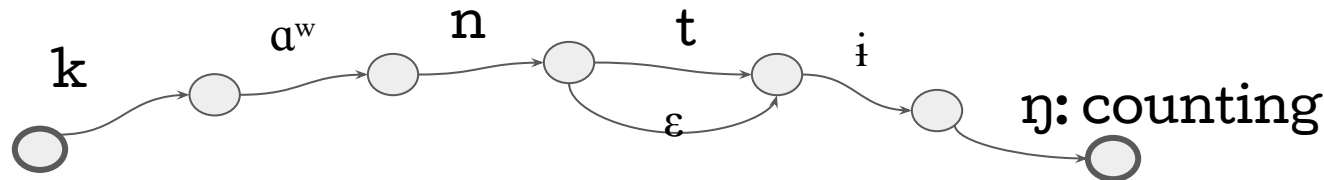
Pronunciation models

DETOUR



Using a pronunciation dictionary:

orthography	SAMPA	IPA
counting	kA_wntlN	kɑ ^w ntɪŋ
counting(1)	kA_wnlN	kɑ ^w nɪŋ
amortization	@mOrt@zeS@n	əməɹtəzeʃən
amortization(1)	{mOrt@zeS@n	æməɹtəzeʃən
amortization(2)	@mOrtA_jzeS@n	əməɹtəjzeʃən
..etc...		



- Pronunciations for a word can be modelled as a wFST
- How to combine this with an HMM?

The graph illustrates a network flow problem with nodes 0 to 24. Node 0 is the source, and node 24 is the sink. The edges and their associated capacities and costs are as follows:

- Node 0 to Node 1: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 2: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 3: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 4: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 5: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 6: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 7: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 8: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 9: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 10: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 11: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 12: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 13: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 14: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 15: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 16: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 17: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 18: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 19: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 20: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 21: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 22: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 23: Capacity 0.00115, Cost 0.00115
- Node 1 to Node 24: Capacity 0.00115, Cost 0.00115
- Node 2 to Node 3: Capacity 0.00115, Cost 0.00115
- Node 3 to Node 4: Capacity 0.00115, Cost 0.00115
- Node 4 to Node 5: Capacity 0.00115, Cost 0.00115
- Node 5 to Node 6: Capacity 0.00115, Cost 0.00115
- Node 6 to Node 7: Capacity 0.00115, Cost 0.00115
- Node 7 to Node 8: Capacity 0.00115, Cost 0.00115
- Node 8 to Node 9: Capacity 0.00115, Cost 0.00115
- Node 9 to Node 10: Capacity 0.00115, Cost 0.00115
- Node 10 to Node 11: Capacity 0.00115, Cost 0.00115
- Node 11 to Node 12: Capacity 0.00115, Cost 0.00115
- Node 12 to Node 13: Capacity 0.00115, Cost 0.00115
- Node 13 to Node 14: Capacity 0.00115, Cost 0.00115
- Node 14 to Node 15: Capacity 0.00115, Cost 0.00115
- Node 15 to Node 16: Capacity 0.00115, Cost 0.00115
- Node 16 to Node 17: Capacity 0.00115, Cost 0.00115
- Node 17 to Node 18: Capacity 0.00115, Cost 0.00115
- Node 18 to Node 19: Capacity 0.00115, Cost 0.00115
- Node 19 to Node 20: Capacity 0.00115, Cost 0.00115
- Node 20 to Node 21: Capacity 0.00115, Cost 0.00115
- Node 21 to Node 22: Capacity 0.00115, Cost 0.00115
- Node 22 to Node 23: Capacity 0.00115, Cost 0.00115
- Node 23 to Node 24: Capacity 0.00115, Cost 0.00115

Et voilà!

A°P

Pronunciation models

DETOUR
ENDS

Using a pronunciation dictionary:

orthography	SAMPA	IPA
counting	kA_wnt1N	kɑ ^w ntɪŋ
counting(1)	kA_wn1N	kɑ ^w nɪŋ
amortization	@mOrt@zeS@n	əmərtəzeʃən
amortization(1)	{mOrt@zeS@n	æmərtəzeʃən
amortization(2)	@mOrtA_jzeS@n	əmərtəʤzeʃən
..etc...		

→ **But what if you don't have pronunciations for all of your words?**
Or you want to add words?

1. Phonological rules (hand crafted)

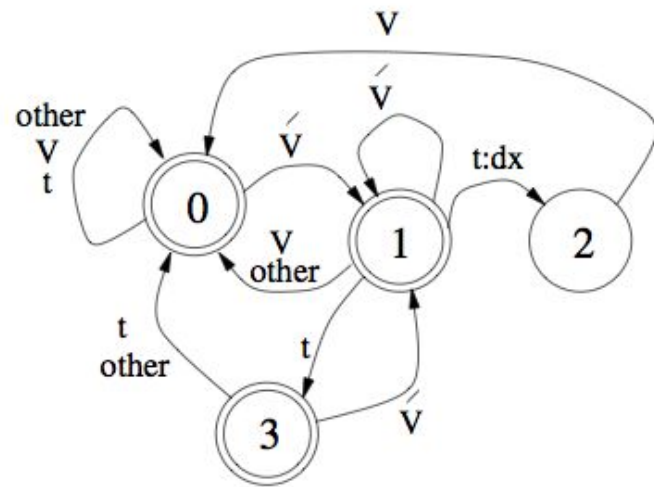
Ex: english flapping

$$/t/ \rightarrow [dx] \quad / \quad [+vowel] \text{ — } \begin{bmatrix} +vowel \\ -stress \end{bmatrix} .$$

Input symbols: ax t eh n y uw ey t ax d

State sequence: ① → ① → ① → ② → ① → ① → ① → ② → ③ → ① → ①

Output symbols: ax t eh n y uw ey dx ax d



Represent rules as a wFST!

Name	Rule	Example	Prob
Syllabic Rules*			
Syllabic n	[ax ix] n → en	button	.35
Syllabic m	[ax ix] m → em	bottom	.32
Syllabic l	[ax ix] l → el	bottle	.72
Syllabic r	[ax ix] r → axr	butter	.77
Flapping	[tcl dcl] [t d] → dx /V ____ [ax ix axr]	button	.87
Flapping-r	[tcl dcl] [t d] → dx /V r ____ [ax ix axr]	barter	.92
H-voicing	hh → hv / [+voice] ____ [+voice]	ahead	.92
L-deletion	l → Ø/ ____ y [ax ix axr]	million	n/a
Gliding	iy → y / ____ [ax ix axr]	colonial	n/a
Nasal-deletion	[n m ng] → Ø/ ____ [-voice -consonant]	rant	n/a
Function words			
h-deletion	h → Ø/ # ____	he, him	n/a
w-deletion	w → Ø/ # ____	will, would	n/a
dh-deletion	dh → Ø/ # ____	this, those	n/a
Dental-deletion	[tcl dcl] [t d] → Ø/ [+vowel] ____ [th dh]	breadth	n/a
Final dental-deletion	([tcl dcl]) [t d] → Ø/ [+cons +continuant] ____ #	soft (as)	n/a
Slur	ax → Ø/ [+consonant] ____ [r l n] [+vowel]	camera	n/a
Stressed slur	[+vowel +stress] r → er	warts	n/a
Pre-stress contraction	ax → Ø/ [+cons] ____ [+cons] [+vowel +stress]	senility	n/a
Ruh-reduction	r ax → er / [-word bdry] ____ [-word bdry]	separable	n/a
Transitional stops			
t-introduction	Ø → tcl / [+dental +nasal] ____ [+fricative]	prin[t]ce	n/a
t-deletion	[tcl] → Ø/ [+dental +nasal] ____ [+fricative]	prints	n/a

2. Phonological rules (data driven)

Using a phone recognizer to learn a pronunciation dictionary

a	baseball					game
ax	b	ey	s	b	ao	l g ey m
ax	b	eh ey	s	b	el	g eh m
ax	b	eh ey	s	b	el	g eh m

word sequence

canonical phone sequence

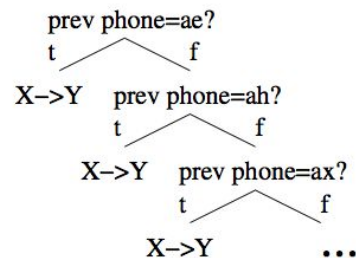
aligned surface phones

proposed new pronunciations

Inducing pronunciation rules

- trigram models
- decision trees

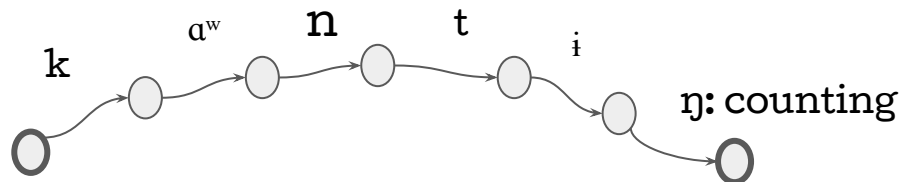
$$P(S^i) = \prod_{j=1}^n (P(s_j^i | c_{j-1}, c_j, c_{j+1})) .$$



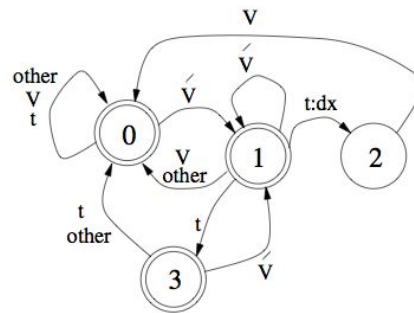
→ **Learning a pronunciation wFST**

Improved recipe

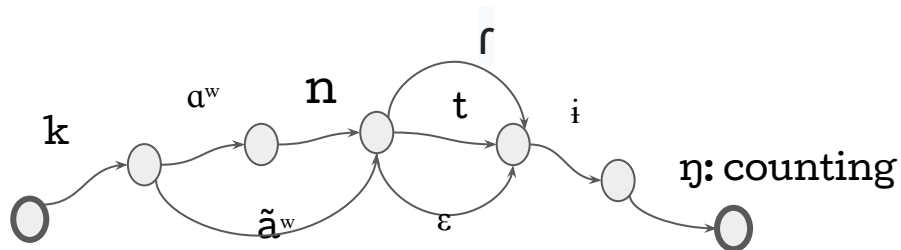
1. Build a lexicon wFST (L)



2. Build or learn a pronunciation wFST (F)

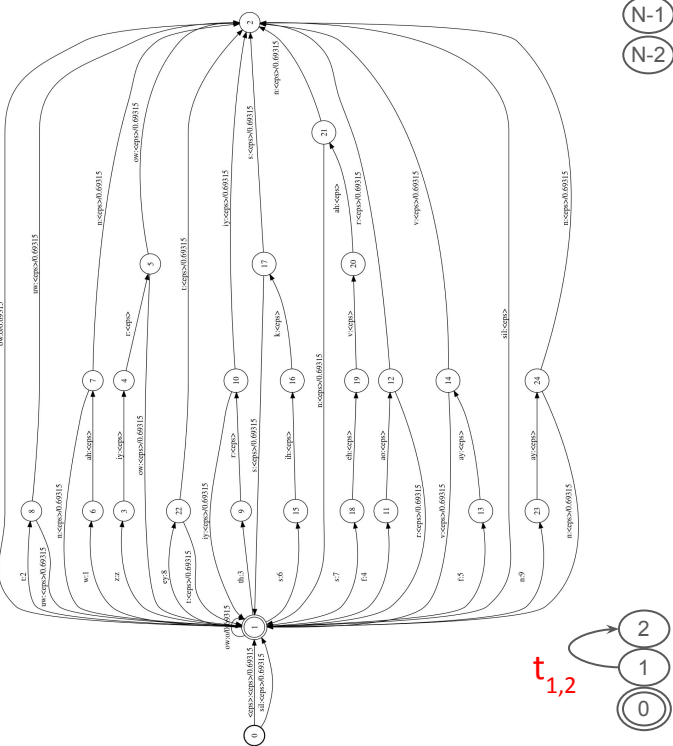


3. Compose: $P = F \circ L$

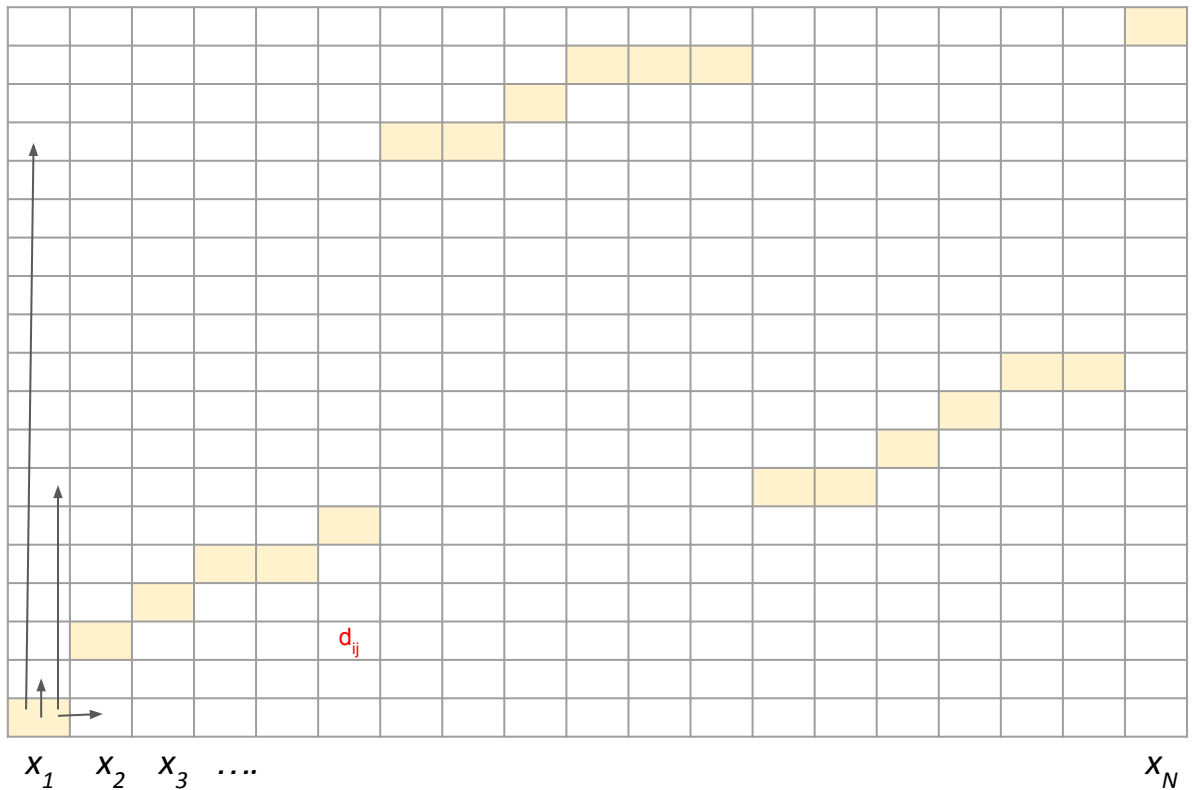


Improved recipe

K states



- (N-1)
- (N-2)



Reminder: the fundamental equation of ASR

$X_T = x_1, \dots, x_T$ A sequence of features frames

$W_M = w_1, \dots, w_M$ A sequence of words

$$\hat{W} = \operatorname{argmax}_W P(W|X)$$

$$\hat{W} = \operatorname{argmax}_W P(X|W)P(W)$$



ACOUSTIC MODEL



LANGUAGE MODEL

Reminder: the fundamental equation of ASR

$$\hat{W} = \operatorname{argmax}_W P(W|X)$$

$X_T = x_1, \dots, x_T$ A sequence of features frames

$W_M = w_1, \dots, w_M$ A sequence of words

$P_m = p_1, \dots, p_m$ A sequence of phonemes

$$\hat{W} = \operatorname{argmax}_W P(X|W)P(W)$$

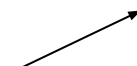


ACOUSTIC MODEL



LANGUAGE MODEL

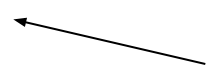
$$P(X|N)P(W|N)P(W)$$



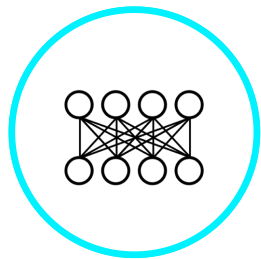
ACOUSTIC MODEL



PRONUNCIATION MODEL



LANGUAGE MODEL



Chap 4. Language modeling for ASR

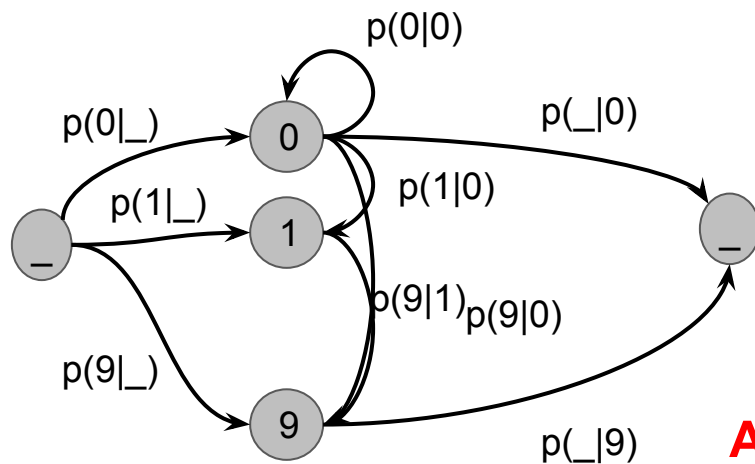
HMMs (again!)

$$P(W_n \mid W_1, W_2, \dots, W_{n-1}) \approx P(W_n \mid W_{n-1})$$

Reminder: bigram language models

$$P(W_n \mid W_1, W_2, \dots, W_{n-1}) \approx P(W_n \mid W_{n-1})$$

Eg, with digits zero-ten



Another wFST!!

Reminder: trigram language models

$$P(W_n \mid W_1, W_2, \dots, W_{n-1}) \approx P(W_n \mid W_{n-2}, W_{n-1})$$

$$P(W_1, \dots, W_L) = \prod_{i=1}^{L+1} P(W_i \mid W_{i-2}, W_{i-1})$$

$$P(W_i \mid W_{i-2}W_{i-1}) = \frac{c(W_{i-2}W_{i-1}W_i)}{c(W_{i-2}W_{i-1})} \quad \text{Count in the corpus}$$

A larger wFST!!

Reminder: smoothing, etc

Unobserved ngrams

- most ngrams have been seen 0 time
- problem grows exponentially with n
- true 0s versus sampling
- Solutions:
 - Smoothing
 - backoff, interpolation, clustering

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

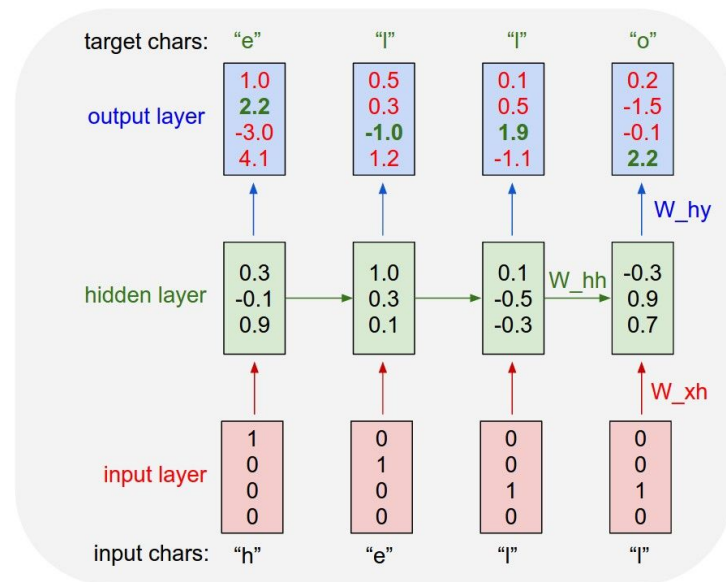
$$P_{\text{Laplace}}(w_i) = \frac{c_i + 1}{N + V}$$

$$c_i^* = (c_i + 1) \frac{N}{N + V}$$

$$\begin{aligned}\hat{P}(w_n | w_{n-2} w_{n-1}) &= \lambda_1 P(w_n | w_{n-2} w_{n-1}) \\ &\quad + \lambda_2 P(w_n | w_{n-1}) \\ &\quad + \lambda_3 P(w_n)\end{aligned}$$

Preview for next course: RNN language models

- RNNs can approximate $P(W)$ better than ngram models by relying on indefinite amount of past information
- But, the Markov assumption no longer holds
- Hence Viterbi is no longer possible (greedy search, beam search)



<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Summary so far: HMM-GMM for large vocabulary ASR

Using phonemes as building blocks for Acoustic Modeling (A)

- Modeling temporal dynamics with three states HMMs
- Modeling coarticulation by replacing phonemes with tied triphones

Building a pronunciation model for words (P)

- Modeling pronunciation variations through wFSTs

Building a language model (G)

- Using n-gram wSFT

Combining everything

- A single model (a large wFST!): $G \circ P \circ A$
- Decoding using Viterbi
- Learning : forward-backward (A), by hand (P), by counting (G)

A lot of linguistic expertise

- List of phonemes
- Pronunciation dictionary
- List of phonological rules

Reasonable amounts of annotations (50h+)

- Parallel corpus of speech + transcription

Assumptions and approximations

- Phones states = stationary emission probability
- Local context assumption (Markov property)
- Linguistic knowledge on phonemes, rules, etc.



Chap 5.

Why is ASR difficult?

Handling variability in speech

- Word Error Rate
- Noisy environment
- Gender
- Speaker
- Accents

Word Error Rate (WER)

Reference metric in ASR

Smallest amount of Insertion, Deletion, Substitution applied on the prediction to match the true transcription

Also called Levenshtein Distance, Edit Distance

$$WER = \frac{S + D + I}{N}$$

N = number of words in the référence

WER is in percentage

WER can be > 100%

Example:

$$WER = \frac{3}{4} = 75\%$$

Reference: SHOW ME THE INTERFACE

Hypothesis: I SHOW ME FACE

Alignment: I D S

Recursive formulation

$$\text{lev}(a, b) = \begin{cases} \max(|a|, |b|) & \text{si } \min(|a|, |b|) = 0, \\ \text{lev}(a-1, b-1) & \text{si } a[0] = b[0], \\ 1 + \min \begin{cases} \text{lev}(a-1, b) \\ \text{lev}(a, b-1) \\ \text{lev}(a-1, b-1) \end{cases} & \text{sinon.} \end{cases}$$

Solved by dynamic programming in $O(|a|*|b|)$

Noisy environment, different accentuations, biological differences

Two ways to address them all:

- Bigger database: include different recording conditions, genders, ethnicities
- Data augmentation: reverb, add gaussian noise, time stretch, pitch shift,...

Can we do better than that?

Difficulty of noisy environments

- Real life VS studio recordings: huge drop in WER
- No real method to address this (apart bigger database + data aug)

Dataset	wav2vec 2.0 Large (no LM)	Whisper Large V2	RER (%)
LibriSpeech Clean	2.7	2.7	0.0
Artic	24.5	6.2	74.7
Common Voice	29.9	9.0	69.9
Fleurs En	14.6	4.4	69.9
Tedlium	10.5	4.0	61.9
CHiME6	65.8	25.5	61.2
VoxPopuli En	17.9	7.3	59.2
CORAAL	35.6	16.2	54.5
AMI IHM	37.0	16.9	54.3
Switchboard	28.3	13.8	51.2
CallHome	34.8	17.6	49.4
WSJ	7.7	3.9	49.4
AMI SDM1	67.6	36.4	46.2
LibriSpeech Other	6.2	5.2	16.1
Average	29.3	12.8	55.2

from Robust Speech Recognition via Large-Scale Weak Supervision

Variability in speakers is one of the main challenges in speech recognition:

- different vocal tract and larynx
- some accents are not well recognize

HMM-GMM can be easily tuned to a specific speaker
(not the case of neural models, overfits too quickly)

Model	med. avp	P ₉₀ avp	top accent	worst accent
SB transformer	22.2	30.6	19.6	28.7
SB crdnn	28.4	40.0	24.5	39.7
W2v base 960-960	23.3	32.5	19.6	30.7
W2v large 960-960	18.2	25.9	14.7	24.0
W2v large 60k-100	18.9	28.6	15.6	26.8
W2v large 60k-960	20.4	28.3	17.9	27.7
RASR small	18.6	29.6	15.7	25.6
RASR small-distill	20.2	28.9	16.9	26.0
RASR big	15.7	23.5	12.3	21.9
Google API video	16.0	23.4	12.9	19.7

Table 3. Median WER on accented rehearsed speech On accented Vox Populi (AVP), the global median WER (med.) is compared to the best median WER by accent and the worst one.

Fitting the GMMs to a new speaker

Given a small speaker set (<1 min of speech)

- **MLLR** (maximum likelihood linear regression):
Learn an affine transformation of the mean of the gaussian mixture to maximize the likelihood on the speaker set (regression loss)
- **fMLLR** (feature space maximum likelihood linear regression):
Same thing but on the MFCCs features

Advantage: big gain in WER

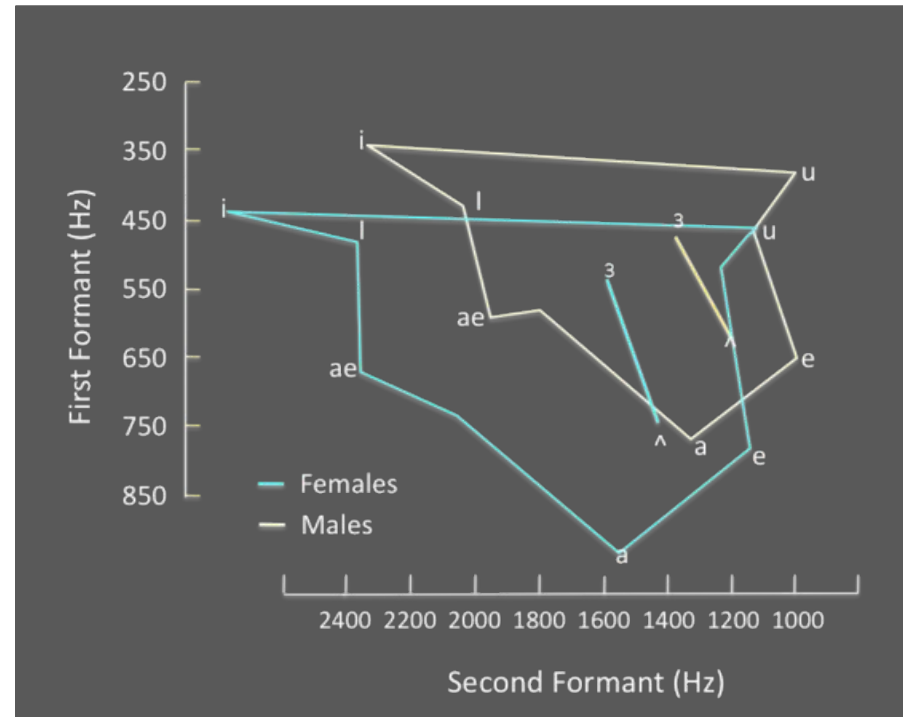
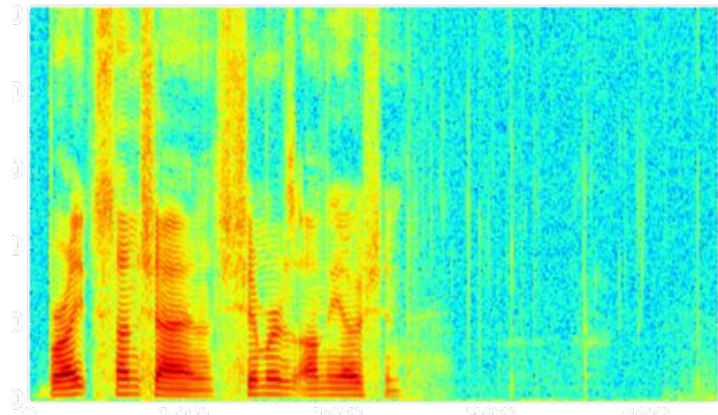
Disadvantage: may overfit the speaker set

- Female voices have a higher pitch than male voices, and children's voices have a higher pitch than female voices
- There are also deformations in the formants (spectrogram patterns that characterize phonemes)

Dealing with variations in gender

Formants:

Main resonances that characterize the phonetic content

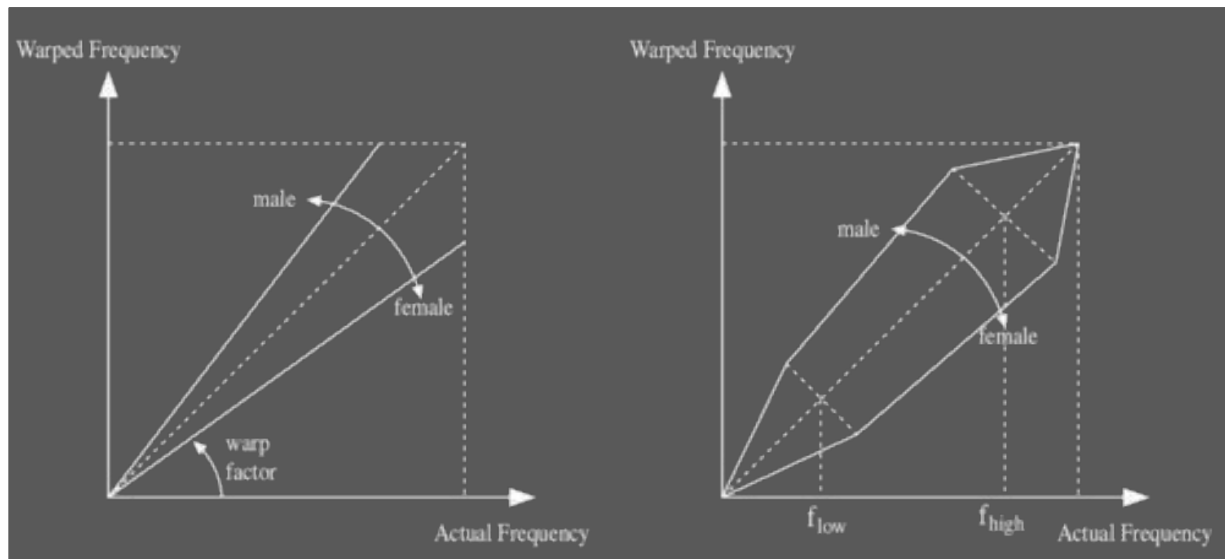


Dealing with variations in gender: Vocal Tract Length Normalization

Shift the spectrum up or down based on the gender

Fit the scaling factor after training to maximize the likelihood

(gain a few WER points)



Linear scaling

Piecewise-linear scaling

Dealing with variations in gender

- The main way of dealing with this variability is to have a balanced dataset
- This way your classifier will not be biased towards males or female voices
- Same solution for the system to work on children's voices

- Speech and speech features
- Modeling speech with Hidden Markov Models
- Modeling speech features with Gaussian Mixture Models
- Handling variability in gender and speaker

Question and quizz